01

IT Architecture Course Book

Book 1 • Foundations, reasoning and worked architectures

Prepared for Clifford Valiente

Vendor neutral concepts with Azure and AWS examples

25 chapters 81 exercises 12 practical labs

A twelve-week study route, four cases and 32 architecture figures

Second edition review edition
October 2026

Contents

How to use this course 3
01 Thinking like an architect 5
02 Business requirements and quality scenarios 7
03 Architecture models and technical communication 9
04 Operating systems networking and the request journey 12
05 Software foundations for architecture work 16
06 Databases and transactional data design 18
07 Data architecture analytics and AI 23
08 APIs and integration contracts 25
09 Identity authentication and authorisation 27
10 Messaging events and reliable handoffs 30
11 Software architecture patterns and boundaries 33
12 Distributed systems performance and resilience 35
13 Cloud architecture with Azure and AWS examples 40
14 Containers orchestration and infrastructure as code 43
15 Security privacy and threat modelling 45
16 DevOps testing and controlled delivery 50
17 Observability service objectives and operations 52
18 Economics procurement and build or buy decisions 55
19 Enterprise architecture governance and migration 57
20 Architecture evaluation and evidence 59
21 Worked architecture customer portal and enterprise integration 62
22 Worked architecture commerce inventory and payments 68
23 Worked architecture a governed analytical platform 72
24 Worked architecture cloud migration and modernisation 76
25 Architect practice and the final capstone 80
Practical labs 82
Exercise answers and assessment guidance 87
Glossary and final review 95
Research study guide 97
References 98

How to use this course

This course connects technical foundations with the decisions an IT architect makes. It is written for a graduate in IT or Information Systems who is comfortable with programming and has studied business information systems. The intended level is upper undergraduate through introductory postgraduate study: explain a mechanism, analyse its assumptions, compare alternatives, design a solution, and evaluate the evidence for it. It is a self-study course, not an accredited qualification or an independently peer-reviewed textbook.

The outcome is a portfolio you can defend. You will translate business needs into measurable scenarios, draw consistent architecture views, define data ownership, analyse concurrency and partial failure, assess security, estimate capacity and cost, and plan a feasible transition. The material refreshes fundamentals while developing professional judgement. Real delivery experience and feedback remain essential to role preparation.

Four cases with different architectural pressures

Meridian Service Portal is the connecting case for Chapters 1–19 and receives a complete design treatment in Chapter 21. Meridian supports equipment used by 600 customer organisations through three internal service business units and 40 service agents. Its customers report problems with registered assets. CRM owns the support case after confirmed handoff; ERP owns asset, contract, inventory, and work-order facts. The portal owns the durable submission receipt and a customer-facing projection of status. A customer organisation is a tenant. Business units are Meridian's internal teams, not synonyms for tenants.

The initial planning workload is 100,000 service requests per year, with an exceptional incident peak of 20 valid submissions per second for ten minutes. Files may be up to 10 MB. Six engineers build and operate the first release with shared identity and security support. The existing CRM remains for two years. The receipt API targets 95 percent of valid submissions within one second, excluding file upload, and 99.9 percent monthly eligible-request success. Normal CRM status propagation targets one minute; a thirty-minute CRM outage must not silently lose accepted requests. These are different promises and need different measures.

Customers may read requests they submitted, with organisation-wide delegation only where policy explicitly permits it. Staff access requires the service-agent role, an assigned business unit, and an authorised customer-organisation scope. Neither a supplied tenant identifier nor a broad role proves access. Service approval can initiate an ERP work order; it does not turn a customer into another kind of person.

The later cases introduce independent pressures. Aster Retail examines orders, scarce inventory, and payment uncertainty. Beacon Analytics examines analytical grain, lineage, historical corrections, and access. Orion Modernisation examines legacy coexistence, data authority, cutover, and rollback. Their assumptions are local to their chapters. None represents a real employer or deployment. All workloads, budgets, numerical datasets, and proposed architectures are fictional teaching examples unless an external source is explicitly identified.

What academic study means here

Research papers support foundational ideas such as information hiding, relational data independence, distributed ordering, consensus, and architecture evaluation. Standards support protocol and security claims. Official implementation documentation supports product-specific behaviour. Practitioner methods support useful working practices, with their scope stated. A vendor's design guidance is not independent evidence of superior business outcomes.

Do not learn only the terminology. For each important proposition, ask: what is the claim, what assumptions make it true, what evidence supports it, what counterexample limits it, and how would it affect this design? Each chapter includes a worked application and at least three exercises; six additional advanced exercises extend the second edition. The deeper case chapters require synthesis across topics. The answer section gives reasoning and permits defensible alternatives.

Read selected original sources as part of the course. The research study guide after the glossary identifies what to inspect and what to critique. You do not need to reproduce the proofs of every paper, but you should be able to explain the boundary of a guarantee. For example, an argument about a replicated log does not automatically establish exactly-once payment processing.

A twelve-week study route

An estimated study day of 4.5 hours comprises 20 minutes of retrieval practice, 70 minutes of reading and annotation, 90 minutes of exercises or experiments, 50 minutes of portfolio work, and 40 minutes of source critique and solution review. Take breaks between blocks. Shift time between activities when a difficult experiment needs it; the timetable is a guide, not a completion guarantee.

The route assumes six study days per week: approximately 324 hours over twelve weeks. It includes source reading, practical work, design revisions, and assessment. Reading the prose alone will take much less time. Use a lighter seventh day for rest or optional review.

Week	Main work	Evidence to retain
1	Chapters 1–4; Lab 1	Problem brief, requirements, context and network views
2	Chapters 5–6; Labs 2–3	Module boundaries, schema, concurrency notes
3	Chapters 7–9	Data model, API contract, identity and access policy
4	Chapters 10–12; Labs 4–5	Handoff state machine, failure experiment, capacity model
5	Chapters 13–15; Lab 6	Cloud mappings, deployment view, threat assessment
6	Chapters 16–17; Lab 7	Release plan, SLO worksheet, operational runbooks
7	Chapters 18–20; Labs 8 and 12	Cost model, transition roadmap, evaluation report
8	Chapter 21	Complete integration case and diagram review
9	Chapter 22; Lab 9	Commerce saga, invariant analysis, compensation tests
10	Chapter 23; Lab 10	Analytical contract, corrected metrics, lineage review
11	Chapter 24; Lab 11	Migration rehearsal, reconciliation, rollback argument
12	Chapter 25 and final assessment	Architecture dossier, presentation, personal development plan

Diagnose before you revise

Without searching, trace a database-backed web request; distinguish authentication from authorisation; sketch customers, assets, and service requests; explain an API timeout after a successful commit; define an availability measurement; and explain why a technically elegant design might be a poor business choice. Record uncertainties. Repeat after weeks six and twelve. Improvement means a more precise explanation and better evidence, not more jargon.

Working files and navigation

Keep your requirement register, editable diagrams, ADRs, experiment notes, cost assumptions, threat model, and case dossiers under version control. The accompanying lab bundle uses fictional data and Python's standard library. The authoring bundle includes the manuscript and editable SVG figures. Examples are teaching models rather than production-ready systems.

The contents page links to chapters and the answer section. Figure captions identify the purpose of each view; surrounding text explains boundaries, arrow meanings, assumptions, and failure implications. IEEE references are numbered by first citation and reused consistently. Sources were accessed on 4 October 2026. Recheck changing implementation details before a real deployment.

Second edition study and evidence route

Read the new advanced sections alongside their host chapters, then attempt exercises numbered .4 and .5 before consulting the advanced answers. Allow an additional estimated 12 to 18 hours per book for the new source reading, experiments and design comparisons. This extends the original study route; it is not an empirically measured completion time. The advanced practice folder contains local teaching programs with stated limits.

This review edition has an accompanying claim-to-source record, change record and external-review package. Full-text verification means checking the passages supporting a specified claim in an accessible

full source; it does not imply that every sentence in every cited publication has been audited. Three enterprise references remain checked only at abstract level and are identified in Book 3 and the evidence record. Actual architect, academic and learner feedback is still pending. The edition must not be represented as independently peer-reviewed.

CHAPTER 1

Thinking like an architect

Learning objectives

Explain the scope of different architect roles, distinguish a decision from a diagram, and connect a proposed design to evidence about business value and technical risk.

Architecture as decisions with consequences

An IT architecture describes the significant structure of a system or organisation and the reasoning behind it. Structure includes responsibilities, interfaces, data ownership, deployment boundaries, trust boundaries, and operational arrangements. The important decisions are often those that are expensive to reverse or that affect several teams.

Consider Meridian's instruction to provide a single service management portal. A developer might implement a service request form. An architect also asks which system is authoritative for the service request, how a person is identified across systems, whether the portal can accept work while the CRM is unavailable, who operates the integration, and how deletion propagates. Each question changes responsibilities and failure behaviour.

Architecture is present even when nobody draws it. If every service can modify a shared database, that is an architectural choice. If the business relies on one person to deploy production changes, that is an operational dependency. The architect's work makes such choices visible, examines alternatives, and helps the team agree on an approach.

A good architectural statement has a reason and a consequence. “Use a queue” is incomplete. “Accept service requests into durable storage and queue the CRM handoff because a temporary CRM outage must not prevent submission; operate a reconciliation process and expose a pending status” is a decision that can be reviewed.

The different roles

A solution architect coordinates the design for a particular business outcome across applications, data, infrastructure, security, and delivery. A software architect concentrates on application structure, module boundaries, programming interfaces, and qualities such as maintainability. A cloud architect focuses on cloud organisation, networking, identity, deployment, and operational design. A data architect concentrates on information models, ownership, movement, quality, and use.

An enterprise architect looks across a portfolio: which business capabilities need support, which applications duplicate one another, how technology standards are governed, and what transition roadmap is affordable. These responsibilities overlap. In a small organisation one person may perform several of them. In a large programme the challenge is to connect their decisions coherently.

The role is not defined by seniority alone or by having access to a diagramming tool. You should be able to explain the relationship between the problem, the design, the expected result, and the evidence that will confirm or challenge your reasoning.

Quality attributes are design inputs

A feature describes something the system does. A quality attribute describes how well or under what conditions it must do it. Search for service requests is a feature. Returning search results within two seconds for a defined workload is a performance requirement. Restricting the results to authorised service agents is a security requirement. Recovering service after a database failure is a reliability concern.

Quality attributes can conflict. Strong isolation between tenants can increase cost. Immediate consistency across remote systems can reduce availability during a network fault. Highly flexible configuration can make testing harder. The architect does not eliminate tradeoffs by naming a fashionable pattern; the architect makes tradeoffs explicit.

The AWS and Azure Well-Architected frameworks provide useful review perspectives. AWS organises its framework around six pillars, including sustainability; Azure uses five pillars. Both address reliability, security, cost, performance, and operational concerns. Use them as prompts for questions, not as proof that a design is correct. [1] [2]

A decision loop

Begin with the outcome and constraints. Identify the riskiest unknowns. Form two or three credible options, including a simple option. Compare them against measurable requirements. Investigate uncertainty with a small prototype, data sample, stakeholder discussion, or recovery test. Record the decision and the conditions that would justify revisiting it.

For Meridian, the unknown may be the CRM API's behaviour under duplicate submissions. Do not start by building an entire integration platform. Inspect the contract and conduct a controlled test with fictional records. A duplicate-safe upsert could simplify the solution. If the CRM only supports non-idempotent creation, the design needs a stronger reconciliation strategy.

Treat architecture as a sequence of decisions at different levels of confidence. Mark an unverified assumption as such. A useful architecture document can say that a dependency's rate limit is not yet confirmed and that the release depends on confirming it. An attractive diagram that conceals this uncertainty is less useful.

Worked example

Meridian considers a custom portal, buying a CRM vendor's portal, and keeping its current forms while improving integration. The business wants fewer duplicate records and less manual follow-up, not a custom interface for its own sake.

The architect discovers that most duplication occurs when a service agent manually copies an approved service case into ERP. A better form would not remove that handoff. The initial design therefore includes an approved service-case event, a stable correlation identifier, and a monitored integration to ERP. A custom portal may still be justified for accessibility or user experience, but that requires separate evidence.

This is a small example of architectural leverage: understanding where the problem arises changes what must be built.

Your first architecture decision record

An architecture decision record, or ADR, captures a significant decision. Michael Nygard's original format includes context, decision, status, and consequences. [3] Add alternatives and evidence where they help your team. Keep one ADR focused on one decision.

For the case, a first ADR might choose the CRM as the authority for service management status while the portal owns the initial submission receipt. Its consequences include mapping status codes, defining a pending state, and reconciling handoffs. It should not claim that two systems are both authoritative for the same status without explaining conflict resolution.

Record a review trigger such as “revisit if the CRM cannot expose stable request identifiers.” A review trigger makes a decision adaptable without making it vague.

Reasoning at university level

Separate a definition, an empirical claim, a formal result, and a recommendation. “A deployment unit contains these modules” defines your model. “This service achieved a certain throughput” reports an observation under a workload. “This algorithm preserves a property under a failure model” states a conditional result. “Choose this design” requires an argument connecting evidence to the user's constraints. They cannot substitute for one another.

For Meridian, the statement that asynchronous handoff permits receipt during a CRM outage is conditional: local durable storage must remain available, backlog capacity must suffice, and the business must accept delayed case creation. The recommendation also depends on operational ownership and reconciliation cost. Challenge these premises before arguing about products. A source can establish a mechanism while leaving the contextual decision to the architect.

Keep a decision ledger with claim, evidence, assumption, confidence, consequence, and revisit condition. Confidence should describe the basis of your belief, not a decorative percentage. “Verified with a duplicate-key integration test in the vendor sandbox” is more informative than “90 percent confident.”

Exercises

1.1 Explain why choosing PostgreSQL is an architectural decision in one project but a routine implementation choice in another.

1.2 Write a five sentence problem brief for Meridian. Include an outcome, a current obstacle, two constraints, and one unknown. Avoid naming a solution.

1.3 A stakeholder requests microservices because the platform must be modern. Give a response that respects the concern and establishes a decision process.

Key points. Architecture connects structure, responsibility, tradeoffs, and evidence. Start with the outcome, identify costly decisions, and investigate the assumptions that could invalidate the design. The next chapter turns this approach into requirements that can be tested.

CHAPTER 2

Business requirements and quality scenarios

Learning objectives

Separate outcomes from features, turn vague expectations into testable scenarios, and maintain traceability between requirements, decisions, and acceptance evidence.

Understand the work before automating it

Begin by asking people to describe a recent instance of the process. Who initiated it? What information was available? Where did work wait? Which exceptions required judgement? A process diagram drawn from an ideal procedure can miss the spreadsheets, email approvals, and manual corrections that make the actual process work.

For Meridian, interview a customer, a service agent, ERP operations, the system owner, security, and the support team. They see different failures. Customers may experience no confirmation, service agents may see duplicate records, and support may have no identifier with which to trace a complaint. These are related but distinct problems.

Describe a business capability as an ability the organisation needs, such as managing service requests or scheduling field maintenance. A process describes how work flows. A software application is one means of supporting the capability. Keeping these concepts distinct helps you avoid assuming that the current application defines the business.

Measure the baseline before promising improvement. If the outcome is shorter time from service approval to a scheduled ERP work order, agree on the start event, end event, and excluded cases. Averages alone may hide a small group of cases that take weeks.

Classify what you hear

A functional requirement describes behaviour, such as recording a service request. A quality requirement describes a measurable property, such as response time at a stated load. A constraint limits the design, such as retaining an existing CRM for two years. An assumption is something currently believed but unverified. A dependency is something another party must provide.

Do not disguise an assumption as a constraint. “We must use a queue because the CRM is slow” contains both a proposed solution and an unmeasured claim. Rewrite it as an observed response-time problem and determine whether a queue, bulk API, better query, or process change addresses the cause.

Every important requirement should have an owner who can clarify its intent and accept the result. The technical team can propose a success measure, but the business must decide whether that measure represents a satisfactory outcome.

Write quality scenarios

Use this structure: under a specified condition, a named actor or event affects a defined part of the system; the system responds in a stated way, and success is measured by a threshold. This forces you to identify the workload, operating mode, and observation point.

“Fast submission” becomes: during a service incident peak of 20 valid submission requests per second, the submission API acknowledges 95 percent of requests within one second, excluding file upload time,

while preserving accepted records. The response boundary matters. Acknowledging a queued request is different from completing every downstream action.

“Highly available” becomes: in a calendar month, at least 99.9 percent of eligible submission requests receive a successful acknowledgement within the agreed timeout. Define eligible requests, measurement location, treatment of client errors, maintenance, and dependency outages. A percentage without a denominator is not a complete requirement.

A service-level indicator, or SLI, is the measurement. A service-level objective, or SLO, is the target for it. A service-level agreement, or SLA, is an agreement with consequences and conditions; it may use different measurement rules. Google's SRE treatment provides a useful starting distinction. [4]

A compact requirement register

ID	Requirement or constraint	Acceptance evidence	Owner
R01	Repeated use of one valid submission key creates one service request	Retry and concurrency test	Product owner
R02	95 percent of API acknowledgements within one second at 20 requests per second	Load report with workload details	Engineering lead
R03	Service agents access only assigned business units	Positive and negative access tests	Business owner
R04	Status updates visible within one minute in normal operation	End to end lag measurement	Service management operations
C01	Existing CRM retained for two years	Approved programme brief	Sponsor
A01	CRM supports stable external correlation identifiers	Contract review and test	Integration owner

The entries are illustrative. R02 does not by itself establish the complete customer experience; file upload, accessibility, and confirmation delivery need their own treatment. A01 should remain an assumption until tested.

Prioritise with consequences

Ask what happens if a requirement is not met. A missing visual preference may be tolerable for a first release. A customer being unable to submit during a critical equipment outage may be unacceptable. A disclosure of another person's service request is a security failure, even if it affects only a few requests.

Separate mandatory constraints from weighted preferences when comparing options. An option that cannot satisfy a mandatory requirement should not win because it is cheap. For the remaining options, use weighted scores as a discussion aid. Explain the score and examine how the result changes if a weight changes.

A score of four out of five is not evidence by itself. Link it to a measured result, documented capability, or explicit expert judgement. Use confidence levels to distinguish tested facts from estimates.

Traceability without bureaucracy

A requirement register becomes useful when it connects to work. R01 might link to an idempotency ADR, the request table's unique constraint, an API contract, a concurrency test, and an operational duplicate-detection report. If the requirement changes, these links show what to revisit.

Keep the chain short enough to maintain. You do not need a separate document for every sentence. A small project can use one table and a few ADRs. A regulated programme may need more formal records. The purpose is to preserve reasoning and evidence as people and systems change.

Worked example

The sponsor asks for real-time reporting. Interviews reveal that managers review service management trends each morning, while service agents need a near-current operational status during calls. These are two use cases.

The design supplies operational status through the service request workflow with a one minute target, and refreshes management analytics on a scheduled pipeline. The analytics screen shows its data timestamp. This avoids paying for streaming infrastructure solely because “real time” sounded desirable.

The next step is to validate the schedules with the users. If the business later adds fraud detection or urgent capacity routing, the reporting requirement may change and a faster pipeline may become justified.

Exercises

2.1 Write a testable requirement to replace “the system must be secure.” Choose one specific risk rather than trying to cover all security.

2.2 A 30 day service month contains 43,200 minutes. Calculate the time allowance corresponding to 99.9 percent availability. Explain why a request-based SLO may produce a different judgement.

2.3 Meridian asks for zero data loss and a five minute recovery time. List the questions required before estimating the solution.

Key points. A requirement should make an important expectation observable. Keep outcomes, constraints, assumptions, and evidence distinct. Requirements are the reference point for the diagrams in the next chapter.

CHAPTER 3

Architecture models and technical communication

Learning objectives

Choose an appropriate view for a question, keep views consistent, and communicate decisions to audiences with different responsibilities.

One system needs several views

A diagram is a selective model. A business stakeholder needs to understand who uses the system and what outcome it supports. A developer needs component responsibilities and interfaces. An operator needs deployment locations, failure domains, dependencies, and recovery paths. Security needs trust boundaries and sensitive data flows.

Trying to put all of this into one diagram makes it difficult to read and easy to misinterpret. Give each view a purpose. State its scope, audience, level of detail, and what the arrows mean. Use names consistently across views.

The C4 model offers a hierarchy of system context, containers, components, and code. In this terminology a container is a separately runnable or deployable application or data store; it does not necessarily mean a Docker container. Use only the levels that answer a real question. [5]

The context view

The context view places the system of interest among people and external systems. Meridian's portal interacts with customers, service agents, a CRM, an identity provider, ERP, and analytics. It does not need subnet names, programming classes, or database indexes.

A useful arrow says “submits service request” or “receives approved work-order request,” not simply “uses.” The label clarifies the direction of responsibility. Two systems exchanging data may need two labelled relationships because each direction has a different purpose.

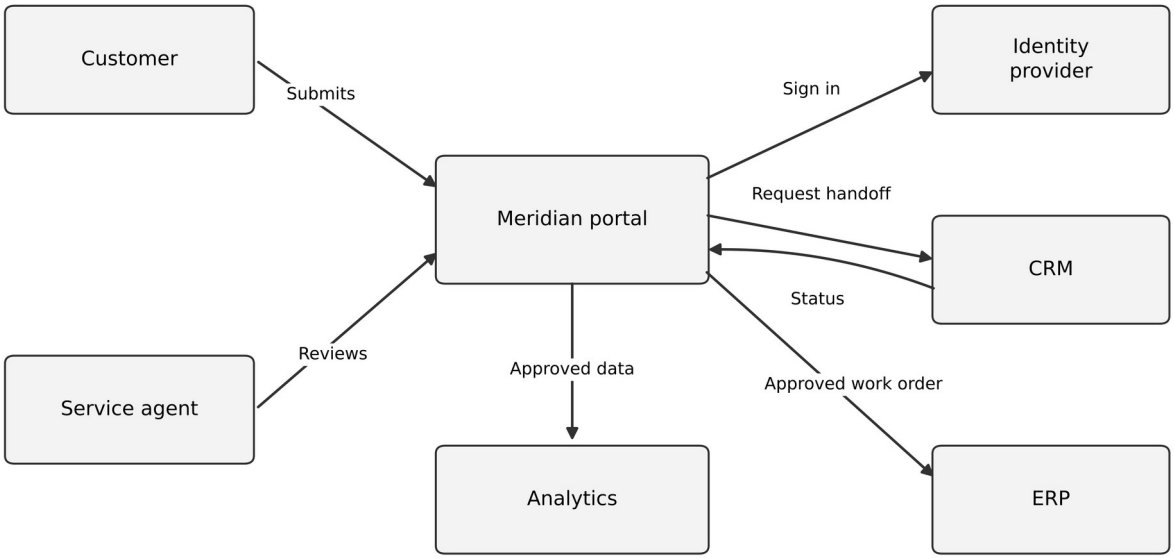


Figure 1 Meridian context view showing people and external system responsibilities

The view also helps establish ownership. The portal team cannot promise to repair the CRM database. It can define a handoff contract, observe failures, and agree an escalation route with the CRM owner.

The container and deployment views

A container view might separate the web interface, application API, background worker, operational database, and object storage. It shows major runtime responsibilities and communication mechanisms. A deployment view shows where instances run and how they are connected, including zones, network boundaries, routing, and external dependencies.

Do not put three application boxes on a diagram and assume the system is resilient. Show whether they share one database, one network route, one credential, or one deployment pipeline. Shared dependencies can defeat apparent redundancy.

A deployment view is also where an abstraction must become sufficiently concrete to test. “Cloud” is too vague to assess a regional outage. You need to identify regions, availability zones where relevant, the failover mechanism, and which components or data are replicated.

Sequence and data flow views

A sequence diagram describes an interaction over time. Use it to reason about retries, asynchronous work, failure paths, and responsibility for responses. An authentication service does not necessarily sit in a serial chain between an API and its database. Sign-in, token validation, authorisation, and business processing are different interactions.

A data flow view asks where information is created, transformed, stored, and disclosed. It is useful for privacy and threat modelling. Label the data, transport, classification, and trust boundaries that matter. A data flow need not show every software method.

UML is a family of modelling notations. For architecture work, a small subset is often sufficient: sequence diagrams for interactions, component diagrams for dependencies, and deployment diagrams for runtime placement. Precision in the model matters more than using every notation feature.

Describe what a diagram cannot show

A diagram should be accompanied by a short explanation. State the governing requirements, responsibilities, failure behaviour, and major open questions. For Meridian, explain that a receipt means the portal has accepted responsibility for durable processing, while “synchronised” means the CRM has confirmed the service request.

Make the legend explicit. A solid arrow could mean a synchronous request and a dashed arrow an asynchronous message, but that convention is only helpful if readers know it. Avoid using colour as the only carrier of meaning. Keep labels readable when printed.

Each diagram should have a stable identifier, such as D02, a revision date, and a link to the relevant decisions. Avoid copying the same view into multiple files and then allowing the copies to diverge.

Worked ADR

Context. Customers must receive a receipt during temporary CRM outages. The CRM remains the service management system of record. The team can operate a database and a worker, but has not yet established a streaming platform.

Decision. The portal will store a submission and its outbound handoff record in one database transaction. A worker will send the handoff to the CRM and record the result. The API will distinguish accepted from synchronised status.

Alternatives. Direct synchronous creation is simpler but makes portal success depend immediately on CRM availability. A distributed transaction is not available across the vendor API. A large streaming platform adds operational work without a demonstrated need at the initial scale.

Consequences. The team must implement duplicate protection, retry limits, a failed-work queue or state, reconciliation, and user-visible pending status. Reporting must account for temporary differences between portal and CRM state.

Review trigger. Revisit if the CRM cannot store the external request key, or if measured throughput and delay exceed the worker design's capacity.

This record contains an argument. A reviewer can dispute its assumptions and propose a test. It is more informative than “we chose event-driven architecture.”

Presenting to stakeholders

For a business audience, begin with the outcome, cost, delivery sequence, and main residual risks. Use the context view and one decision that explains the design. For engineers, show contracts, deployment, data ownership, failure handling, and the evidence still needed.

When challenged, clarify whether the disagreement concerns a requirement, a fact, a preference, or a risk tolerance. These need different resolutions. A benchmark can resolve a performance uncertainty; it cannot decide how much downtime the business accepts.

End a design review with recorded decisions, owners for unanswered questions, and explicit evidence required before release. Do not let a meeting's silence become assumed agreement.

Consistency between architectural views

Treat a diagram set as a model with referential integrity. The portal API in a container view must have a place in the deployment view. A CRM call in a sequence view must cross an interface in the context view. A database table containing tenant data must belong to a data owner and appear in the lifecycle analysis. The same term should denote the same responsibility across views.

For each relationship record source, destination, purpose, protocol, authentication, timeout or acknowledgement rule, and data classification. Some fields are deliberately absent at context level; absence there must not imply absence from the design. A dashed line should have a declared meaning in each figure. None of the original figures in this course claims complete UML conformance; they are scoped architecture views with explicit legends.

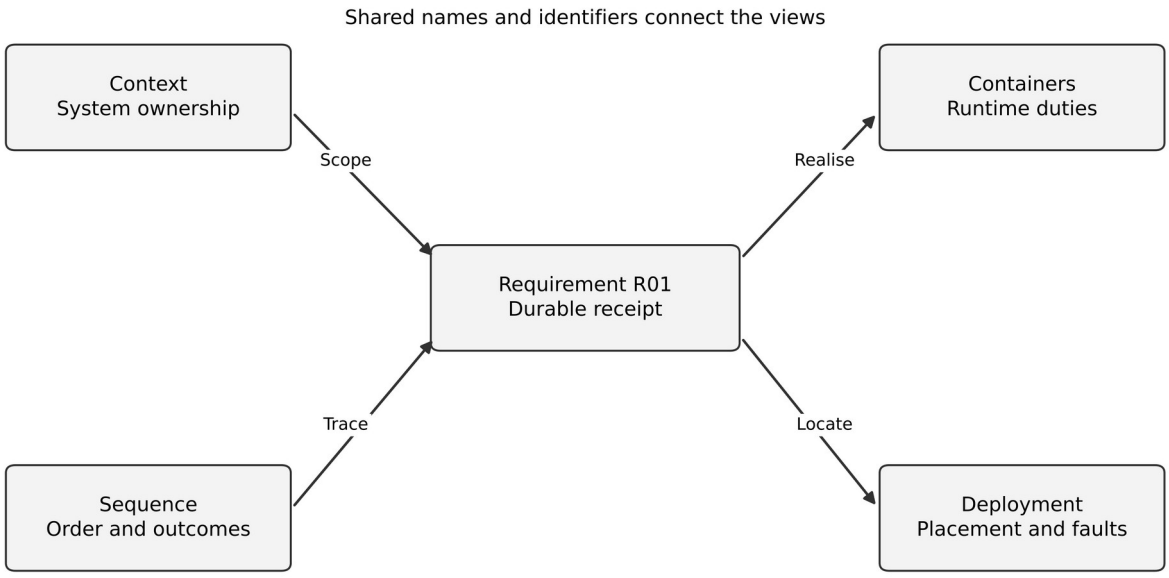


Figure 2 Architecture views answer different questions and meet at shared identifiers

Read the view map from the central requirement to the surrounding views. The model is consistent only if the same protected action can be followed through all of them. A useful review exercise is to pick one request identifier and locate its creation, storage, transfer, failure handling, and authorised display. Chapter 21 demonstrates this for a complete case.

Exercises

3.1 Explain the difference between a C4 container view and a deployment view using the Meridian database.

3.2 Draw a context view with no more than eight elements. Label every relationship with a verb and the information exchanged.

3.3 An architect draws an analytics platform after the operational database in a single request chain. Explain the misunderstanding this may cause and propose a better representation.

Key points. A view should answer one class of questions. Use stable names, labelled relationships, and supporting prose. The next chapter gives you the infrastructure knowledge needed to explain the request paths you draw.

CHAPTER 4

Operating systems networking and the request journey

Learning objectives

Trace a request across layers, recognise common infrastructure failure modes, and reason about network boundaries and capacity without becoming dependent on a particular cloud.

Start with a running process

A program on disk becomes a process when the operating system runs it. The process has memory, execution state, access to files, and operating system resources such as network sockets. Threads share parts of a process's memory; separate processes normally have distinct address spaces. Either model still requires coordination when work touches shared data.

CPU, memory, disk operations, network bandwidth, and connection counts are different resources. Adding CPU does little for a service that spends most of its time waiting for a remote database. Increasing

workers can make matters worse if each worker opens more database connections than the database can sustain.

A virtual machine provides a guest operating system on virtualised hardware. A container packages a process and its dependencies while relying on an operating system kernel. Both need patching and resource management, but at different boundaries. A serverless function still runs on infrastructure; the provider manages more of its provisioning and scheduling.

For each runtime, ask what persists across a restart. An application that writes submitted files to its local temporary directory may lose them during replacement. Durable object storage or another explicitly persistent service is needed when the business expects those files to survive.

From a name to a connection

DNS maps names to records through a distributed hierarchy. A resolver may return a cached answer until its time to live expires. Authoritative name servers hold the records for a zone. An A record provides an IPv4 address, an AAAA record an IPv6 address, and a CNAME points a name to another name. DNS is not the same service as routing or HTTP. RFC 1034 establishes the original DNS concepts; RFC 3596 specifies the AAAA record for IPv6. [6] [7]

This distinction matters during a migration. Changing a DNS record does not guarantee that every client immediately uses the new endpoint. Caches and active connections can outlive the change. Plan a transition period in which both old and new routes behave safely.

An IP address identifies a network interface or endpoint within an addressing scheme. A subnet groups addresses under a prefix. A route determines where a packet goes next. A firewall policy determines which traffic is allowed. A correct route with a blocked port fails differently from an unresolved DNS name, so diagnosis should separate them.

For an IPv4 /24 network, eight bits remain for addresses: 2 to the power of 8 gives 256 total addresses. Conventional subnet and cloud-specific reservations reduce usable host addresses. Do not apply one provider's reservation rules to all networks. Plan address space for growth and connections to other networks; overlapping ranges complicate routing.

Transport and encryption

TCP provides an ordered byte stream with mechanisms for reliable delivery and congestion control. It does not define a service request message or prove that a business transaction completed. UDP provides datagrams without TCP's delivery guarantees. Protocols can build additional behaviour on UDP; HTTP/3 uses QUIC, so it is inaccurate to assume every HTTP connection uses TCP.

TLS protects communication in transit and authenticates endpoints according to its configuration and certificate validation. For a typical website, the client validates a server certificate and negotiates encryption. Mutual TLS also authenticates the client with a certificate. Encryption does not decide whether a signed-in service agent is allowed to read a particular application. [8]

A load balancer may terminate TLS and then establish another connection to the application. Draw these as separate transport segments and decide how the internal segment is protected. A label saying “HTTPS” on the public edge does not describe every downstream connection.

HTTP in an architecture

HTTP defines request and response semantics. A request includes a method and a target, with headers and possibly content. A response includes a status code, headers, and possibly content. GET is intended for retrieval; PUT expresses replacement of a target resource; POST has resource-specific processing semantics. Idempotent method semantics concern the intended effect of repeated identical requests, not identical response bodies. [9]

An API timeout is an observation by the caller. The server may still have committed its work. Retrying a POST can therefore create a duplicate unless the API contract provides duplicate protection. Conversely, an HTTP 200 response is only useful if the contract clearly states what it confirms.

A content delivery network, or CDN, can serve cached public content nearer users. A cache key and cache policy determine which responses are reused. Accidentally caching personalised service request data as public content is a disclosure risk. Static assets and private API responses need deliberately different rules.

A complete submission journey

The customer's device resolves the portal name and establishes a protected connection to the edge. The edge serves the interface and routes an authenticated or otherwise controlled API request to the application. The application validates the input, checks access where applicable, and commits a submission with its handoff record. It returns a receipt. A background worker later sends the handoff to the CRM.

Files may follow a separate upload route into object storage. A safe workflow checks allowed size and type, authorises the upload, verifies completion, and prevents unreviewed files from being treated as trusted content. The database stores a reference and processing state, not necessarily the file bytes themselves.

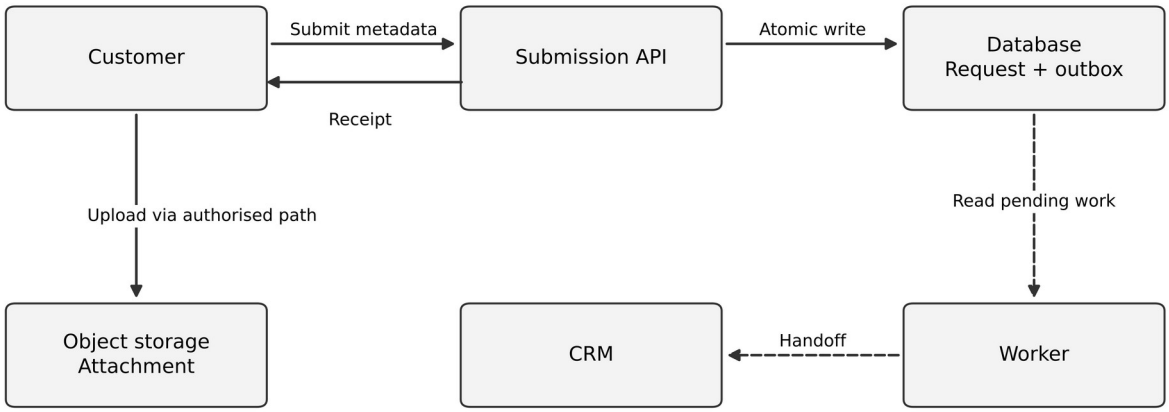


Figure 3 Submission and background processing are separate paths with different completion meanings

The database, identity provider, and analytics system are not simply consecutive stops in one universal chain. The exact dependency graph determines which failures interrupt the user journey.

Private networks and controlled access

A private address space does not automatically make a workload trustworthy. Access may arrive through peering, a VPN, a compromised host, or a misconfigured route. Combine network controls with identity, application authorisation, and monitoring.

An IPsec or TLS based VPN can provide an encrypted connection between endpoints or networks. The broader term VPN also includes arrangements such as provider BGP/MPLS VPNs that do not inherently provide cryptographic confidentiality. Verify the actual technology and protection. [10] A private connection changes the path and exposure but does not remove the need for application security. An API gateway can apply routing and policy at an interface; a load balancer primarily distributes traffic. Products can combine these functions, but the responsibilities remain distinct.

Keep inbound and outbound access explicit. A private application may need outbound access to an identity provider, package repository, or vendor API. An egress restriction can improve control, but it can also break recovery or deployment if required dependencies were never identified.

A disciplined diagnosis

When Meridian's portal fails, check in layers. Does the name resolve to the expected destination? Can the client reach the address and port? Does TLS validation succeed? Does the HTTP request reach the application? Does the application have a usable database connection? Which dependency consumed the timeout budget?

Do not change several variables at once. Record the symptom, hypothesis, test, and result. A successful ping does not prove that an HTTPS API works; a blocked ping does not prove that it fails. Test the protocol and path that the application actually uses.

End-to-end correctness and network design

Saltzer, Reed, and Clark argue that some functions require knowledge at the communicating endpoints for complete correctness; lower layers may still provide useful performance or reliability support. This is a placement argument, not a rule to remove all lower-layer controls. [11] In the course design, a TCP acknowledgement proves neither a database commit nor a correctly authorised business action. The portal receipt contract must therefore state its own durable boundary.

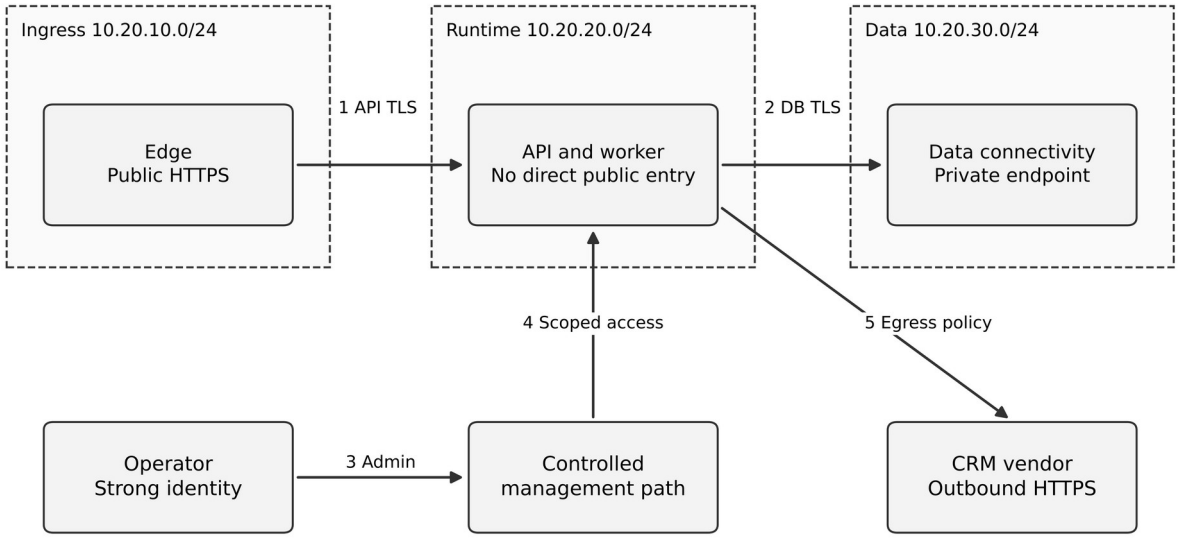


Figure 4 Example network boundaries with separate ingress egress and administrative routes

The network figure is a neutral deployment sketch, not a provider configuration. Only the edge accepts public application traffic. The runtime initiates database connections and controlled outbound vendor calls. Operators enter through a separately authenticated management path. Prefixes describe address allocation; numbered arrows describe permitted flows. A /24 route does not itself grant access, and a firewall rule does not create a route. Private service endpoints, name resolution, and return paths need concrete validation in the selected cloud.

For the proposed three subnets, reserve 10.20.10.0/24 for ingress components, 10.20.20.0/24 for runtime instances, and 10.20.30.0/24 for suitable private data connectivity. In some managed platforms these responsibilities are implemented without placing the service itself in your subnet. Preserve the responsibility map while adapting the deployment mechanism.

Exercises

4.1 An API has healthy CPU but long response times and a growing database connection wait queue. Explain why adding application instances may worsen the problem.

4.2 Design three non-overlapping /24 subnets inside 10.20.0.0/16. State the purpose of each and distinguish address planning from firewall policy.

4.3 The DNS record was changed five minutes ago, but some clients still use the old endpoint. Give two plausible causes and a migration design that tolerates them.

Key points. Understand processes, resources, naming, routing, transport, encryption, and application semantics as separate layers. The next chapter brings the same separation of responsibility into software.

CHAPTER 5

Software foundations for architecture work

Learning objectives

Use programming knowledge to assess boundaries, dependencies, tests, and failure behaviour, and explain how code structure influences architectural change.

Read for responsibility

When reviewing code, first locate the entry point, business rules, data access, and external calls. Follow one user request through these responsibilities. Then identify where errors are translated and where transactions begin and end.

A common design problem is a single function that validates input, calculates policy, writes to a database, calls a vendor API, and sends email. It may be short, but its responsibilities change for different reasons. A vendor outage can affect unrelated business logic, and a unit test may require several live services.

Separate stable business rules from volatile adapters. For Meridian, determining whether an asset is eligible for service is a domain rule. Parsing a vendor's status code is an integration responsibility. Storing a submission is a persistence responsibility. The boundaries should support clear tests and controlled change.

Cohesion and coupling

Cohesion describes how strongly the responsibilities within a module belong together. Coupling describes dependencies between modules. A highly cohesive application module handles a coherent business concept; low coupling allows another module to change without forcing widespread edits.

These are design goals, not numerical laws. Splitting every class into a separate service may lower one kind of code dependency while increasing network, deployment, and operational coupling. A well-structured monolith can have cleaner boundaries than a collection of tightly dependent services.

Depend on small interfaces that express the needs of the caller. A service management workflow might require “store submission” and “publish handoff,” rather than exposing every method of a database client. The interface should not conceal important guarantees: callers need to know whether an operation is atomic, retryable, or eventually consistent.

A small example

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Submission:
    customer_id: str
    asset_id: str
    description: str

def validate_submission(item: Submission) -> None:
    if not item.customer_id or not item.asset_id:
        raise ValueError("Customer and asset are required")
    if not 1 <= len(item.description) <= 2000:
        raise ValueError("Description must contain 1 to 2000 characters")
```

This function is deliberately narrow. It checks a structural rule, not whether an authenticated user owns the customer record or whether the asset is currently eligible for service. Those checks need trusted context and possibly database state. Clear naming helps prevent a developer from treating one validation function as complete authorisation.

A frozen data object discourages accidental mutation of this value, but it does not make every nested object immutable or provide security. Avoid attributing guarantees to a language feature that it does not actually provide.

Object orientation and alternatives

Encapsulation groups state and behaviour behind an interface. Polymorphism allows callers to use different implementations through an agreed contract. Inheritance can share behaviour, but deep

hierarchies can make changes difficult to understand. Composition often provides a simpler way to assemble capabilities.

Functional techniques emphasise explicit inputs and outputs and minimise hidden mutable state. A pure calculation function is easy to test and reason about. Real systems still need effects such as database writes and network calls. One useful design is to keep calculations pure and place effects at controlled boundaries.

An architect need not insist on one programming style across every team. The important questions are whether the code expresses the domain, whether dependencies are controlled, and whether the team can safely change and operate it.

Errors are part of the contract

Distinguish invalid input, an unauthorised operation, a conflicting state, a temporary dependency failure, and an internal defect. They require different responses. Repeating an invalid request will not make it valid. Retrying a temporary connection failure might succeed, but the caller must consider whether the first attempt already took effect.

Preserve enough error context to diagnose a problem without exposing secrets or personal data. Return a stable error code and a correlation identifier to the client. Keep detailed internal diagnostics in access-controlled logs.

Cancellation and timeouts also need design. A cancelled HTTP connection does not necessarily cancel an in-progress database operation. Use request deadlines, bounded work, and durable workflow state rather than assuming that a disconnected caller undoes the business action.

Tests as architectural evidence

A unit test checks a narrow behaviour with controlled dependencies. An integration test checks interactions with a real dependency or representative implementation. A contract test checks whether a provider and consumer agree on an interface. An end to end test checks a complete journey.

Each reveals different defects. Unit tests can prove a calculation for selected inputs, but cannot prove that a production network route or identity policy works. End to end tests provide broader evidence but are usually slower and harder to diagnose. Use a combination based on risk.

Test difficult cases explicitly: duplicate submission keys, unauthorised access to another tenant, a database commit followed by a lost response, and a CRM outage. These tests reveal architectural behaviour more clearly than another successful-path demonstration.

Version control and change

Version control records changes and supports collaboration and recovery. Git stores project history through commits; branches provide lines of development that can later be integrated. [12] Keep meaningful commits small enough to review, and explain why a change is needed.

Version more than application code. API specifications, deployment definitions, schema migrations, ADRs, and runbooks all affect the system. Keep generated secrets and private datasets out of the repository. A secret that has been committed should be treated as exposed and rotated; deleting the latest copy does not erase the history or every clone.

Worked design review

A pull request adds a call to the ERP API inside the database transaction that accepts a service request. The happy path works, but the external call may take thirty seconds. During that time, locks and connections remain occupied.

A better design records the local state and a durable intent to perform the ERP operation, commits promptly, and processes the external action separately when the business permits delayed completion. If the action must be synchronous, specify a short deadline and a clear compensation or reconciliation process. The right answer depends on the business invariant; asynchronous processing is not an automatic solution.

Information hiding as a decomposition criterion

Parnas compares decompositions based on processing steps with modules that hide difficult or changeable design decisions. His argument makes the hidden decision, rather than the flowchart step, a

useful basis for a module boundary. [13] This does not mean every module must become a separately deployed service.

Apply the idea to Meridian's CRM adapter. Hide the vendor's field names, pagination details, and error representations behind an interface that returns a stable internal outcome: confirmed, rejected, or unknown. Expose uncertainty rather than pretending every exception means rejection. The domain workflow should not need editing when the vendor renames a JSON field, but it must be revisited if the vendor removes an idempotency guarantee.

Evaluate the boundary with two hypothetical changes: replacing the CRM vendor and changing service eligibility policy. Count which modules must change and which tests provide confidence. This change-impact analysis is more useful than claiming that an interface automatically produces low coupling. If eligibility rules are buried in vendor mapping code, the supposed adapter boundary is misleading.

Exercises

5.1 Identify which checks in a submission workflow can be pure functions and which require trusted state or external effects.

5.2 Design four tests for a duplicate-safe submission function. Include one concurrency or lost-response case.

5.3 A team proposes one shared “common” module for all business rules. Explain the potential benefit and the change risks.

Key points. Code structure should reveal responsibilities and make important behaviour testable. Boundaries, contracts, errors, and change history matter to architecture even when you are not writing most of the production code.

CHAPTER 6

Databases and transactional data design

Learning objectives

Design a relational model, choose constraints and indexes deliberately, and reason about concurrent updates, transactions, and storage alternatives.

Begin with business facts

A data model expresses what the organisation considers true. A customer can submit several service requests; an asset can be associated with many requests over its lifetime. Each request identifies one reporting customer, one asset, and a workflow state. Storing all of these facts in one repeated spreadsheet row creates duplication and update problems.

Distinguish entities, attributes, relationships, identifiers, and lifecycle. A person's email address is an attribute that may change, and it is not always a safe global identifier. Use a stable internal identifier and manage external identifiers explicitly.

Normalisation reduces avoidable duplication and the anomalies it creates. If a department's name is repeated on every service request, a rename can leave inconsistent values. A separate department relation can make the dependency explicit. Deliberate denormalisation may help reporting or performance, but then the design needs a refresh and consistency strategy.

Keys and constraints

A primary key identifies a row. A unique constraint enforces uniqueness for a candidate key. A foreign key enforces a relationship to an existing referenced row. Check constraints restrict allowed values. Nullability states whether absence is permitted. PostgreSQL's constraints documentation explains the database mechanisms and their limits. [14]

In a shared database for multiple tenants, include tenant identity in the relevant keys and relationships. A foreign key on service_request.asset_id alone may establish that an asset exists while failing to establish that it belongs to the same tenant.

```
CREATE TABLE asset (
    tenant_id TEXT NOT NULL,
    asset_id TEXT NOT NULL,
    title TEXT NOT NULL,
    PRIMARY KEY (tenant_id, asset_id)
);

CREATE TABLE service_request (
    tenant_id TEXT NOT NULL,
    request_id TEXT NOT NULL,
    asset_id TEXT NOT NULL,
    customer_id TEXT NOT NULL,
    submission_key TEXT NOT NULL,
    status TEXT NOT NULL
        CHECK (status IN ('accepted', 'synchronised', 'failed')),
    PRIMARY KEY (tenant_id, request_id),
    UNIQUE (tenant_id, submission_key),
    FOREIGN KEY (tenant_id, asset_id)
        REFERENCES asset (tenant_id, asset_id)
);
```

In this small schema, status denotes receipt/handoff delivery state. A full design stores the CRM business state separately, as explained in Chapter 10. The schema protects part of the model. It does not authenticate a caller, check who may use a submission key, or prevent every invalid workflow transition. The application must bind requests to a trusted principal and enforce business permissions. For a real idempotency design, also record the request's relevant payload hash and owner so that reuse with different content can be rejected.

Transactions and invariants

A transaction groups operations into an atomic unit: either the committed unit takes effect or its uncommitted work is rolled back according to the database's guarantees. Atomicity is only useful when the transaction includes the operations that protect the invariant.

For Meridian, the invariant might be that every accepted service request has a corresponding durable handoff intent. Insert the request and the outbox row in one transaction. Sending to a remote API after the commit is a separate action, because the external vendor usually cannot participate in the same transaction.

The familiar ACID properties are atomicity, consistency, isolation, and durability. Database consistency here concerns preserving defined constraints and valid states; it should not be confused with the consistency models of distributed replicas discussed later.

Isolation determines what concurrent transactions can observe. PostgreSQL's default Read Committed level takes a new snapshot for each statement. Serializable isolation aims for results equivalent to some serial execution, and applications must handle transactions that are aborted to preserve that guarantee. Isolation details differ across products. [15]

A concurrency problem

Suppose two workers read a request with status accepted. Both decide to send it to the CRM, and both create a remote record. A “check then act” sequence did not make the claim exclusive.

Possible designs include a transactional claim with a lease, a lock where supported, or a conditional update that succeeds for only one worker. None removes the lost-response problem at the remote system. You still need idempotent remote operations or a reconciliation lookup using a stable external identifier.

An optimistic update can include a version:

```
UPDATE request_state
SET status = 'approved', version = version + 1
WHERE request_id = :id
  AND version = :expected_version;
```

If zero rows are updated, the caller must determine whether the record was missing or changed. This is illustrative SQL for a separate request_state table; it is not a migration for the earlier schema. The point is that the expected prior version becomes part of the write condition.

Indexes and query shape

An index provides an access structure that can reduce the work required to find rows. It consumes space and must be maintained during writes. Adding an index to every column is therefore not a general optimisation strategy.

Start from queries and workload. If service agents often list pending service requests within one tenant ordered by creation time, an index beginning with tenant_id and status may be useful, with the rest chosen to support filtering and ordering. Measure with representative data, including skew. One tenant with most of the rows can behave differently from an even distribution.

Use an execution plan to see how the database expects to obtain rows. In PostgreSQL, EXPLAIN describes a plan, while EXPLAIN ANALYZE executes the statement and reports observed behaviour. Be careful with statements that modify data. [16] Estimates, cache state, statistics, and production concurrency all affect interpretation.

Choosing a storage model

Relational databases are a strong fit when relationships, constraints, and multi-record transactions matter. Document stores can fit aggregate-shaped data with flexible fields. Key-value stores can support simple access by key. Graph databases emphasise traversing relationships. Column-oriented analytical systems support different workloads from transactional row stores.

Choose from access patterns and guarantees, not from whether a product is called SQL or NoSQL. A product may support several models, and a NoSQL database may support transactions within stated boundaries. Read the exact contract.

Use object storage for large files when appropriate and keep controlled references in the operational database. Plan what happens if file upload succeeds but metadata creation fails, or vice versa. A staged state and cleanup process are often necessary.

Backup and recovery

Replication copies changes for availability or distribution. It can also copy an accidental deletion. A backup provides a recoverable historical copy, with its own integrity, access, and retention requirements. Test restoration into an isolated environment.

A recovery point objective, or RPO, describes the acceptable data loss measured back in time. A recovery time objective, or RTO, describes the acceptable restoration time. A backup every night does not meet a five minute RPO by itself, and a successful backup job does not prove a two hour restore.

Relational reasoning beyond a table sketch

Codd's relational model addresses data independence: the logical representation should avoid forcing application logic to depend on physical ordering, indexing, or access paths. [17] The lesson for architecture is to distinguish business facts from the mechanisms used to retrieve them. A new index should not change the meaning of a request.

Consider R(tenant, asset, asset_type, contract, contract_expiry, request, submitted_at). In this hypothetical domain, (tenant, asset) determines asset_type and contract; (tenant, contract) determines contract_expiry; and (tenant, request) determines asset and submitted_at. These are declared business dependencies, not patterns inferred safely from a small sample. Repeating contract_expiry on each request introduces an update anomaly: one extension requires several consistent changes.

Decompose into Asset, Contract, and ServiceRequest relations with explicit keys. Reconstructing the original facts must not invent combinations: the join needs the complete tenant-qualified key. A decomposition that joins only on contract number is unsafe if tenants reuse numbers. Normalisation does not require every historical value to point to the latest master record. A contractual snapshot taken at approval is a different fact from the contract's current state; model the time and meaning explicitly.

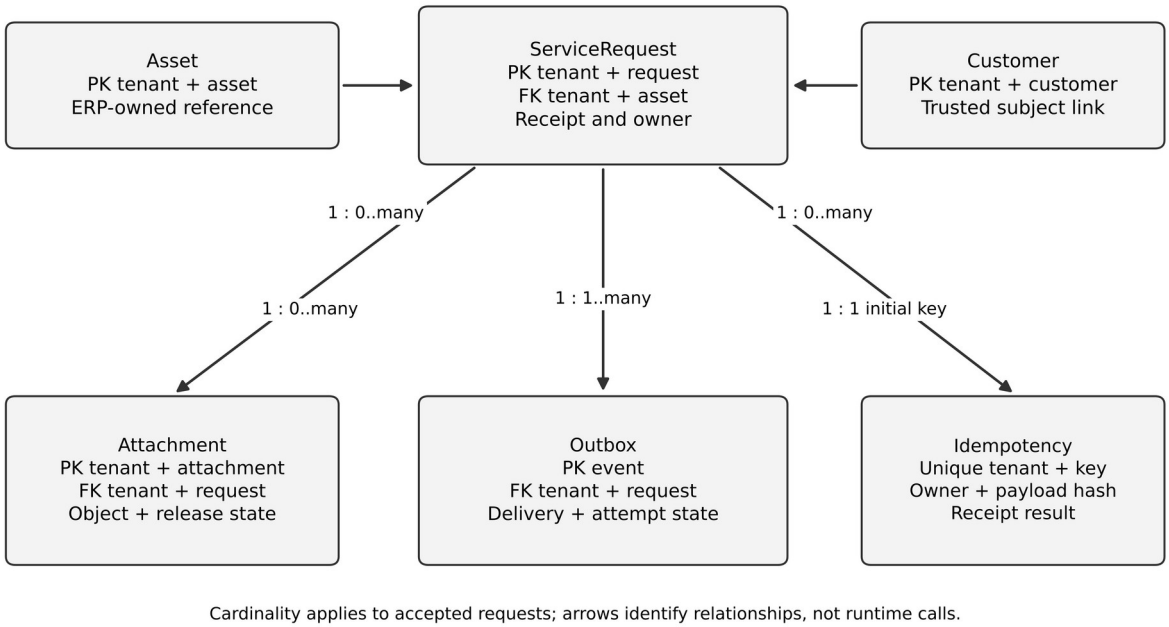


Figure 5 Tenant-qualified relationships and independent delivery state in the portal schema

Read 1 and 0..many as cardinalities, not flow direction. One asset can have zero or many service requests; each request names one asset. Each accepted request has an initial handoff intent, while later messages may add further outbox rows. Attachment metadata references a request and a separately controlled object. The figure extends the small SQL example: production design also needs principal scope, payload hashes, timestamps, attempt history, and lifecycle policy.

When an invariant spans two rows, ask what isolation prevents concurrent writers from violating it. Checking a condition in application code is insufficient if another transaction can invalidate it before commit. Use a suitable atomic update, constraint, locking strategy, or serializable transaction with retry as the database contract permits. Explain which invariant the mechanism protects; do not name an isolation level as a substitute for the argument.

A schedule that exposes write skew

Consider an original staffing example with two rows, Ada on call and Ben on call. The invariant is that at least one engineer remains on call. A transaction may remove its engineer only after observing the other engineer on call. This is a predicate over the pair of rows; separate constraints saying that each field is either true or false cannot enforce it.

Suppose both transactions read the same initial snapshot. Transaction A observes Ben on call and sets Ada off call. Transaction B observes Ada on call and sets Ben off call. The updates affect different rows, so a rule detecting only competing writes to the same row need not reject either transaction. The final state has nobody on call. Each decision was reasonable in its snapshot; the combined outcome violates the business rule. PostgreSQL's description of Repeatable Read and Serializable distinguishes the anomalies and retry requirements relevant to this example. [15]

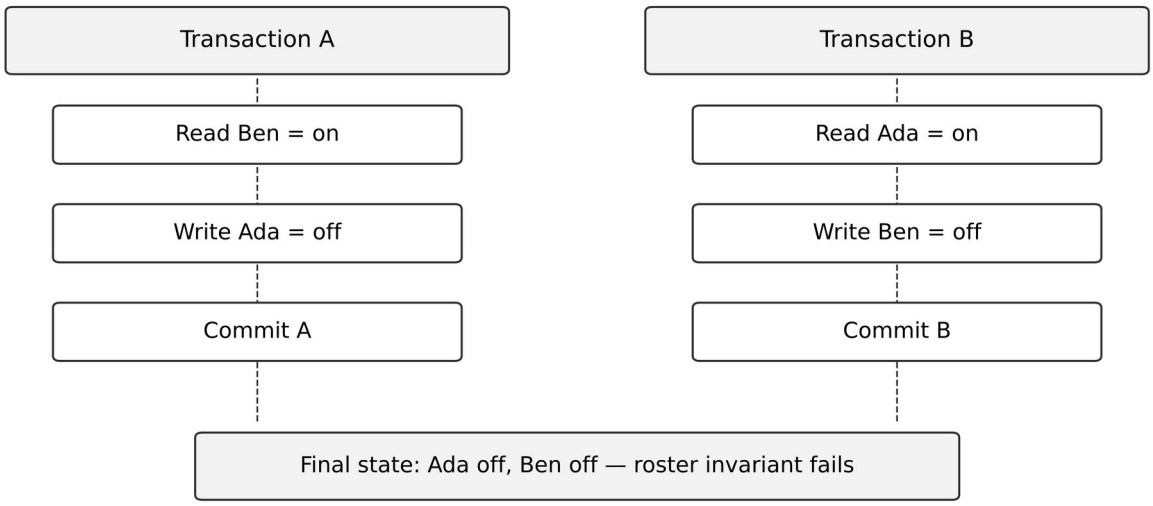


Figure 6 Two transactions preserve a rule locally while their combined writes violate it

The figure shows a possible interleaving, not a claim that every database implements its isolation levels identically. Read it from top to bottom: both reads precede either final commit. The important dependency is each transaction's decision about a row changed by the other. Moving both fields into one row may simplify this particular invariant, but is a modelling choice with its own contention and ownership consequences.

Three alternatives can be defended. First, use a serializable transaction and retry the whole transaction after a serialization failure. Its decision must be recomputed; resubmitting only the final update would reuse an invalid premise. Second, lock a common roster record before reading either engineer, and require every writer of that roster to follow the same protocol. This introduces deliberate contention around the invariant. Third, redesign the command as a single guarded operation on a representation that directly maintains the required count. This can be efficient, but correctness still depends on all assignment and removal paths maintaining that representation.

The test must include two sessions and a controlled barrier after the reads. A serial execution cannot reveal this anomaly. Record the isolation level, the transactions that commit or abort, and the final invariant. A retry loop needs a limit and an outcome visible to the caller. It must not repeat an external payment or email merely because a database transaction was retried. Keep externally visible actions after a durable intent boundary and give them their own recovery contract.

Now change the requirement: each geographical region must retain one qualified engineer, and some engineers cover several regions. A single global counter is no longer an adequate representation. This illustrates why architects start from the predicate and its ownership before selecting a locking strategy. A performance optimisation that silently weakens the predicate changes the business contract.

Exercises

6.1 Explain how a composite tenant and asset foreign key prevents a defect that an asset-only relationship might allow.

6.2 Two service agents approve the same record based on version 7. Show what happens when both use the optimistic update pattern and one commits first.

6.3 A dashboard runs large aggregations on the transactional database and slows submissions. Compare three responses, including their disadvantages.

6.4 Explain why two independent boolean constraints cannot prevent the on-call write-skew schedule. Compare a common lock with a serializable retry.

Key points. Model business facts, protect invariants with constraints and transactions, and measure query behaviour. Storage decisions must include concurrency, lifecycle, recovery, and the operational workload.

CHAPTER 7

Data architecture analytics and AI

Learning objectives

Separate operational and analytical data needs, design trustworthy pipelines, and evaluate AI features as systems with data, quality, and operational requirements.

Operational facts and analytical questions

An operational system supports an action now: submit a service request, approve a service case, or change an asset. An analytical system supports questions across time and across many records: where does the process slow down, which channels produce completed service requests, and how does workload change by business unit?

These workloads often require different models. Transactional design avoids inconsistent updates. Analytical design often organises measurements around a declared grain and dimensions. The grain states exactly what one row represents. A row per service request is different from a row per request status change or a row per daily snapshot.

If you join a request table to five status events and then sum a request-level amount, the amount may be counted five times. Declaring the grain before selecting dimensions and measures helps prevent this mistake. Kimball's discussion of fact tables distinguishes transaction, periodic snapshot, and accumulating snapshot approaches. [18]

A small dimensional model

For Meridian, a service request fact might contain a request identifier, submission date key, asset key, channel key, and a count of one. A status-event fact might contain one row for each status transition, its event time, and its processing time.

Dimensions describe the context, such as asset, department, channel, or date. Historical changes need an explicit policy. If a department is renamed, should last year's report show the old name or the current name? Slowly changing dimension techniques provide different answers. The correct choice depends on the question being asked.

Do not include every operational attribute in the analytical model. Customer contact details may be unnecessary for aggregate service analysis. Reducing data reduces both exposure and the work needed to manage it.

Warehouses lakes and lakehouses

A warehouse emphasises governed, structured analytical queries and curated data. A data lake stores data in object storage, often in multiple forms, with processing and governance layered on top. A lakehouse combines lake-style storage with table management and analytical capabilities. Product boundaries vary; these terms do not replace an evaluation of consistency, access controls, performance, metadata, and cost.

Choose the smallest arrangement that supports the workload. A small organisation may need a managed analytical database and scheduled transformations rather than a broad platform. A large organisation with diverse data and machine learning workloads may benefit from more flexible storage and processing.

Raw storage is not a governance strategy. Without ownership, cataloguing, quality checks, retention, and access control, a lake can become a collection of files whose meaning nobody can establish.

ETL ELT and change capture

Extract, transform, load, or ETL, changes data before loading it into the target. Extract, load, transform, or ELT, loads first and performs transformations in the target environment. Each has implications for access, cost, lineage, and where sensitive raw data exists.

A batch pipeline processes a bounded set on a schedule. A streaming pipeline continuously handles arriving records. Change data capture, or CDC, obtains changes from a source database or another change mechanism. A CDC record describes a source change, which is not always the same thing as a meaningful business event.

Handle late arrival, duplication, deletion, and schema change explicitly. Store both event time and ingestion time when they answer different questions. A watermark or checkpoint says how far processing has progressed; it does not guarantee that no earlier event will ever arrive.

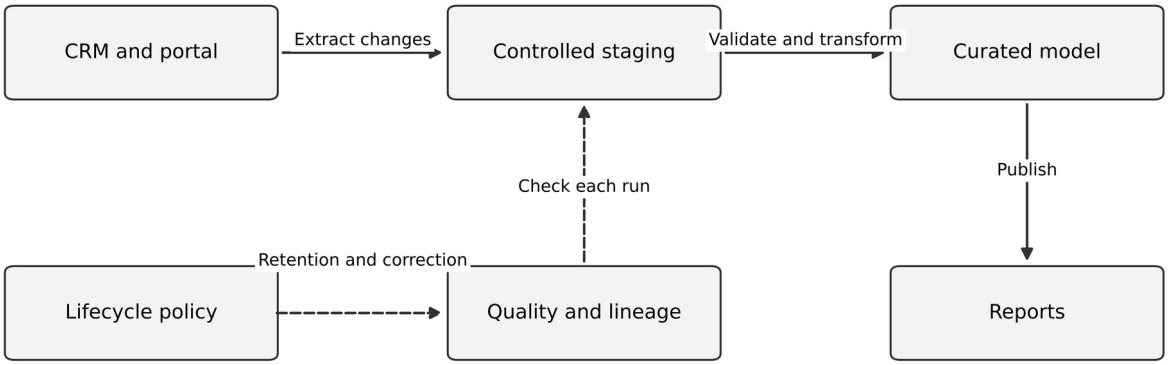


Figure 7 Operational records become governed analytical data through an explicit pipeline

Reconciliation and quality

A pipeline should be able to explain its results. Compare record counts, key coverage, sums where meaningful, and exceptions between source and target. A count match alone is weak evidence because missing and duplicate records can cancel each other out.

Define data quality rules that correspond to business meaning. A submission date must be present. A status event should reference a known service request. A completion time earlier than the submission time needs investigation. Classify invalid records and decide whether to stop the pipeline, quarantine them, or publish with a visible limitation.

Track lineage: which source, transformation version, and processing run produced a value? This matters when a manager challenges a report or a privacy request requires locating copies.

Master data and ownership

Master data describes shared business entities such as customers, assets, suppliers, and service contracts. Multiple systems may carry copies, but one clear process should govern key attributes and identifiers. The authoritative system can differ by attribute; ERP may own asset and contract status while an identity system owns credential state.

Create a mapping between internal and external identifiers. Avoid treating an email address as a universal join key. Mergers, contact changes, account transfers, and changed addresses can make that assumption fail.

Data governance assigns decision rights and responsibilities. A data owner determines appropriate business use; a steward helps maintain definitions and quality; technical custodians operate storage and controls. The roles may be combined in a small team, but the responsibilities still exist.

AI as an architectural feature

Suppose Meridian proposes an AI assistant that summarises service requests for service agents. The architecture needs more than a model endpoint. It needs authorised data retrieval, input limits, handling of untrusted document content, evaluation criteria, a human review step, logging rules, and a fallback when the model is unavailable or wrong.

Retrieval-augmented generation retrieves relevant information and supplies it to a model. Retrieval can improve grounding, but it does not guarantee factual correctness or enforce access control by itself. Apply permissions before retrieval and when presenting results. A vector index is another data store that must respect updates, deletion, and tenant boundaries.

Define quality against the actual task. For a summariser, assess omission of important facts, unsupported statements, and whether a reviewer can trace statements to source passages. For a decision-support system, assess how errors affect people and whether staff can challenge or override the output.

NIST's AI Risk Management Framework provides a voluntary structure for governing, mapping, measuring, and managing AI risks. [19] Use it to organise responsibility and evaluation. For a real deployment, assess the intended use, affected people, contractual duties, and applicable law. This course uses the assistant only to draft summaries for human review, not to autonomously deny service.

Worked example

A dashboard reports that one department takes twice as long to process service requests. Investigation shows that its CRM emits the final status only in a nightly batch, while another department's system emits it immediately. The measured delay combines business processing and data delivery.

The solution is to distinguish business event time from ingestion time, document the limitations of each source, and validate the metric with operations staff. Faster infrastructure would not correct the definition.

Exercises

7.1 Declare the grain for a table used to measure time spent in each request status. Explain why one row per customer is insufficient.

7.2 Design four quality checks for a CRM-to-warehouse pipeline, including one that could detect duplicated records even when total row counts appear correct.

7.3 An AI summary service proposes retaining all prompts forever to improve quality. Identify the architectural questions and propose a bounded alternative.

Key points. A trustworthy data platform has clear definitions, ownership, lifecycle, and reconciliation. AI introduces another set of dependencies and evaluation needs; it does not remove the need for disciplined data architecture.

CHAPTER 8

APIs and integration contracts

Learning objectives

Design an API around a business contract, handle retries and evolution safely, and recognise where interface documentation needs operational guarantees.

An API is an agreement

An application programming interface defines how a consumer requests or receives behaviour from a provider. The contract includes syntax, meaning, permissions, errors, timing, versioning, and operational limits. A JSON schema describes structure; it does not by itself explain what acceptance means.

For Meridian, POST /service-requests might accept a submission and return a receipt before the CRM has processed it. State this clearly. If a consumer assumes that the response means the CRM has synchronised, it may trigger work order creation too early.

REST-style APIs organise interaction around resources and HTTP semantics. RPC-style APIs emphasise operations. GraphQL provides a query interface in which clients select fields. None is automatically superior. Evaluate consumer needs, caching, complexity, authorisation, query cost, and support.

Design the resource and response

A submission resource should have a stable identifier, a state, and links or documented endpoints for subsequent actions. Keep internal database details out of the contract unless they are intentionally part of it.

```
{
  "request_id": "req-204",
  "status": "accepted",
  "status_url": "/service-requests/req-204"
}
```

A 202 Accepted response is appropriate when processing has been accepted but is not complete. A 201 Created response can be appropriate when the requested resource has been created, even if other work follows. Choose based on the resource contract and explain the distinction. Do not select a status code solely because the implementation uses a queue.

Use consistent error responses containing a stable code, a safe explanation, and a request identifier. Do not expose stack traces, credentials, or details about another tenant.

Idempotency as a business guarantee

Idempotency means that repeating an operation has the same intended effect as performing it once. For a submission API, use a client-supplied key within a defined scope and retention window. Persist the key, owner, relevant request hash, and result atomically with the action.

On a repeated request, return the prior result when the owner and payload match. Reject conflicting reuse. A random key is not an access credential; the service still authenticates and authorises the caller. Concurrency requires a database constraint or another atomic mechanism, not only an in-memory dictionary.

Define what happens after the key expires. A delayed retry can be dangerous if the server has forgotten the original result. Keep business identifiers and reconciliation methods that make long-delayed duplicates detectable.

Validation and compatibility

Validate syntax, types, ranges, allowed states, and permissions at the service boundary. Treat every external input as untrusted, including responses from an internal-looking integration. A successful schema validation does not establish that a transition is allowed.

Prefer compatible evolution. Adding an optional field is often easier than removing or changing a field, but even additive changes can break strict consumers. An enum with a new value can fail a client that assumes it has seen all possible values. Define consumer behaviour for unknown values.

Version a contract when a breaking change cannot be avoided. Give consumers a migration path, measure use of old versions, and agree an end date. Maintaining versions indefinitely has security and support costs.

OpenAPI provides a machine-readable description for HTTP APIs. Use it for request and response schemas, authentication declarations, examples, and tooling. Keep narrative guarantees such as idempotency scope, ordering, and asynchronous completion explicit because a schema alone cannot express all of them. [20]

Pagination and controlled consumption

A list endpoint must define order and pagination. Offset pagination is simple but can repeat or skip records when the underlying collection changes. Cursor pagination can provide a more stable continuation when built around an appropriate sort key and consistency model.

Limit page size, payload size, and expensive queries. Rate limits protect service capacity but need useful consumer behaviour: a documented response, retry guidance where appropriate, and a way to distinguish temporary throttling from invalid requests. An unbounded export endpoint can become an accidental denial of service.

Use bounded timeouts for every external call. Allocate an end to end deadline across dependencies. Repeating a request three times with a ten second timeout does not fit inside a two second user response target.

Webhooks

A webhook delivers an event to a consumer's HTTP endpoint. The consumer should verify authenticity, guard against replay where relevant, validate the payload, and acknowledge according to the delivery contract. The provider may retry, so the consumer should deduplicate.

Accepting the webhook and finishing all business processing are different actions. Often the endpoint records the event durably and processes it later. Provide a failed-event review path; silently dropping invalid messages makes reconciliation difficult.

Do not place secrets in query strings or log complete sensitive payloads. Signature verification must use the exact bytes and algorithm expected by the provider, with a controlled key rotation process.

Gateways and integration ownership

An API gateway may handle routing, policy, throttling, and some authentication checks. Business authorisation still belongs in a layer that understands the requested operation and resource. A gateway cannot infer whether a service agent is assigned to the relevant business unit and customer organisation unless the design supplies and verifies that context.

Assign an owner to each contract. Record provider, consumers, data classification, expected volume, failure behaviour, change policy, and escalation route. The code can be correct while the organisational contract is missing.

Worked contract review

Meridian's CRM integration sends a submission, times out, then retries. The CRM creates a second record. The vendor confirms that an external_reference field can be searched but is not unique.

The architecture cannot claim exact duplicate prevention from that field alone. Possible mitigations include a controlled single-writer process with reconciliation and manual duplicate handling, or negotiating a vendor capability that enforces uniqueness. Record the residual race risk. Do not promise a stronger guarantee than the provider offers.

REST as constraints rather than a file format

Fielding's dissertation derives REST through constraints including client-server separation, stateless interaction, caching, a uniform interface, layering, and optional code on demand. The uniform interface includes resource identification, representations, self-descriptive messages, and hypermedia-driven interaction. [21] JSON over HTTP alone does not establish conformance. The examples here use resource-oriented HTTP contracts; they do not claim that every example implements full REST.

For a concrete review, ask whether the response tells a consumer which state transitions are currently permitted and whether cache behaviour is explicit. A request in a completed state should not advertise an unsupported update. Conversely, a server-side browser session can be a practical identity choice even when an application is not a strict example of REST's stateless constraint. Assess the actual architecture rather than awarding a style label.

Exercises

8.1 Explain the difference between syntactic compatibility and semantic compatibility using a request status field.

8.2 Specify an idempotency contract for POST /service-requests, including scope, payload conflicts, concurrency, retention, and authorisation.

8.3 An endpoint returns every request for a tenant in one response. Identify performance, reliability, and security concerns and propose a safer interface.

Key points. A usable API contract describes effects and limits as well as fields. Retries, evolution, pagination, and ownership determine whether integrations remain reliable as systems change.

CHAPTER 9

Identity authentication and authorisation

Learning objectives

Distinguish identity concepts, explain a modern sign-in flow, and design access checks that protect individual resources and tenant boundaries.

Identity is a lifecycle

An identity represents a person, service, device, or other principal. Authentication establishes evidence about who or what the principal is. Authorisation decides which action that principal may perform on which resource under current conditions.

A user account also has a lifecycle: creation, verification, role assignment, role changes, suspension, and removal. Meridian's service agent who transfers to another business unit may need different permissions immediately. A valid login does not imply that old permissions remain appropriate.

Separate workforce identities, customer identities, and workload identities when they have different owners and lifecycle requirements. Reusing one directory is a product choice; reusing one permission model for all three is usually a design mistake.

OAuth and OpenID Connect

OAuth 2.0 provides a framework for delegated access to protected resources. OpenID Connect adds an identity layer and an ID token that communicates authentication information to the client. An access token is intended for a resource server; an ID token is intended for the client relying on the sign-in result. Do not use them interchangeably. [22]

A common browser application flow uses an authorisation code with PKCE. The client starts an authorisation request, the identity provider authenticates the user, and the browser returns a short-lived code to the registered redirect URI. The client exchanges the code with the required verification data and obtains tokens. Use a maintained library rather than implementing the protocol yourself.

Current OAuth security guidance recommends code-based flows with appropriate protections and rejects the resource owner password credentials grant. Apply exact redirect URI handling, PKCE, and token audience restrictions as required by the protocol and deployment. [23]

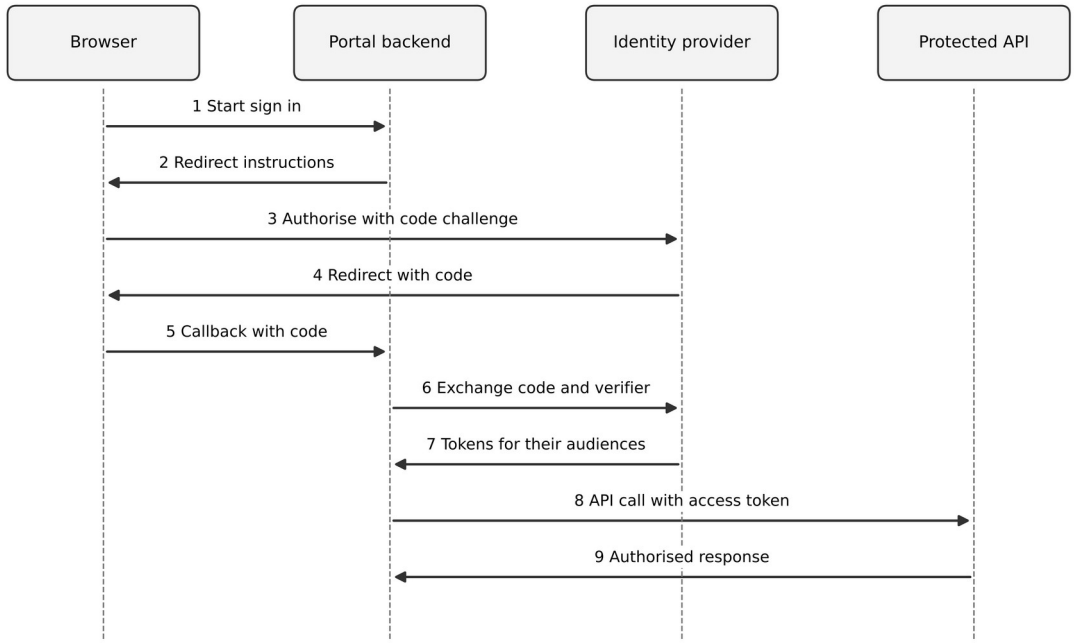


Figure 8 Sign in and resource access involve different messages and token audiences

Token validation

A resource server must validate a token according to the issuer's supported contract. For a signed JWT access token, this normally includes signature verification using trusted keys, an allowed algorithm, expected issuer, intended audience, validity times, and relevant permissions. Merely decoding the JSON proves nothing.

An opaque access token may require introspection or another provider-supported validation mechanism. Do not assume every access token is a JWT. Cache decisions only within bounds that preserve the security and revocation requirements.

Key rotation is normal. Support new trusted keys through a controlled discovery mechanism and handle unavailable metadata carefully. Never accept an arbitrary key URL supplied by an untrusted token as authoritative.

Sessions and browsers

A server-side session can keep tokens out of browser-accessible storage and present the browser with a secure session cookie. Cookie settings such as Secure, HttpOnly, and appropriate SameSite behaviour

reduce some risks, but they do not replace cross-site request forgery protection or protection against script injection.

If tokens are held in the browser, understand the consequences of malicious script execution. Architecture should reduce exposure and keep lifetimes and privileges appropriate. The correct pattern depends on the client type and identity provider capabilities.

Logout, token expiry, and account suspension are related but different. Ending a browser session may not invalidate every already issued token. State the expected access revocation time and design around the provider's actual mechanisms.

Roles attributes and resource checks

Role-based access control, or RBAC, assigns permissions through roles. It is useful for broad job responsibilities such as service agent or administrator. Attribute-based rules consider properties of the principal, resource, action, and context, such as assigned business unit and request ownership.

For Meridian, a service agent role is necessary but insufficient. The service must also determine whether this service agent can access this request's business unit. Apply the check on every relevant operation, including exports, attachments, search, and background jobs.

Never trust a tenant_id supplied in the request body as proof of tenant membership. Derive the allowed scope from trusted identity and server-side policy. Use data access controls that make cross-tenant mistakes difficult, and test denial paths explicitly.

OWASP's API Security Top 10 includes broken object-level authorisation as a major API risk. [24] The architectural lesson is to bind access decisions to the actual object being accessed rather than checking only whether a user has logged in.

Workload identity and secrets

A service also needs an identity when calling storage, a queue, or another API. Prefer short-lived credentials and platform-supported workload identity where available. A long-lived shared secret copied into several deployments makes rotation and attribution harder.

Grant the service only the required actions on the required resources. The submission API may need to write request records without permission to manage production infrastructure. The worker may need to send a handoff without permission to read every analytics dataset.

Keep secrets in a suitable secret management service, protect access to it, and plan rotation and emergency revocation. Encryption of a secret file does not help if the decryption key is stored beside it with identical access.

Worked access policy

A customer may read a request they submitted and upload a permitted attachment while the service request is open. A service agent may read service requests for assigned business units and authorised customer organisations and record an approved status transition. ERP operations may receive only the data needed after a service case has been approved for fulfilment. Support may see diagnostic identifiers but not full request content by default.

This policy is easier to review than a vague “admin versus user” split. Translate it into positive and negative tests and record who can approve exceptions.

Derive the access decision from the resource

The security question is not merely whether a token is authentic. Meridian must establish which principal is making which request against which resource under which current policy. Treat tenant and business-unit identifiers received in a URL as requested scope. Obtain the resource's actual tenant and unit from an authoritative lookup, and compare these with independently established entitlements. Otherwise a caller can select the authority under which its own request is evaluated.

For an original example, principal P belongs to tenant T1 and has permission to read requests that P submitted. Request R belongs to T1 but was submitted by Q. A valid token for P and equality of tenant identifiers are insufficient: the ownership predicate is false. If an authorised delegation grants P organisation-wide access, the same request may become permitted. Staff policy is separate: an agent role, assigned service unit and customer scope must all match. Do not convert a customer delegation into a staff permission.

Check the resource again when processing an asynchronous command if the action needs current authority at execution time. A policy may instead permit execution based on a valid approval captured when the command was accepted; that is a distinct business rule. Record the approving identity, decision, policy version, action and resource. An ordinary access token copied into a queue is a poor substitute for a deliberate delegation contract, especially when its lifetime is shorter than the backlog.

RFC 9700 requires PKCE for public clients using the authorization code grant and recommends it for confidential clients. It also recommends limiting token privileges and resource audiences. [23] These protocol protections do not establish the resource ownership predicate above. A token intended for the analytics API should not be accepted by the service-request API just because both trust the same issuer. Use established libraries and a defined token profile, including trusted issuer and key configuration, audience, lifetime and applicable claims; decoding a token is not validation.

A test plan should distinguish invalid identity evidence, valid identity without permission, wrong resource audience, a correct tenant with the wrong owner, and access removed after the token was issued. Include allowed cases too: a policy that denies every request passes many negative tests while failing the service. The architecture review should ask which component enforces each predicate and how a route added later obtains the same enforcement.

Exercises

9.1 Why is a correctly signed ID token not automatically a suitable credential for an API?

9.2 Write an authorisation rule for downloading a request attachment. Include identity, resource, action, and lifecycle conditions.

9.3 A service agent loses access in the directory but can still call the API for another hour. Explain possible causes and design choices.

9.4 A valid tenant T1 token requests another T1 user's attachment. Specify the missing authorisation predicate and two tests that distinguish it from tenant-only enforcement.

Key points. Authentication supplies identity evidence; authorisation governs specific actions. Token validation, resource checks, lifecycle, and workload identity must work together.

CHAPTER 10

Messaging events and reliable handoffs

Learning objectives

Choose between synchronous calls, queues, and event streams, and design retries and reconciliation without overstating delivery guarantees.

Commands events and queries

A command asks a receiver to do something, such as create an ERP work order. An event reports something that happened, such as a service case being approved. A query asks for information. Names should reflect these meanings because consumers make different assumptions about them.

A synchronous call is useful when the caller needs an immediate result and can tolerate the dependency's response time and availability. Asynchronous messaging separates acceptance from later processing. It can absorb bursts and temporary outages, but introduces delayed state, operational queues, and more complex user communication.

Do not introduce messaging solely to make a diagram look decoupled. The business must tolerate the delay, and someone must own failed work.

Queue and stream semantics

A work queue distributes tasks to consumers, often with one consumer completing each logical task. An event stream retains ordered records within defined partitions so consumers can read and replay according to their own positions. Products can support several behaviours, so read the actual semantics.

RabbitMQ distinguishes publisher confirms from consumer acknowledgements. The first relates to the publisher's interaction with the broker; the second relates to the broker's interaction with a consumer. Neither automatically proves that an external business side effect completed exactly once. [25]

Kafka provides partitioned logs and supports transactional processing within stated boundaries. Ordering is normally meaningful within a partition, not across every partition. Its documentation explicitly warns that exactly-once claims require careful interpretation, especially when processing involves external systems. [26]

Delivery and side effects

At-most-once delivery avoids redelivery by accepting the possibility of lost work. At-least-once delivery allows repeats so that unacknowledged work can be attempted again, subject to retention, durability, and retry limits. Exactly-once processing is a scoped property of a specific system and transaction boundary.

Suppose a worker sends an email and crashes before acknowledging the queue message. Redelivery can send the email again. A broker cannot generally know whether the external email action happened. Use a provider-supported idempotency key where possible, a durable operation record, or an explicit tolerance and reconciliation strategy.

A deduplication table also needs the right transaction boundary. Recording “processed” before the business update can lose work after a crash. Recording it after the update can duplicate work unless both writes commit atomically.

The transactional outbox

The outbox pattern stores a business change and an outgoing event or command in the same local database transaction. A separate publisher reads the outbox and delivers it. This prevents the local dual-write problem in which the business change commits but the intent to notify is lost. The publisher can still send duplicates, so consumers need an appropriate duplicate-handling design. [27]

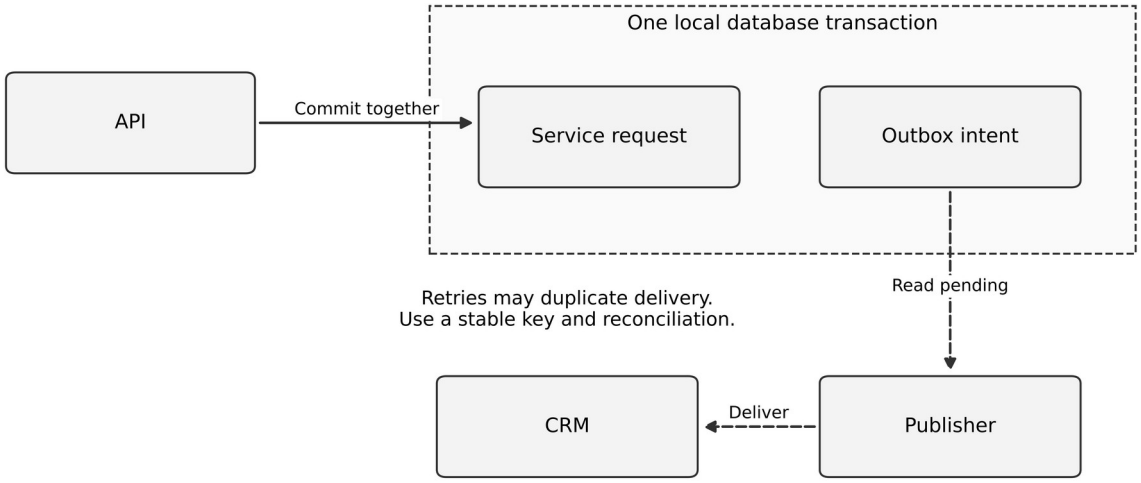


Figure 9 One local transaction stores the request and handoff intent while delivery remains retryable

For Meridian, the outbox row contains an event identifier, request identifier, event type, schema version, creation time, and enough controlled information to perform the handoff. Avoid copying the complete request into every message when a smaller payload or authorised retrieval is sufficient.

An inbox or processed-event table can deduplicate incoming messages. Use a unique event identifier and commit the inbox record with the consumer's local state change. If the consumer then calls another external system, the same external side-effect problem appears again and needs its own design.

Retry policy

Retry only failures that may be transient. Use bounded attempts, backoff, and jitter to avoid many clients retrying in synchrony. Apply an overall deadline where the business action has an expiry. A rejected schema or an unauthorised request will not become valid after waiting.

Move repeatedly failing work into a visible state or dead-letter queue. Preserve the event identifier, error category, retry history, and safe diagnostic context. Define who reviews it, when it can be replayed, and how replay avoids duplicate side effects.

Monitor age as well as count. A queue with ten old critical messages can be more serious than a queue with a thousand recent messages during a planned burst.

Ordering and state transitions

If status events arrive out of order, a later “accepted” event could overwrite an already processed “withdrawn” state. Use a domain version, ordered partition key, or transition rules that recognise stale events. Event time alone may be insufficient because clocks differ and events can be delayed.

A partition key that preserves order per service request can be useful. A single key for an entire large tenant can create a hot partition and limit parallelism. Choose the smallest ordering boundary the business actually needs.

Sagas and compensation

A saga coordinates a business process across local transactions. If a later step fails, compensation performs another business action to address the earlier one. It is not a magical database rollback. A sent email cannot be unsent, and a paid fee may require a refund with its own rules.

For field service, reserve a service slot, create an ERP work order, and arrange parts after approval. If parts cannot be reserved, the workflow may remain pending for manual handling rather than deleting every earlier record. Model the states and the person responsible for exceptions.

Separate business state from delivery state

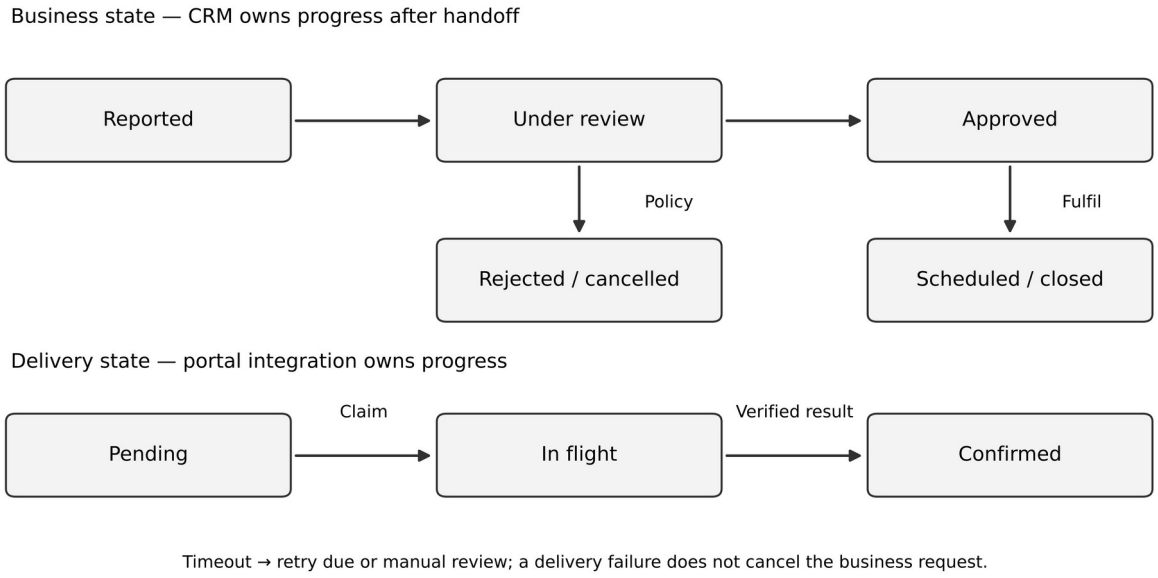


Figure 10 Business progress and integration delivery are two distinct state machines

The upper state machine belongs to service operations: reported, under review, approved, scheduled, and closed, with rejection or cancellation where policy allows. The lower belongs to integration: pending, in flight, confirmed, retry due, and manual review. Approval does not mean that ERP received a work order. A failed delivery attempt does not mean that a customer withdrew the request. Store the two dimensions separately so one cannot accidentally overwrite the other.

A transition has a source state, triggering event, guard, durable effect, and authorised actor. For example, pending to in flight requires an atomic lease claim; a stale lease holder must not overwrite a newer attempt's result. Confirmation should retain the vendor reference and the identity of the command. A timeout enters uncertainty or retry handling according to the remote contract. It must not erase the durable intent.

For each transition, test a repeated event, an unexpected predecessor, and a restart between the external action and local acknowledgement. This is a small, practical form of state-space reasoning. It exposes more defects than testing each endpoint once.

Exercises

10.1 A worker updates its database, then crashes before acknowledging a message. Explain the risk and the role of an inbox table.

10.2 Design a retry and failed-work policy for the CRM handoff, including the conditions for manual review.

10.3 An event broker advertises exactly-once processing. What must you investigate before promising exactly one ERP record?

Key points. Reliable integration requires clear completion meanings, local atomicity, duplicate handling, and recovery from uncertain outcomes. The next chapter uses these mechanisms to compare larger architectural styles.

CHAPTER 11

Software architecture patterns and boundaries

Learning objectives

Compare major architectural styles, choose boundaries from business and operational needs, and explain why a pattern is suitable under stated conditions.

Patterns describe recurring arrangements

An architectural pattern describes a recurring organisation of responsibilities and interactions. A style can help a team reason about a design, but naming a style does not establish its quality. “Microservices” says little about whether services are secure, coherent, observable, or independently deployable.

Client-server separates a client that requests work from a server that supplies it. Layered architecture groups responsibilities such as presentation, application logic, domain logic, and persistence. These can exist inside a monolith or within each service.

A monolith is deployed as one main application unit. A modular monolith keeps clear internal boundaries while retaining that deployment simplicity. Service-oriented architecture organises capabilities behind service interfaces, often across a broader enterprise. Microservices typically emphasise smaller, independently deployable services with clear ownership. Boundaries and operational independence matter more than an arbitrary line count.

Start from the domain

A domain is an area of business activity and meaning. Within a bounded context, a model and its terms are consistent. These terms and the anti-corruption layer pattern come from domain-driven design. [28] “Service case” in CRM and “work order” in ERP can refer to related work while having different lifecycle rules, attributes, and responsibilities.

Do not force every system into one universal model. Define the relationship and translation between contexts. An anti-corruption layer protects one context from being shaped entirely by another system's terminology or legacy interface.

For Meridian, service management status belongs to the CRM context, while scheduling and fulfilment belong to ERP operations. A service approval event can connect them without making the CRM responsible for inventory and technician scheduling.

The modular monolith

A modular monolith can separate service request submission, asset management, attachments, and integration workflows inside one deployable application. Modules communicate through defined interfaces and own their data access. The team can test most behaviour without a distributed environment.

The advantages include simpler deployment, easier local transactions, and fewer network failures. The risks include accidental cross-module access, a broad deployment impact, and resource contention. Enforce boundaries with code organisation, interface reviews, tests, and explicit database access rules.

For six engineers and a moderate initial workload, this can be a strong starting point. Extract a service when a demonstrated need for independent scaling, ownership, security isolation, or release cadence outweighs the additional operations.

Microservices and distributed coupling

A service boundary introduces a network boundary, data consistency questions, authentication between services, independent deployment, and monitoring requirements. A request that synchronously calls six services inherits many dependency and latency risks.

Independent deployment requires compatible contracts and data ownership. If every release requires coordinated changes to five services, the system is operationally coupled even if its code lives in separate repositories. A shared database that any service may modify can create similar coupling.

Choose a service boundary around a cohesive capability with a clear owner and a meaningful reason to change independently. Do not split an application into separate services for individual CRUD operations simply because the endpoints are small.

Event driven architecture

Event-driven architecture uses events as a major means of coordination. It can support independently evolving consumers and processing at different rates. The cost is that the complete workflow can be harder to see, state becomes temporarily inconsistent, and event schemas require governance.

Choreography allows participants to react to events without one central workflow controller. Orchestration gives a coordinator explicit responsibility for the sequence and state. Choreography can reduce central coupling but hide process logic across many consumers. Orchestration can make a long workflow easier to understand but creates an important coordinating component.

Use an explicit process model when the business requires auditability of a multi-step outcome. An event history alone does not always tell a support worker what action to take next.

Serverless and ports and adapters

Serverless services shift more runtime management to the provider. Functions can suit short event-triggered tasks and variable demand. They still have execution limits, startup behaviour, concurrency controls, observability needs, and downstream dependencies. A sudden increase in function concurrency can overwhelm a database.

Ports and adapters, sometimes called hexagonal architecture, separates core behaviour from external mechanisms through interfaces. The business logic depends on a port, while an adapter implements database or vendor communication. It is compatible with several deployment styles. Cockburn describes the pattern through an inside-outside boundary and ports that admit different adapters. [29]

The abstraction is useful when it protects a real source of change or enables meaningful tests. An elaborate interface around every trivial function can make the code harder to follow.

A comparison for Meridian

Option	Main benefit	Main cost or risk	Evidence that would justify it
Modular monolith and worker	Simple delivery and local transactions	Shared deployment impact	Small team and coherent release needs
Several services	Independent ownership and scaling	Distributed failures and operations	Measured differences in scaling or team autonomy
Event driven workflow	Tolerates delayed processing and new consumers	Event governance and reconciliation	Several consumers and acceptable eventual completion
Managed functions	Low runtime management for bounded tasks	Limits and downstream pressure	Bursty short tasks and tested operating limits
Vendor portal with integration	Less custom product ownership	Vendor constraints and lock-in	Required experience and controls supported

The options can be combined. A modular portal can publish events, and a managed function can perform one integration. Avoid treating these labels as mutually exclusive product packages.

Technical debt and evolutionary design

Technical debt is a future cost created by design or implementation choices that make change harder. Some debt is deliberate, such as a temporary adapter needed for a deadline. Other debt comes from weak understanding or neglected maintenance.

Record the consequence, owner, and trigger for repayment. “Refactor later” is not a plan. A temporary polling integration might be acceptable if it meets the current freshness target and has a review when volume crosses a measured threshold.

Build an architecture fitness check around an important rule. For example, a dependency test can prevent the reporting module from importing internal submission persistence code. A contract test can detect a breaking API change. These checks make design intent enforceable.

Exercises

11.1 Explain how a modular monolith can have lower coupling than a poorly designed microservice system.

11.2 Choose a starting style for Meridian and write an ADR with two rejected alternatives and a measurable review trigger.

11.3 A team proposes a service per database table. Evaluate the proposal using transaction boundaries, ownership, and release independence.

Key points. Choose patterns to satisfy requirements under real team and operating constraints. A simpler initial system with explicit boundaries can evolve more safely than a distributed system without clear ownership.

CHAPTER 12

Distributed systems performance and resilience

Learning objectives

Reason about partial failure and consistency, estimate capacity with explicit assumptions, and design resilience mechanisms that preserve business meaning.

Partial failure changes the problem

Inside one process, a function normally returns a result or raises an error in a shared execution environment. Across a network, a timeout leaves uncertainty: the remote action may have failed, succeeded, or still be running. Messages can be delayed, duplicated, reordered, or lost within the failure model.

Design state so that uncertainty is visible. A service request can be accepted locally while its CRM status is pending. A timeout should not automatically change that state to “not submitted.” Reconciliation can later establish the remote outcome.

Clocks are also imperfect. Timestamps from different systems can differ even when each system is functioning normally. Use monotonic clocks for measuring local elapsed time where available, and use domain versions or explicit ordering mechanisms when correctness depends on sequence.

Consistency models

Strong consistency can make a read reflect the latest completed write within the specified system contract. Eventual consistency allows replicas or views to converge after updates stop and communication succeeds. Read-your-writes is a useful session guarantee that ensures a user can observe their own completed changes.

Choose per operation. A customer may need immediate confirmation of their own submission, while aggregate analytics can lag. A balance or inventory invariant may need a stronger coordination boundary than a notification counter.

The CAP result concerns what a distributed system can guarantee during a network partition: under its formal definitions, linearizable consistency and availability for every request cannot both be maintained across the partition. It is not a rule to choose any two marketing labels in normal operation. Latency, durability, and operational recovery require additional reasoning. [30]

Latency throughput and concurrency

Latency is the time one operation takes. Throughput is completed operations per unit time. Concurrency is work in progress. A system can have high throughput and poor latency when it processes many slow operations simultaneously.

For a stable system, Little's Law relates averages: average work in progress equals arrival rate multiplied by average time in the system. If 20 requests arrive per second and average time is 0.4 seconds, about eight requests are in progress on average. This is not a peak sizing rule. Burstiness and tail latency require headroom and measurement. [31]

Suppose a worker completes five handoffs per second while ten arrive per second. The backlog grows by five per second, or 18,000 in an hour. A larger queue provides time, not additional processing capacity. Increase capacity, reduce work, control intake, or change the service target.

Tail latency

The 95th percentile is the value at or below which 95 percent of observations fall under the stated measurement method. Averages can hide a minority of very slow requests. Tail latency often matters to interactive users and can worsen when a request fans out to many dependencies.

Do not add p95 values and claim the result is an end to end p95. The relationship depends on correlation and the distribution of the combined work. Measure the complete journey and use component data to explain it.

Separate queue waiting time from service time. A fast worker can still produce long end to end delay if work waits in a queue. Observe both the age of the oldest relevant message and processing duration.

Resilience mechanisms

A timeout bounds waiting. A retry gives a transient failure another chance. A circuit breaker stops calls temporarily when a dependency is failing and later permits controlled probes. A bulkhead limits how much one failure can consume shared resources. Microsoft documents the circuit breaker as a way to avoid repeatedly calling a failing dependency; it complements rather than replaces timeouts and appropriate retries. [32]

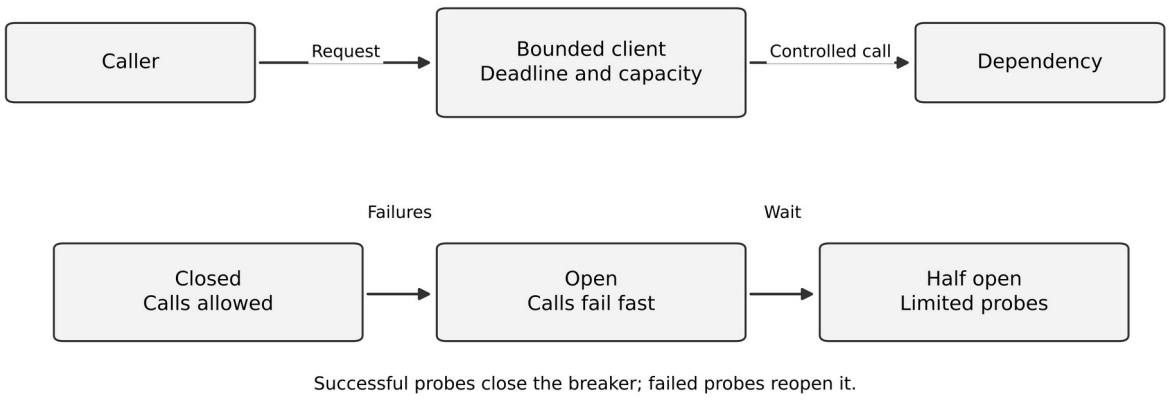


Figure 11 A dependency boundary combines a deadline capacity limit and controlled recovery

Use backpressure to prevent a producer from overwhelming a consumer. This may mean slowing intake, rejecting excess work with a clear response, or applying admission control. An unlimited queue can consume storage and hide a failure until recovery becomes impossible.

A fallback must preserve business meaning. Returning “request accepted” when nothing was stored is not graceful degradation. A safe fallback might permit viewing public service information while submission is unavailable, with an honest message about the unavailable action.

Availability and correlated failure

For a simplified model of four independent serial dependencies, each with availability 0.999, the combined availability is 0.999 to the fourth power, about 0.996006, or 99.6006 percent. This assumes all four are required and failures are independent. Real dependencies often share failure causes, so the calculation is a teaching approximation.

Redundancy also has assumptions. Two instances in one zone may share power, routing, configuration, and deployment failure. Even two zones can share a bad software release or a compromised credential. Identify failure domains rather than counting boxes.

A standby must have sufficient capacity, usable data, routing, permissions, and an exercised failover procedure. Recovery objectives should cover the whole business service, not only the database engine's restart.

Caching and replication

Caching trades repeated work for reuse of a stored result. Define the key, expiry, invalidation, and behaviour when the cache fails. A stale asset list may be acceptable for a short period; stale access permissions can be more serious.

A cache miss storm occurs when many callers simultaneously request data that is absent or expired. Techniques such as request coalescing, controlled refresh, and staggered expiry can reduce the load. Avoid making the cache an unexamined single point of failure.

Replication can improve read capacity or availability, but replicas may lag. Sending a user to a replica immediately after a write can make the new record appear missing. Route the operation according to the consistency guarantee it needs.

Test the failure model

A load test should describe the workload, data distribution, ramp, duration, concurrency, success criteria, and dependencies. A test that sends identical requests against a warm cache may say little about a service incident surge.

A resilience test deliberately exercises a defined failure, such as a worker restart after sending a handoff. Record what should happen, how you will observe it, and how you will restore the environment. Use isolated systems and fictional data for the course.

Causality and consensus boundaries

Lamport defines a happened-before relation from order within a process, sending and receiving a message, and transitivity. Events unrelated by that relation are concurrent in the model. Logical clocks preserve happened-before order, but a smaller logical timestamp alone does not prove causality. [33] In a portal incident, a later wall-clock timestamp from the CRM therefore cannot by itself settle which update depended on which other update. Keep explicit event and predecessor identifiers when the business needs that relationship.

Raft decomposes replicated-log consensus into leader election, log replication, and safety rules. A leader can commit an entry from its current term after replication to a majority under the algorithm's rules; merely counting copies of an arbitrary older entry is insufficient. [34] A three-voter cluster needs two voters for a majority. If a partition leaves one voter isolated, it must not independently commit competing decisions. This describes a consensus boundary, not a guarantee that every client operation finishes during a partition.

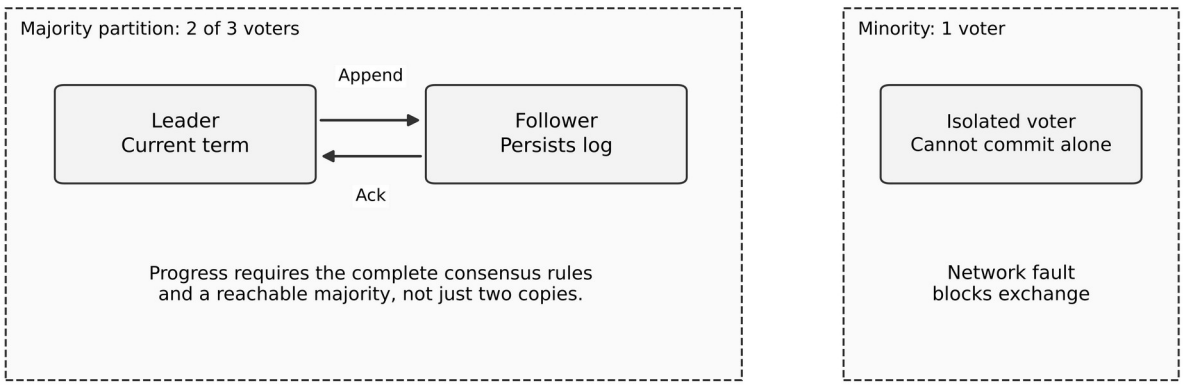


Figure 12 A majority partition can make progress while the minority cannot independently commit

The figure assumes crash and network failures, not Byzantine participants. Quorum membership, durable state, elections, and client read semantics still matter. Do not copy the picture as an implementation of Raft. The architecture implication is to ask which operation crosses the consensus boundary and what the caller observes when that boundary cannot progress. An external CRM side effect remains outside the portal database's replicated log.

A quantitative model of fan-out

Dean and Barroso explain why occasional latency outliers become consequential when a response depends on many components. Their paper discusses sources and mitigation of tail latency. [35] The following calculation is an original simplified model, not a benchmark: if each independent dependency has a 0.99 probability of finishing by a deadline, the probability that all 20 finish is 0.99^20, approximately 0.8179. About 18.21 percent of aggregate requests then have at least one late dependency.

Independence is a strong assumption. Shared overload can correlate failures, and optional dependencies change the completion rule. Replicating or hedging calls also consumes capacity and may duplicate unsafe actions. Use the model to identify a risk, then measure the real joint behaviour. Do not issue duplicate payment or work-order commands simply to reduce tail latency.

Histories make consistency claims testable

Herlihy and Wing define linearizability through a legal sequential account of operations that preserves the order of non-overlapping calls. They distinguish this object-level condition from transaction serializability and its real-time refinement, strict serializability. [36] The following register examples are original teaching histories. They separate observable behaviour from the storage technology used to implement it.

Let a register initially contain 0. Client A writes 1 between times 1 and 3 and receives success. Client B reads between times 4 and 5 and receives 0. With no other write, that completed history is not linearizable: A's completion precedes B's invocation, so the read must appear after the write. If B's read instead starts at time 2 and ends at time 5, the operations overlap. Returning 0 can then fit a linearization in which the read takes effect before the write. The response time alone does not determine the order.

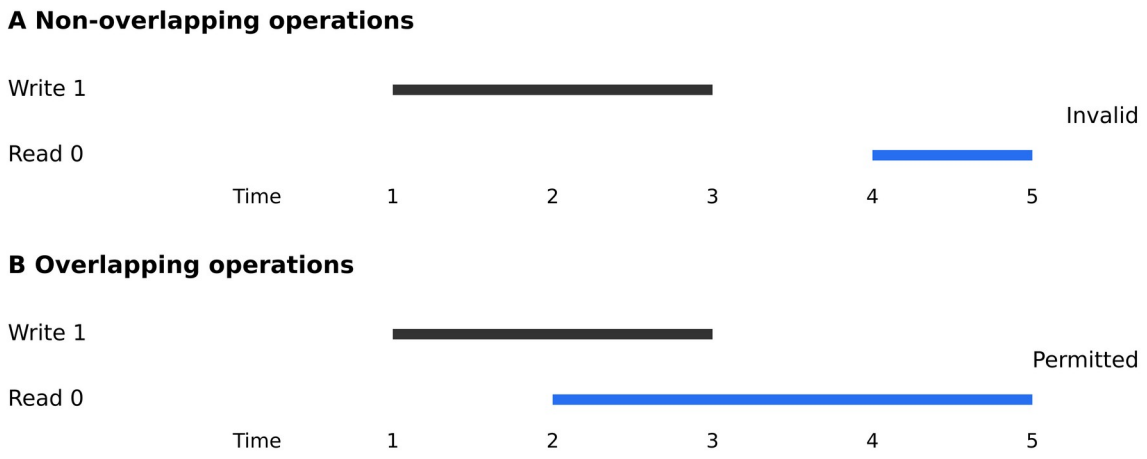


Figure 13 A stale read after a completed write differs from an overlapping read

This reasoning requires a trustworthy observation of the intervals. A test coordinator can establish a completion-before-invocation relation by waiting for one reply before issuing the next call. Comparing timestamps from unsynchronised machines is weaker evidence. Logical clocks help track causal order but do not supply a universal wall clock. Preserve request IDs and causal links as well as timestamps when collecting histories.

An unfinished operation introduces uncertainty. If A sends a write and times out, B may subsequently read 1 without violating linearizability: the write may have taken effect although A did not receive its response. A timeout is an observation about the client wait, not a rollback event. The history checker supplied with the second-edition practice files deliberately handles completed operations only; treating timeouts as definite failed writes would make that small model misleading.

Serializability asks whether complete transactions can be understood as some serial execution. In the abstract definition, the order need not follow real time. Strict serializability adds that requirement. A database can also provide atomic operations on individual objects without making a multi-object business workflow one transaction. Suppose a payment record and an inventory record each offer a linearizable update. Two separate updates still leave an interval in which only one has changed. The missing property is atomic coordination of the business operation, not a stronger spelling of each individual update.

Derive the partition decision from the operation

The CAP proof considers a read/write object under specific consistency and availability definitions. During a partition, an isolated reader cannot distinguish an execution with an unseen remote write from one without that write; requiring a response in both creates the contradiction. [30] This is a statement about possible executions, not a classification of every feature of a database product.

Apply the reasoning to an original last-unit example. Two locations each know that one unit is available. Communication fails. If both independently promise that unit to a different customer, the global no-overselling rule fails. If one location refuses or defers the reservation until it can reach the authority, the rule can be preserved at the cost of that operation's progress. A catalogue page can still show a labelled estimate. Availability therefore needs a definition per operation: viewing an estimate, accepting a tentative request and confirming a reservation are different services.

An alternative allocates separate selling rights before the partition. If there are 100 units and regions receive rights to 60 and 40, each can decrement its own allowance while preserving a global upper bound, provided rights cannot be duplicated, spending is atomic locally, and transfers preserve conservation. With only one unit, one region must hold its right. This design changes what information each region needs for its decision; it does not defeat the impossibility result. Reallocating rights and recovering a lost region require their own protocol. Book 2 develops the operational tradeoff.

What quorum intersection proves and leaves open

For five fixed voters, any two sets of three overlap: together they contain six memberships but there are only five possible members. That arithmetic establishes an intersection, not consensus by itself. An overlapping voter that forgets its durable vote, accepts incompatible rules or follows an unsafe membership change does not protect agreement. In Raft, voting restrictions and the current-term commit rule work together with log replication to preserve the required history. [34]

An original deployment places two voters in region A, two in B and one in C. Losing C leaves four voters and can retain a majority if communication and the algorithm's progress conditions hold. Isolating A leaves three on the other side; isolating both A and B from each other leaves no two-voter side able to commit alone. The placement therefore affects which failures interrupt progress. It does not establish a numerical availability target without failure and recovery evidence.

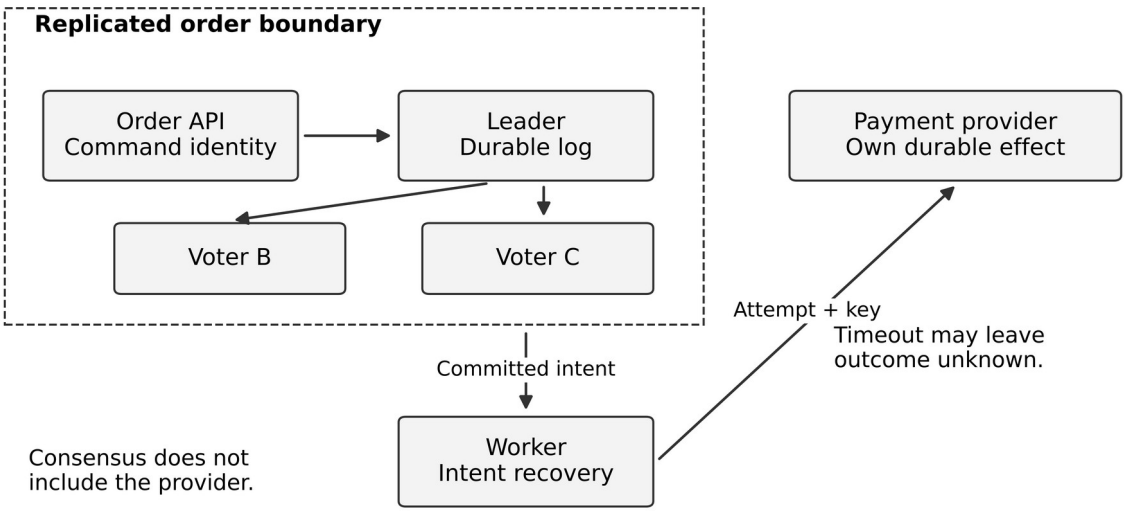


Figure 14 The replicated order decision and the external payment live in different correctness boundaries

The figure separates the durable order decision from the provider's effect. A committed intent can survive a leader change and still require reconciliation after a payment timeout. The worker's retry identifier must be meaningful to the provider, and its retention window must cover the retry policy. A local consensus cluster cannot extend the provider's idempotency window. Likewise, a follower read needs an appropriate read protocol if the caller requires a current result; being a member of a replicated cluster does not itself make every read current.

Safety means that an unacceptable outcome does not occur under the model; progress means that useful work eventually completes under the stated conditions. A design that refuses every request may preserve a safety predicate while being useless. Conversely, replying quickly with a false confirmation is not a successful availability strategy. State both obligations and the conditions for recovery. Crash-fault reasoning does not cover participants deliberately sending inconsistent protocol messages or a compromised administrator altering every replica.

An evidence ladder for a distributed design

Start with the sequential rule: what may a single successful command do? Then define the allowed concurrent histories, the durable commit boundary, and the failure assumptions. Construct at least one adversarial trace: reply lost after commit, leader replacement during a write, duplicate delivery, or policy change during a backlog. Finally, determine what production observation could distinguish a violation from a delayed observation.

A small exhaustive model can find an invalid interleaving that a large load test misses. It cannot show that the implementation follows the model. A load test can reveal a saturation point for the tested workload; it cannot prove all histories safe. A source proof supports a defined algorithm; it cannot certify a service whose configuration, persistence and client protocol differ. Combine these forms of evidence and keep each conclusion within its demonstrated scope.

Run history_checker.py in the second-edition advanced practice folder. It enumerates sequential orders for three tiny, completed register histories and filters out orders that contradict the observed real-time precedence. One history has no valid order; two do. The exercise teaches the decision process and is intentionally unsuitable as a general-purpose distributed-system verifier.

Exercises

12.1 A queue receives 30 messages per second and processes 24. How many messages accumulate in 15 minutes, assuming constant rates?

12.2 An API meets its average latency target but fails the p99 target. Give three hypotheses and the measurements that would distinguish them.

12.3 Design a failure test for a worker that crashes after the CRM accepts a request but before the worker records success.

12.4 A write of 1 completes before a read starts; the read returns 0. Explain why this is not linearizable, and change one timing fact so that returning 0 can be permitted.

12.5 Five fixed voters are split into groups of two, two and one. Can any group independently commit using majority consensus? Explain what additional contract is needed for a payment after a committed order intent.

Key points. Distributed architecture is largely about uncertainty, limits, and recovery. Measure complete outcomes, expose intermediate states honestly, and test the assumptions behind resilience claims.

CHAPTER 13

Cloud architecture with Azure and AWS examples

Learning objectives

Choose cloud service categories from responsibilities and requirements, map a neutral design to Azure and AWS, and assess portability and shared responsibility.

Cloud as a service model

NIST defines cloud computing through characteristics including on-demand access, broad network access, resource pooling, elasticity, and measured service. Its service models distinguish infrastructure,

platform, and software services. [37] These categories describe who manages which parts of a system, although real products can span boundaries.

With infrastructure as a service, you manage more of the operating system and runtime. With a managed platform, the provider takes responsibility for more of those layers. With software as a service, you primarily configure and integrate an application. In every model you still have responsibilities for appropriate use, access, data, and configuration.

A cloud decision should include skills, operating model, compliance needs, integration, availability, cost, and exit options. “The cloud scales” is not a capacity plan; quotas, database limits, and service configuration still matter.

Organisation and isolation

An account or subscription is an important administrative and billing boundary, not simply a folder. Separate environments and responsibilities so that development work cannot casually modify production. Establish identity, logging, policy, network organisation, and budget ownership before many teams deploy independently.

A landing zone is an organised foundation for workloads. It may include account or subscription structure, baseline security policies, network connectivity, audit logging, and deployment standards. It should make the approved path practical for teams rather than requiring every team to design these controls again.

Distinguish a region from an availability zone. A region is a geographical service location; zones are separate availability domains within a supported region. Service support, replication options, and guarantees differ, so verify the chosen service and region together.

A neutral reference deployment

For Meridian, begin with a managed application runtime, an operational relational database, object storage for attachments, a durable handoff mechanism, an identity service, secret management, and central telemetry. Place private dependencies behind controlled access and expose only the required interfaces.

The API and worker can share a codebase while scaling separately. The database transaction records submissions and outbox entries. A worker transfers handoffs. A scheduled analytics pipeline reads approved operational data through a controlled route.

This design does not require Kubernetes or an active-active multi-region database at the initial scale. Those can be evaluated when a requirement justifies their cost and operating complexity.

Mapping responsibilities to services

The following names are examples of services that can perform similar responsibilities. They are not feature-equivalent replacements, and the table is not a deployment specification. Microsoft's official AWS-to-Azure comparison is a useful starting map. [38]

Responsibility	Azure example	AWS example
Virtual machine compute	Azure Virtual Machines	Amazon EC2
Managed container application	Azure Container Apps	Amazon ECS with AWS Fargate
Kubernetes platform	Azure Kubernetes Service	Amazon EKS
Event triggered function	Azure Functions	AWS Lambda
Managed PostgreSQL	Azure Database for PostgreSQL	Amazon RDS for PostgreSQL
Object storage	Azure Blob Storage	Amazon S3
Work messaging	Azure Service Bus queues	Amazon SQS
Event routing	Azure Event Grid	Amazon EventBridge
API management	Azure API Management	Amazon API Gateway
Secrets storage	Azure Key Vault	AWS Secrets Manager
Network isolation	Azure Virtual Network	Amazon VPC

Responsibility	Azure example	AWS example
Monitoring services	Azure Monitor	Amazon CloudWatch

Choose based on required semantics. Service Bus and SQS, for example, have different configuration models and features; ordering and duplicate handling depend on the selected options. A service map is a vocabulary aid, not proof that one architecture can be transferred unchanged.

Identity and access in the platform

Separate workforce sign-in from workload permissions. Azure managed identities and AWS IAM roles can reduce the need to distribute long-lived credentials to workloads, but their trust and assignment models differ. Define exactly which service may assume an identity and what it may do.

Administrative access deserves a separate design: strong authentication, limited privileges, approval where needed, auditability, and an emergency access process. The application being secure does not compensate for an unrestricted production administrator account.

Infrastructure policy can prevent some invalid deployments, such as unapproved public exposure. It should be accompanied by documented exceptions and an owner. A policy that every team must bypass to deliver work is unlikely to remain effective.

Reliability and data placement

Choose where authoritative data lives, how it is replicated, and what happens during regional failure. Active-passive designs can simplify conflict handling but require tested recovery. Active-active designs can improve geographical access while making write conflicts and consistency more demanding.

An availability-zone design addresses some local failures; it does not automatically meet a regional disaster recovery requirement. Backup copies, keys, identity dependencies, DNS, and operational access must remain usable during the failure you are planning for.

Data residency is about where data is stored or processed. Sovereignty and lawful international transfers involve additional legal and organisational questions. Do not equate selecting a European region with completing a privacy assessment.

Portability and lock-in

Every platform creates dependencies. Even virtual machines depend on networking, images, identity, monitoring, and operational procedures. Managed services may increase platform-specific integration while reducing the team's maintenance burden.

Identify which exit scenarios are credible. Can you export the data in a usable form? Can another system interpret the identifiers and schema? How long would migration take? Which contracts and licences apply? Test the important export path before it becomes urgent.

Use abstractions selectively. A generic wrapper that hides every cloud capability can impose the cost of the least capable provider without guaranteeing portability. Protect business logic and data contracts while accepting documented platform choices where they add value.

Worked choice

Meridian compares a managed container runtime with virtual machines. The application has ordinary HTTP and worker workloads, no unusual kernel dependencies, and a small operations team. A managed runtime reduces operating system work and supports separate scaling.

The decision remains conditional on networking, runtime limits, observability, deployment controls, and total cost. The team runs a small proof of concept that includes database access, a simulated CRM outage, and a rollback. A successful “hello world” deployment is insufficient evidence.

Exercises

13.1 List responsibilities that remain with Meridian when it uses a managed relational database.

13.2 Map the neutral submission architecture to both Azure and AWS. Record three differences that need investigation rather than assuming equivalence.

13.3 The sponsor requests multi-cloud to avoid lock-in. Propose questions and an evaluation that distinguishes an exit plan from operating two clouds simultaneously.

Key points. Start with responsibilities and guarantees, then choose services. Managed infrastructure reduces some work while leaving design, access, data, and operational accountability with the organisation.

CHAPTER 14

Containers orchestration and infrastructure as code

Learning objectives

Explain what containers and orchestration provide, read a basic deployment model, and design safe infrastructure change with reproducibility and state management.

Images and running containers

A container image packages application files and dependencies. A running container is an instance of that image with runtime configuration and writable state. Containers generally share the host operating system kernel rather than carrying a complete guest operating system. Docker's introductory documentation explains this distinction. [39]

Keep images reproducible and small enough to maintain. Pin appropriate dependencies, use a trusted base image, and rebuild when security updates are required. A tag such as latest is a moving name; a digest identifies specific image content.

Run with the least privileges and filesystem access practical for the workload. Avoid embedding secrets in an image because layers and registries can preserve them. Supply configuration and credentials through controlled runtime mechanisms.

A container can still consume excessive CPU, memory, disk, or connections. Set and test resource limits. Memory exhaustion may terminate a process abruptly, so a business workflow must tolerate interruption.

What orchestration adds

An orchestrator schedules workloads, monitors desired state, replaces failed instances, and coordinates deployment and networking mechanisms. Kubernetes provides APIs and controllers for managing containerised workloads. It does not automatically provide an application's business-level availability or a complete security policy. [40]

A Pod is the smallest deployable scheduling unit and contains one or more closely related containers. A Deployment manages replicas and updates for suitable workloads. A Service provides a stable way to reach a set of Pods. Configuration and secret mechanisms supply runtime inputs. Persistent storage requires an explicit design.

A StatefulSet supports stable identities and storage-related patterns for stateful workloads, but selecting it does not make a database correctly replicated or safely backed up. Managed data services can still be preferable.

Health and readiness

Liveness asks whether a process should be restarted. Readiness asks whether it should receive traffic. Startup checks can allow a slow initialisation period without premature failure handling. A health endpoint that always returns success gives little evidence.

Avoid making every liveness check depend on the database. A database outage could cause every application instance to restart repeatedly, adding load and delaying recovery. Readiness and degraded-service behaviour should reflect what useful work the application can actually perform.

During shutdown, stop accepting new work, allow bounded completion where appropriate, and preserve retryable state. A worker must not acknowledge a task before durable completion simply because the platform is terminating it.

Scaling and placement

Horizontal scaling adds instances. Vertical scaling gives an instance more resources. Both have limits. More API replicas may increase database connections; more consumers may violate an external vendor's rate limit.

Use a signal that corresponds to the bottleneck. CPU can be useful for CPU-bound services. Queue age or backlog can be more informative for workers, but scaling must respect downstream capacity. Keep a maximum and a recovery strategy for overload.

Place replicas across relevant failure domains when the availability requirement justifies it. Then test disruption and capacity after losing one domain. Two replicas that each run at ninety percent capacity cannot necessarily absorb the loss of one.

Infrastructure as code

Infrastructure as code describes infrastructure through versioned definitions and automated changes. It supports repeatability, review, and comparison between intended and deployed state. It does not guarantee correctness; a mistaken definition can reproduce a mistake very efficiently.

A useful change flow validates the definition, produces a change plan, reviews material consequences, applies through a controlled identity, and checks the result. Separate production permissions from local experimentation.

Terraform state maps configuration to managed resources and stores information needed for subsequent operations. State can contain sensitive values. Protect its storage, access, versioning, and recovery, and use locking where the selected backend supports it. [41]

Drift and recovery

Drift occurs when deployed resources differ from the expected configuration. A manual emergency change may be necessary, but it must be reconciled into code or deliberately reversed. Otherwise the next automated change can unexpectedly undo it.

Treat deletion and replacement carefully. A plan that replaces a database resource is not equivalent to replacing a stateless application instance. Review backups, data migration, dependency changes, and recovery before applying destructive operations.

Keep infrastructure code separate from secret values. A variable named password is not safe merely because it is in a configuration file. Some tools redact display while still retaining values in state.

Reading a deployment model

The following neutral model is a design sketch, not executable vendor configuration. It exposes the responsibilities that a real manifest must express.

Workload	Replicas and placement	Persistent state	Health and limits
API	Two across chosen failure domains	Database and object storage	Readiness for useful traffic and bounded connections
Worker	Initial one with controlled scale out	Outbox and attempt history	Lease expiry and downstream rate limit
Database	Managed availability configuration	Durable records and backups	Capacity monitoring and tested recovery
Attachment processing	Bounded isolated workers	Object processing state	Size limits and controlled release

Suppose a proposed change increases worker replicas from one to eight and reduces shutdown grace from sixty seconds to five. Review claim exclusivity, lease duration, vendor rate limits, and behaviour when a task is interrupted. Eight workers can process the same pending record incorrectly if claims are not atomic. A five second shutdown window is safe only when incomplete work remains durable and a retry cannot produce an unacceptable effect. Test the interruption point rather than accepting the new settings because the deployment tool permits them.

Choosing the operating model

Kubernetes can be valuable when a team needs its orchestration capabilities and can operate or consume a well-supported platform. It also introduces cluster management, policy, upgrades, networking, and troubleshooting responsibilities.

For Meridian, a managed container application platform may meet the initial requirements with less operating work. The architect should compare deployment control, scaling, networking, observability, security, team skills, and total cost. “Industry standard” is not enough reason to add a platform.

A platform team can provide a tested deployment path, common telemetry, identity integration, and secure defaults. Application teams still own application behaviour, data use, and recovery procedures.

Exercises

14.1 Explain why restarting every application instance during a database outage can make recovery harder.

14.2 Sketch a deployment with two API replicas and one scalable worker group. Identify which state must survive replacement and where it lives.

14.3 A Terraform plan proposes replacing a production database. Write the review questions that must be answered before approval.

Key points. Containers package execution; orchestration manages workloads; infrastructure as code manages change. None replaces application-level guarantees or an operating model that the team can sustain.

CHAPTER 15

Security privacy and threat modelling

Learning objectives

Derive controls from specific threats, connect privacy requirements to data flows, and define evidence that shows whether the controls work.

Security is a set of properties

Confidentiality concerns unauthorised disclosure. Integrity concerns unauthorised or incorrect change. Availability concerns whether authorised users can obtain the required service. Accountability and traceability help establish what happened and who or what performed an action.

A design needs to identify what it is protecting. Meridian's assets include request content, credentials, access policies, audit records, source code, deployment permissions, and the continuity of a time-sensitive service process. Different threats affect different assets.

Classify data by its actual sensitivity and business context. A record that looks harmless alone can become identifying when combined with another dataset. Avoid assuming that removing names always produces anonymous data.

Threat modelling

Threat modelling is a structured way to examine what can go wrong, what is being protected, and how the design responds. OWASP's guidance emphasises understanding threats and mitigations in the context of valuable assets. [42]

Start with a current data flow and trust boundaries. Identify entry points and actors, including administrators and service identities. For each flow, ask whether a caller can impersonate someone, change data, deny an action, disclose information, exhaust capacity, or gain privileges. STRIDE is one mnemonic for these categories; use it to generate questions rather than to claim exhaustive coverage.

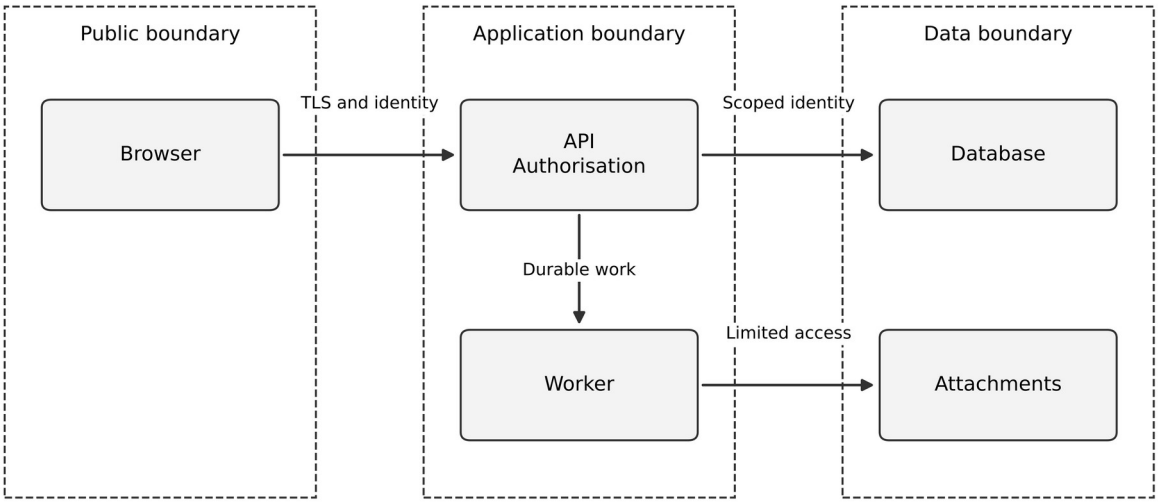


Figure 15 Trust boundaries separate public interaction application processing and controlled data services

For each credible threat, record the affected asset, attack condition, impact, proposed control, owner, and verification method. Prioritise by consequence and likelihood under the actual exposure. A numerical score should make the judgement discussable, not create false precision.

An example threat register

Threat	Main control	Evidence	Residual concern
Service agent reads another unit's service request	Resource-level authorisation	Cross-unit denial tests	Misassigned business roles
Malicious attachment reaches staff	Restricted upload and isolated scanning workflow	Safe test files and state checks	New or undetected malicious content
Leaked worker credential exposes all data	Narrow workload identity and short-lived credentials	Permission review and denied-action test	Compromised authorised worker
Replayed webhook creates duplicate work orders	Signature checks and durable deduplication	Replay and crash tests	Provider's external side effects
Log export discloses request text	Data minimisation and access controls	Log content inspection	Accidental future instrumentation

The controls reduce risk; they do not eliminate the need to respond to incidents. Every control can fail through design, implementation, configuration, or operating mistakes.

Foundational protection principles

Saltzer and Schroeder identify principles including least privilege, fail-safe defaults, and complete mediation. Least privilege limits authority to the task; fail-safe defaults make access depend on permission rather than the absence of a prohibition; complete mediation requires checking authority for access to protected objects. [43]

Applied to Meridian, a new API route should deny access until its policy is defined, a worker should receive only its required permissions, and an attachment download should check the actual object relationship. Caching an access decision introduces a question about how policy changes invalidate that decision. These principles explain why a convenient global administrator role or a login check alone is insufficient. They are design guidance, not a proof that a particular implementation is secure.

Defence in depth and Zero Trust

Defence in depth uses several independent controls around an important property. For an attachment, that might include authorisation, object-level access policy, encryption, malware handling, and audit records. Repeating the same weak assumption in several places is not independent protection.

NIST's Zero Trust architecture rejects implicit trust based only on network location or ownership. Access decisions should consider identities, resources, and context rather than assuming that everything inside a perimeter is safe. [44] This does not require buying a product named Zero Trust.

Separate administrative actions from ordinary application actions. Protect privileged access, use individual identities, and log consequential changes. Test the emergency access process so that a security control does not leave the team unable to recover the service.

Encryption and key management

Encrypt appropriate data in transit and at rest, but understand which threat the control addresses. Storage encryption helps against some forms of storage exposure. It does not stop an authorised application from reading and disclosing plaintext.

Key management includes creation, access control, rotation, backup or recovery where appropriate, and revocation. Separation of duties may be needed for particularly sensitive data. Losing access to an encryption key can make data unrecoverable even when backups exist.

Hashing is different from encryption. Password storage requires a suitable password-hashing mechanism, not a fast general-purpose hash or reversible encryption chosen without a security design. Prefer an established identity provider rather than creating a new password system for this course.

Privacy through the lifecycle

The GDPR establishes principles such as purpose limitation, data minimisation, storage limitation, and accountability. Articles 25 and 32 address data protection by design and appropriate security, while Article 35 addresses impact assessments for processing likely to result in high risk. The exact obligations depend on the processing and organisational roles. [45]

Translate these principles into architecture questions. Why is each field needed? Who may access it? How long is it retained? Which copies exist? How is a correction or deletion propagated? Which processors and locations are involved? What evidence supports the approved policy?

For Meridian, do not invent a universal retention period for closed service requests. Obtain the business and legal policy, then implement scheduled lifecycle actions, exceptions, audit records, and verification. Backups and legal holds need explicit treatment. A deletion request may interact with lawful retention duties; the architecture should support the approved decision rather than assuming every request means immediate erasure everywhere.

Secure development and verification

Use a security requirements baseline appropriate to the application. OWASP ASVS provides versioned verification requirements for application security. Reference the exact version and requirement identifiers when using it in a real assurance plan. [46]

Combine design review, dependency management, code review, automated checks, configuration review, and targeted testing. A vulnerability scanner cannot establish every business authorisation rule. A penetration test is a point-in-time investigation, not a permanent guarantee.

Keep security evidence connected to the release. Record which build, configuration, environment, and scope were tested. A test result from an old deployment may not apply after a significant identity or network change.

Incident readiness

Prepare for credential exposure, data disclosure, destructive change, and service disruption. Define detection, triage, containment, recovery, communication, and evidence preservation. Assign roles before an incident begins.

A runbook should tell an operator how to identify the affected service, restrict a compromised identity, preserve relevant logs, and restore a known-good state. Exercise the process using fictional scenarios. Legal reporting decisions should follow the organisation's current incident and privacy procedures.

A security argument for attachment delivery

Consider a Meridian attachment containing a customer's equipment incident details. The required property is that only an authorised principal can obtain its bytes through supported application paths. Define the adversary: an ordinary customer can alter request parameters and replay a previously issued download link; a stolen worker credential is a separate scenario. This boundary excludes a compromised customer device after a legitimate download. Once plaintext has been delivered, server-side revocation cannot erase every recipient copy.

List the paths to the bytes: the application API, a storage URL, a preview service, a support export, a log containing the link, and a restored backup used by operators. A control applied only to the visible download button misses the other paths. Saltzer and Schroeder's principles motivate checking authority and limiting privileges; their account also treats changes to authorisation as a substantive design problem. [43] The following alternatives are course designs, not a claim that their paper prescribes a cloud implementation.

Alternative A uses an application proxy. The browser requests an attachment by ID. The API retrieves its ownership metadata, validates the user's current entitlement, verifies that the attachment has reached the approved scan state, and streams the bytes using a workload identity. The object store permits the workload identity and denies public access. The same predicates apply to previews and exports. Benefits include a central point for request-time authorisation and an audit trail tied to the user. Costs include data transfer through the application, proxy capacity and a larger role for the application's availability.

Alternative B issues a short-lived storage capability. After the same policy check, the API issues a narrowly scoped download authorisation with a short lifetime and an exact object identifier. The browser downloads directly from storage. This can reduce proxy load, but possession of the capability may permit use until it expires under the selected storage mechanism. Do not promise immediate revocation unless that mechanism actually provides and enforces it. Avoid placing capabilities in analytics events or routine logs. A short lifetime reduces one exposure window; it does not repair an object-scope mistake.

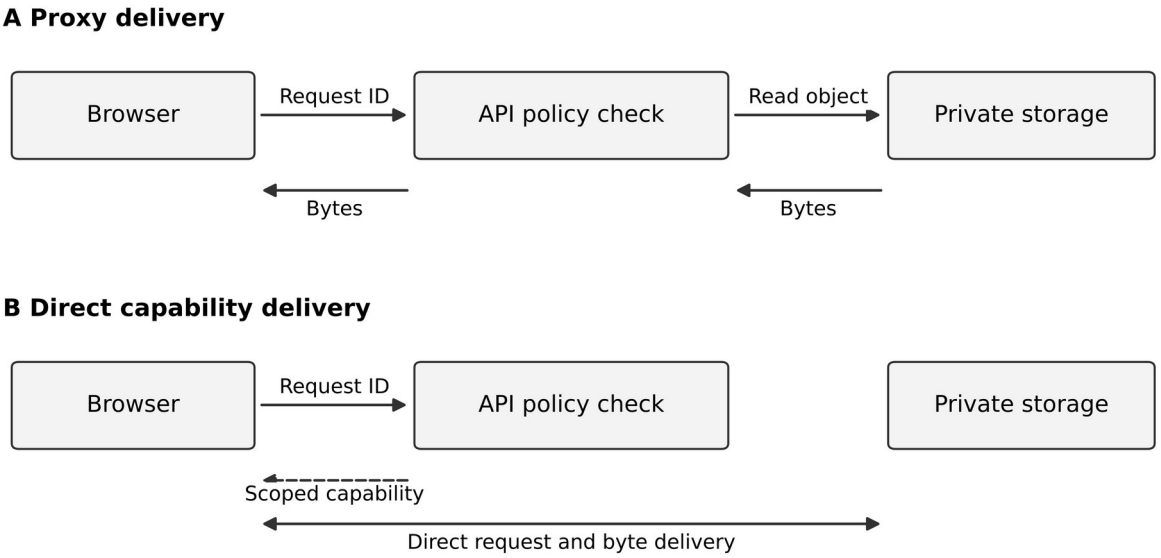


Figure 16 Proxy delivery and direct capability delivery place enforcement at different points

The solid arrows in the figure are requests or byte delivery. The dashed arrow conveys a scoped capability after an access decision; it does not carry the file bytes. Both alternatives keep storage private in their normal configuration. Direct delivery delegates a bounded access decision to storage; it does not remove the need to authorise the user. Neither alternative by itself prevents an already authorised user from taking a screenshot or copying downloaded data.

Choose using the revocation and throughput requirements. If a sensitive document must not be newly fetched after an entitlement is removed, a long-lived bearer URL is a poor fit. A proxy that checks current policy can deny new requests, but an already active stream still needs a defined interruption policy. If attachments are large and a short residual window is acceptable, narrowly scoped direct download may be justified. The approval must specify the window, threat, owner and evidence, rather than label the choice simply secure.

Policy freshness and failure are architectural choices

NIST SP 800-207 describes resource-centred access, dynamic policy and distinct decision and enforcement components. It also discusses disruption of those components as a threat. [44] In this example, an online policy decision service creates a dependency; a cached decision creates a freshness problem. The tradeoff cannot be settled by drawing an extra security box.

Assume an illustrative policy cache has a maximum age of 60 seconds and refresh is unavailable. If access was granted just before revocation, a cache that honours its full lifetime may continue granting for almost 60 seconds, plus any explicitly allowed clock or evaluation tolerance. An event-driven invalidation channel can usually shorten the delay, but it is not an upper-bound proof when messages can be delayed. A required revocation bound needs a specified enforcement rule, measurement point and failure behaviour.

For new access to restricted attachments, one defensible policy is to deny when authoritative policy cannot be obtained and any permitted cached decision has expired. A less sensitive public status page can remain available using cached data. This is a classification of operations and data. It is not a universal instruction to make every business service unavailable whenever one security service fails. Emergency access must be a separately authorised and audited process with an owner, not an undocumented bypass flag.

Build tests that could refute the security claim

Create a table of principal, resource, action, policy state and expected outcome. Include a user from another tenant, the right tenant but wrong request owner, an agent assigned to the wrong unit, a revoked delegation, an unscanned object, and a capability for a different object. Exercise API, preview and export routes. Do not use real customer data. If the design permits a bounded stale decision, the expected result must explicitly distinguish before and after the bound.

Test a compromised workload credential separately. Remove permissions that the worker does not need and demonstrate that it cannot enumerate unrelated objects, change policies or grant itself broader authority. A denied call establishes something about the tested identity and configuration at that time; it does not prove the credential can never be abused. Record build, policy version, environment and evidence location. Combine configuration inspection with behavioural tests so a permissive alternative path is less likely to be overlooked.

Finally, ask whether the logs needed for accountability introduce a new disclosure path. Record identifiers and decision reasons where sufficient; restrict detailed content and credentials. If a full payload is necessary for a specific investigation, treat capture, access and deletion as an approved exception. The evidence used to show that a system is secure must itself have an access policy.

Exercises

15.1 Choose one Meridian trust boundary and describe two threats with controls and test evidence.

15.2 Explain why encryption at rest does not prevent an overprivileged application account from disclosing data.

15.3 Design a deletion workflow that covers the operational database, attachments, analytics, search indexes, and backups without inventing a retention policy.

15.4 Compare proxy and direct-capability attachment delivery for a policy requiring revocation of new access within ten seconds. State the evidence each design needs.

15.5 A sixty-second authorisation cache receives best-effort invalidation events. Can the design promise immediate revocation? Define a defensible bound and failure policy.

Key points. Controls should address explicit threats and be supported by evidence. Privacy must be designed across the data lifecycle, including copies, third parties, and recovery arrangements.

CHAPTER 16

DevOps testing and controlled delivery

Learning objectives

Design a delivery pipeline that produces trustworthy releases, handle schema evolution, and use delivery measurements to improve the system.

Delivery is part of the architecture

A design that is difficult to release safely will accumulate operational risk. The delivery path determines who can change production, which checks run, how artefacts are identified, and how a failed change is recovered.

Continuous integration regularly integrates changes and checks them. Continuous delivery keeps software in a state that can be released through a controlled process. Continuous deployment automatically releases qualifying changes to production. The terminology should be defined within the team because organisations sometimes use it differently.

The goal is a repeatable path from reviewed change to observable production behaviour. Automating an unreliable manual process without understanding it can simply make mistakes faster.

A practical pipeline

A useful pipeline checks source changes, builds an immutable artefact, runs relevant tests and security checks, deploys to a representative environment, verifies behaviour, and promotes the same artefact to production under the agreed controls.

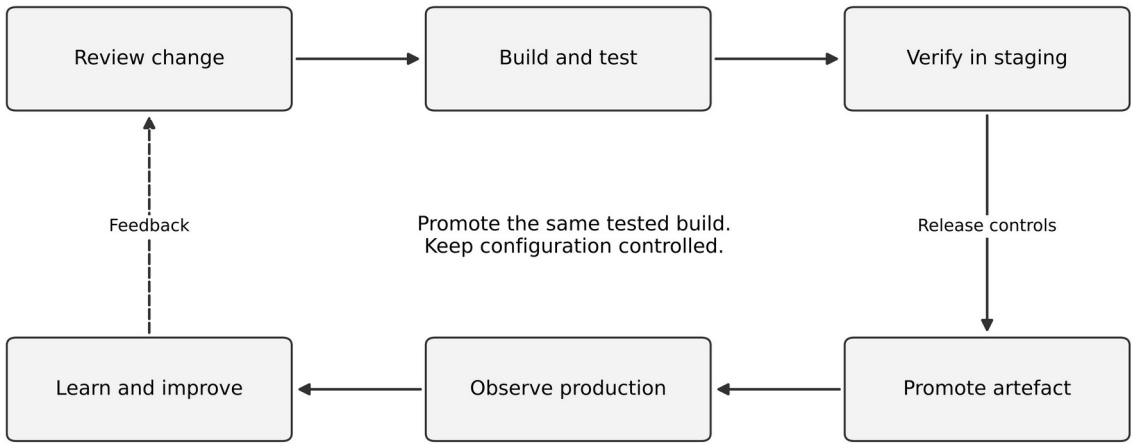


Figure 17 Delivery promotes a tested artefact and checks behaviour after release

Use the same built artefact across environments where practical. Rebuilding separately for production can introduce a different dependency set or build result. Supply environment-specific configuration through controlled mechanisms.

Protect the pipeline itself. A compromised build identity may have production reach. Limit credentials, review third-party actions and dependencies, preserve audit logs, and separate build permissions from deployment permissions where appropriate.

Test boundaries

Use unit tests for deterministic domain behaviour, integration tests for real dependency interactions, contract tests for provider and consumer agreements, and end to end tests for critical journeys. Add load, security, and recovery tests where the requirements demand them.

A representative environment should match the aspects that matter to the risk. It may not need production's full capacity, but an identity integration test needs realistic token validation and permissions. A recovery test needs realistic backup formats and access.

Avoid asserting that test coverage percentage measures system quality. Coverage says that code was executed by tests under a chosen metric. It does not prove that the assertions were meaningful or that operational failure modes were exercised.

Database change

A common safe approach is expand, migrate, and contract. First add a compatible field or structure. Next deploy code that can operate through the transition and migrate data. Finally remove the old structure after consumers no longer depend on it.

For example, replace one name field with separate display and legal names only after defining their meaning. Add the new fields, populate them through an approved rule or user action, update consumers, and remove the old field later. A schema migration is not a substitute for resolving ambiguous data semantics.

Rolling back application code does not automatically roll back a destructive data change. Before release, decide whether to roll back, roll forward, restore, or reconcile. Test the path with realistic data volumes.

Deployment strategies

A rolling deployment replaces instances gradually. A blue-green deployment maintains two environments and switches traffic. A canary exposes a limited portion of traffic to a new version and compares outcomes before wider release.

Each strategy has constraints. Two versions may run concurrently and must share compatible contracts and data. A canary with only low-risk traffic may miss a defect that affects another user group. A fast rollback is helpful only if the previous version remains compatible with the current data and configuration.

Feature flags separate deployment from exposure of a feature. They can support gradual rollout and emergency disabling, but create additional states to test. Remove obsolete flags so they do not become permanent hidden branches.

Supply chain and provenance

Know which source revision, dependencies, build process, and configuration produced the running artefact. Retain enough provenance to investigate a vulnerable dependency and rebuild or replace the artefact.

A software bill of materials can help inventory components, but it must be connected to the actual build and kept current. Signing or verifying artefacts helps establish origin and integrity; it does not prove that their contents are safe.

Do not download arbitrary deployment scripts into a privileged pipeline without review. A convenient setup step can become a path around otherwise careful access controls.

Delivery measurements

As of the source check for this edition, DORA describes five software delivery performance metrics: change lead time, deployment frequency, failed deployment recovery time, change fail rate, and deployment rework rate. Use them in the context of one application or service and focus on improvement over time. [47]

Define the start and end events for each measurement. A deployment count is not useful if teams count fundamentally different operations. Do not use the metrics as individual productivity rankings; the delivery system includes shared dependencies and organisational conditions.

Connect measures to action. If change lead time is high because reviews wait four days, faster build hardware will not address the main delay. If failures come from configuration changes outside the pipeline, improve that path too.

Worked release plan

Meridian adds a new CRM status code. The provider publishes the change before all clients are updated. The portal first deploys a compatible parser that preserves an unknown status safely and shows a controlled pending label. It records the unexpected value for investigation without exposing raw data to customers.

After the mapping is agreed and tested, the team enables the new behaviour. This is safer than assuming every new enum value is an error and failing the entire workflow. The decision still needs business review because a new status may have important meaning.

Assurance across the software lifecycle

NIST's Secure Software Development Framework version 1.1 describes high-level practices that can be integrated into different development lifecycles and used in supplier discussions. It addresses reducing vulnerabilities, limiting their consequences, and responding to their causes. [48] It is guidance for an assurance process rather than evidence that a particular build is safe.

For each consequential release, connect the claim to the artefact: source revision, dependency inventory, build identity, image digest, configuration revision, migration version, test environment, and approval. A signed image with an unreviewed production configuration does not establish the full deployed state. A design review without a corresponding implementation check leaves a similar gap.

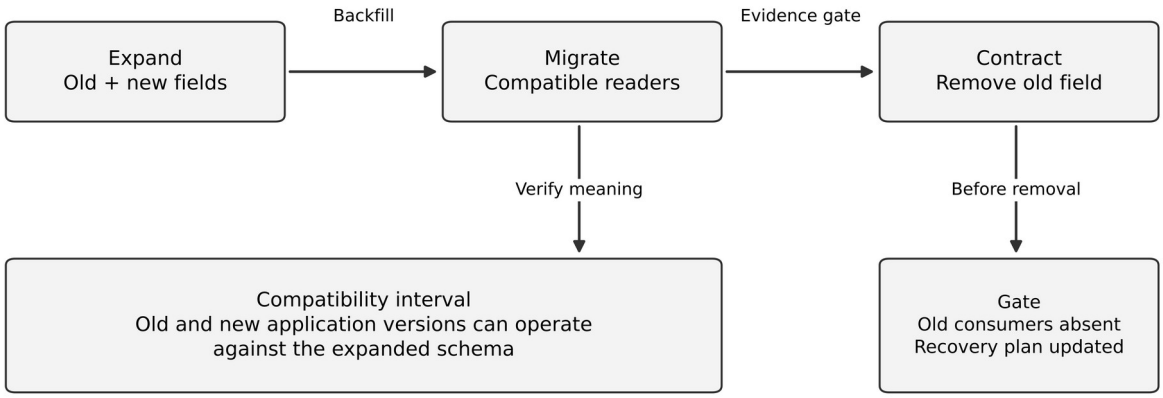


Figure 18 Expand migrate and contract preserve an explicit compatibility interval

Read the figure as a sequence of compatible states. Both application versions can use the expanded schema during the migration interval. The contract step is gated on evidence that old readers and writers are absent and rollback expectations have changed. For a renamed status field, validate semantic equivalence as well as row counts; copying the wrong meaning perfectly is still a failed migration.

Exercises

16.1 Describe a release pipeline for the submission API and name the evidence required before promotion.

16.2 A release removes a database column used by the previous application version. Explain why an application rollback alone may fail and propose a safer sequence.

16.3 Deployment frequency improves but failed changes and rework also rise. What should the team investigate before declaring success?

Key points. A safe delivery system makes changes reproducible, testable, observable, and recoverable. Architecture should include the pipeline and data transition, not just the running application.

CHAPTER 17

Observability service objectives and operations

Learning objectives

Choose useful telemetry, calculate service indicators and error budgets, and design an operating response that turns signals into action.

Observe a service through outcomes

Monitoring checks known conditions. Observability concerns the ability to understand internal behaviour through available outputs. In practice the terms overlap, but the design question is concrete: can the team explain why this customer did not receive a valid receipt?

Metrics summarise measurements over time. Logs record events and context. Traces follow a request or workflow across operations. OpenTelemetry provides vendor-neutral concepts and instrumentation for signals including traces, metrics, and logs. [49]

Use correlation identifiers that connect an API request, request record, outbox item, worker attempt, and vendor interaction. A request identifier should help investigation without becoming a secret or exposing personal data.

Instrument what matters

For an HTTP service, measure traffic, latency, errors, and resource saturation. Google's SRE guidance calls these the four golden signals. [50] Add domain-specific measures: accepted submissions, synchronisation delay, failed handoffs, and unresolved duplicate cases.

Do not log complete service requests by default. Use structured, bounded fields with controlled access and retention. Avoid putting high-cardinality identifiers such as every service-request ID into metric labels; that can make telemetry expensive and difficult to operate. Logs or traces are usually more appropriate for individual identifiers.

Define success at the observation point. A load balancer returning a response is not necessarily evidence of a durable submission. Instrument the business boundary where responsibility is accepted.

SLO calculations

Suppose Meridian observes 2,000,000 eligible requests in a month. At a 99.9 percent success target, the allowed unsuccessful count is 0.001 multiplied by 2,000,000, or 2,000 requests. If 500 are unsuccessful, 25 percent of that allowance has been consumed.

This error budget is based on request counts. A time-based budget for the same percentage uses minutes or seconds instead. Do not mix denominators. Define treatment of invalid requests, planned maintenance, external failures, and monitoring gaps before reporting.

A burn rate compares the observed bad-event fraction to the permitted bad-event fraction. If the current bad fraction is 1 percent and the permitted fraction is 0.1 percent, the burn rate is ten. The rate helps describe how quickly the budget would be consumed if conditions continued.

A short severe incident and a long mild incident need different detection windows. Use multi-window reasoning so that alerts respond to sustained impact while avoiding unnecessary noise from tiny samples.

Alert on action

An alert should have an owner, a meaningful condition, and an expected response. “CPU is 70 percent” may be an informational trend. “Submission success is below the SLO and the queue age threatens the business deadline” may require action.

A good alert includes service, scope, severity, observation time, a link to the dashboard, and a runbook. Avoid including secrets or customer content. Test that the notification reaches the responsible team and that the runbook can be followed.

Use dashboards to support diagnosis, not merely to display every available metric. A first dashboard should show user impact, workload, dependencies, recent deployments, and capacity. Drill down from there.

Runbooks and recovery

A runbook describes a repeatable operational task. For a CRM outage, it might confirm provider status, inspect the handoff backlog, stop unsafe manual retries, communicate expected delay, and resume processing under a controlled rate after recovery.

Include conditions for escalation and for stopping an action. Replaying all failed messages at once can overwhelm a recovered dependency. A repair script should be reviewed, bounded, and auditable.

Test backups and restores, credential rotation, queue replay, and failover. Record the actual recovery time and the manual steps that caused delays. Recovery evidence should influence architecture priorities.

Incident learning

During an incident, separate coordination, technical investigation, and communication. Keep a timeline of observations and decisions. After recovery, examine contributing conditions and improvements without reducing the event to one person's mistake.

An incident review should identify why the system allowed the failure to affect users and why detection or recovery took the time it did. Actions should have owners and follow-up. “Be more careful” is not a strong corrective action.

For Meridian, repeated CRM throttling may reveal that the worker's scaling policy ignores the vendor's limit. The improvement could be a shared rate limiter and backpressure, rather than simply adding retries.

Capacity and operating cost

Observe trends before hard limits are reached. Database storage, connection counts, queue retention, certificate expiry, and log volume can all become operational risks. Forecast using actual demand and planned changes, with uncertainty recorded.

Telemetry also has a cost. Choose retention, sampling, and detail according to investigation and assurance needs. A cost reduction that removes the only evidence of an important failure can be a poor trade.

Trace a delayed outcome without leaking content

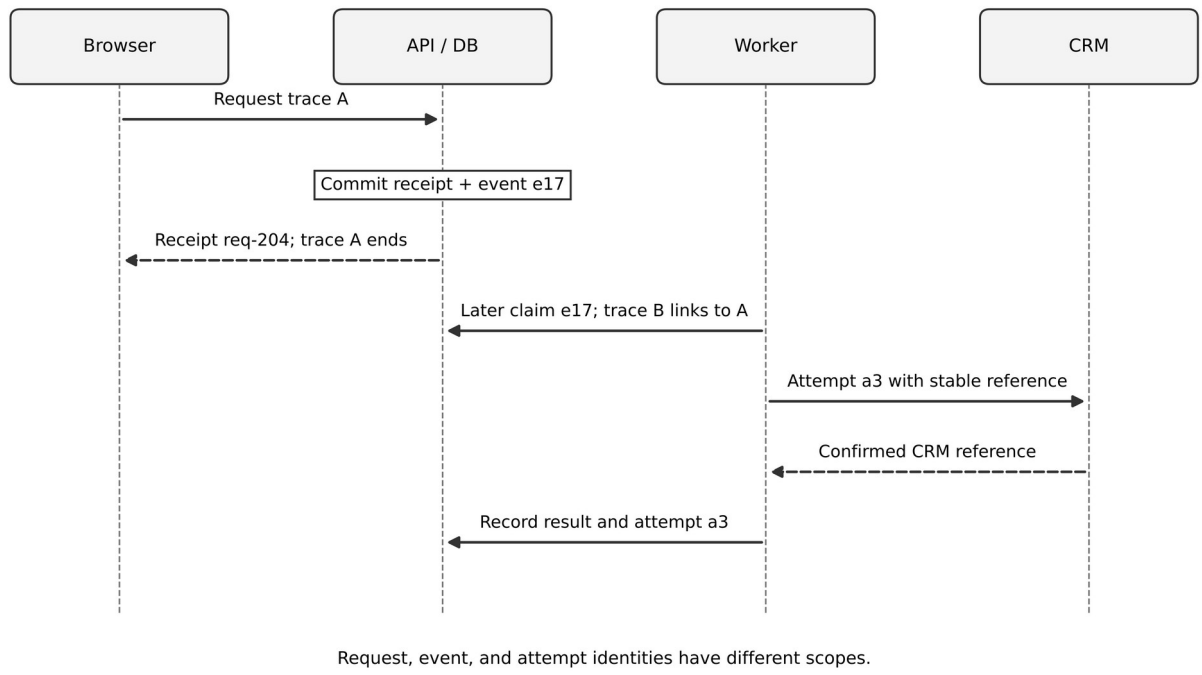


Figure 19 Correlation links a short receipt request to a later asynchronous handoff

The receipt trace ends when the durable acknowledgement is returned. A later worker trace links to the handoff identifier rather than pretending the browser connection stayed open. Request identifiers, event identifiers, and attempt identifiers have different meanings: one accepted submission may generate several delivery attempts. Keep that distinction in dashboards so retries do not inflate successful business outcomes.

To investigate a complaint, first locate the receipt and its tenant scope using authorised support access. Then inspect the durable handoff state, last attempt category, and vendor reference. Finally inspect the corresponding downstream result or reconciliation record. A useful event says that request req-204 was confirmed by attempt 3; it need not contain the customer's description or attachment contents. Sampling traces should not remove the durable workflow evidence needed to account for every accepted request.

Exercises

17.1 With 800,000 eligible requests and a 99.95 percent success objective, calculate the allowed bad requests. If 120 fail, what fraction of the allowance is used?

17.2 Define one useful metric, log event, and trace span for the CRM handoff. Explain what question each answers.

17.3 Write an actionable alert for an ageing handoff queue, including owner, threshold rationale, and first response.

Key points. Telemetry should make business outcomes and failure causes understandable. SLOs need precise definitions, and operational signals need owners and tested responses.

CHAPTER 18

Economics procurement and build or buy decisions

Learning objectives

Construct a transparent cost model, compare options without hiding assumptions, and evaluate a vendor as an operational dependency.

Cost follows design and use

A system's cost includes implementation, licences, infrastructure, support, security, operations, training, migration, and eventual exit. Some costs are fixed over a range of demand; others vary with requests, storage, data transfer, users, or operating effort.

The cheapest infrastructure line item can produce the most expensive service if it requires substantial manual operation. Conversely, a managed service can be wasteful when selected without understanding its charging units or the workload.

Separate cash expenditure from internal effort, and separate one-time costs from recurring costs. An internal engineer's time may not create an extra invoice, but it still consumes capacity that could be used elsewhere.

A worked three year comparison

The following figures are fictional euros for Meridian, with no discounting, tax, inflation, or vendor quotation implied.

Cost component	Custom portal	Purchased portal
Initial build or configuration	120,000	45,000
Initial migration and integration	35,000	55,000
Annual platform or subscription	24,000	54,000
Annual operations and support effort	60,000	30,000
Estimated exit work at year three	20,000	35,000

The simple three year total for the custom option is 120,000 + 35,000 + 3 × (24,000 + 60,000) + 20,000 = 427,000. The purchased option is 45,000 + 55,000 + 3 × (54,000 + 30,000) + 35,000 = 387,000.

The difference is 40,000 under these assumptions. It does not prove that buying is preferable. The purchased option may fail a mandatory requirement, have a weaker exit path, or save more time than the model captures. The custom option may require more uncertain maintenance effort. Show the assumptions and test sensitivity.

If the purchased subscription is 20 percent higher, its annual platform cost becomes 64,800 and the three year total becomes 419,400. If custom operations effort is 25 percent higher, the custom total becomes 472,000. These changes reveal which estimates deserve investigation.

Unit economics

A unit cost connects spending to a meaningful outcome, such as cost per completed service request or per completed work order. FinOps guidance uses unit economics to relate technology spending to business value and demand. [51]

For a fictional annual operating cost of 84,000 euros and 100,000 accepted service requests, average operating cost is 0.84 euros per accepted service request. This does not include the initial build or exit costs. Label the numerator and denominator so readers do not mistake it for full lifecycle cost.

Average and marginal cost differ. The next thousand service requests may cost little when capacity is idle, while a small increase at a threshold may require a larger database or another licence tier. Model the shape of the cost, not only one point.

Track the unit that matters to the decision. Cost per API request can improve while cost per completed work order worsens if the process creates more retries or manual work.

Benefits and uncertainty

A benefit model should connect an intervention to an outcome. If integration saves ten minutes on each of 5,000 work orders, the gross time saving is 50,000 minutes, or about 833 hours per year. It becomes a realised benefit only if the work is actually removed and the released capacity is used meaningfully.

Include adoption, process change, and failure handling. An automation that saves time on normal cases but creates difficult exceptions may yield less benefit than expected.

Use ranges or scenarios for uncertain inputs. Distinguish an estimate from a commitment and a forecast from an observed result. A precise spreadsheet is not evidence that the assumptions are precise.

Build versus buy

Buy when the capability is sufficiently standard, the product satisfies important requirements, and the operating and commercial terms are acceptable. Build when the capability needs distinctive behaviour, integration, control, or economics that available products cannot provide at acceptable cost.

A hybrid option is often credible: buy the CRM, build a narrow integration and experience layer, and use managed infrastructure. The key question is which responsibilities the organisation is prepared to own for years.

Evaluate configuration limits and upgrade behaviour. A heavily customised purchased product can become difficult to maintain, while a lightly customised one may not support the required process. Ask how customisations survive vendor releases.

Vendor due diligence

Assess functional fit, interface quality, identity integration, data export, resilience, security evidence, support, incident handling, pricing units, contract terms, and exit. Test the capabilities that are essential rather than relying entirely on demonstrations.

Ask for a representative export and verify whether another system can interpret it. Examine identifiers, attachments, historical states, and audit records. A CSV download that omits relationships or files may not be an adequate exit mechanism.

For an API dependency, investigate quotas, throttling, version changes, idempotency, sandbox behaviour, incident communication, and support escalation. A product with a capable interface can still be hard to integrate if the contract and operating process are weak.

Public-sector procurement and regulated environments may add requirements for accessibility, records, transparency, security, and procurement procedure. Obtain the organisation's applicable requirements and current advice rather than assuming the same process applies everywhere.

A decision matrix with limits

Define mandatory requirements first. Then compare eligible options using a small set of weighted criteria such as functionality, integration, operability, cost, and exit. Record the evidence and confidence behind each score.

Test sensitivity by changing plausible weights. If a small change reverses the winner, the decision is fragile and should be discussed. If one option wins across a broad range, confidence may be higher, subject to the quality of the evidence.

A matrix supports judgement; it does not transfer accountability to arithmetic. Record the decision maker and the risks accepted.

Exercises

18.1 Recalculate the purchased option if its annual support effort rises from 30,000 to 45,000 euros. State whether its total remains below the baseline custom option.

18.2 Design a cost model for the submission workflow. Include at least six cost drivers and identify which are sensitive to data volume.

18.3 A vendor is cheapest but cannot produce a usable complete data export. Explain how you would present the decision to the sponsor.

Key points. A useful cost model is transparent about scope and uncertainty. Procurement is an architectural decision because vendor contracts, interfaces, operating capabilities, and exit options shape the system.

CHAPTER 19

Enterprise architecture governance and migration

Learning objectives

Connect a solution to an application portfolio, build a feasible transition roadmap, and govern decisions without creating unnecessary delay.

The enterprise view

A solution can work well in isolation while making the organisation harder to operate. A new customer database may duplicate an existing source of truth. A new identity system may fragment access management. A local integration shortcut may make future changes expensive.

Enterprise architecture examines relationships across business capabilities, processes, information, applications, and technology. The aim is to coordinate change in support of organisational goals. It requires understanding both the current estate and the ability to move away from it.

A capability map describes what the organisation must be able to do without tying every capability to the current organisational chart or product. Overlay applications, costs, risks, and planned investment to identify duplication, gaps, and dependencies.

Application portfolio decisions

For each significant application, record business purpose, owner, users, lifecycle, cost, integrations, data, criticality, support status, and planned change. The inventory should be useful for decisions rather than merely complete.

Possible actions include retaining, improving, replacing, consolidating, or retiring an application. A retirement decision requires evidence that no essential process, integration, record obligation, or user group still depends on it.

Meridian may have three service management tools because of acquisitions. Consolidation could reduce cost, but each tool may support a different legal entity or specialised process. Investigate the differences before treating duplication as waste.

Frameworks and judgement

The TOGAF Standard provides an enterprise architecture method and related guidance. Its Architecture Development Method connects vision, business and information-system architecture, technology architecture, transition planning, implementation governance, and ongoing change, with requirements management across the process. The standard can be adapted to the organisation rather than followed as a paperwork exercise. [52]

ArchiMate is a modelling language for enterprise relationships. UML and C4 answer different software and system questions. Choose notation by the audience and purpose; a framework name does not make a model understandable.

Keep a distinction between principles, standards, patterns, and decisions. A principle is a broad direction, such as assigning clear data ownership. A standard is a required or preferred rule. A reference pattern is a reusable approach. A project ADR records a contextual choice.

Governance that helps delivery

Establish which decisions a team can make independently and which require wider review. Escalate choices that affect shared data, security boundaries, significant spending, or other teams. Do not require a central committee to approve every local implementation detail.

Provide an exception process with an owner, rationale, expiry or review date, and compensating measures. Without one, teams may hide deviations or delay essential work. An approved exception is still a risk to track.

A design review should focus on consequential assumptions and evidence. Use checklists to avoid omissions, but spend the discussion on the parts that could change the decision. Record unresolved issues and release conditions.

Current target and transition architectures

A target architecture describes a desired future state. A transition architecture describes an intermediate state that can actually be delivered and operated. Most large changes need several transitions because data, people, contracts, and dependencies cannot all change at once.

For Meridian, the first transition may improve duplicate-safe handoffs while leaving existing interfaces in place. The second introduces the portal for one business unit. The third consolidates analytics definitions. Only after evidence of successful operation does the organisation retire redundant forms.

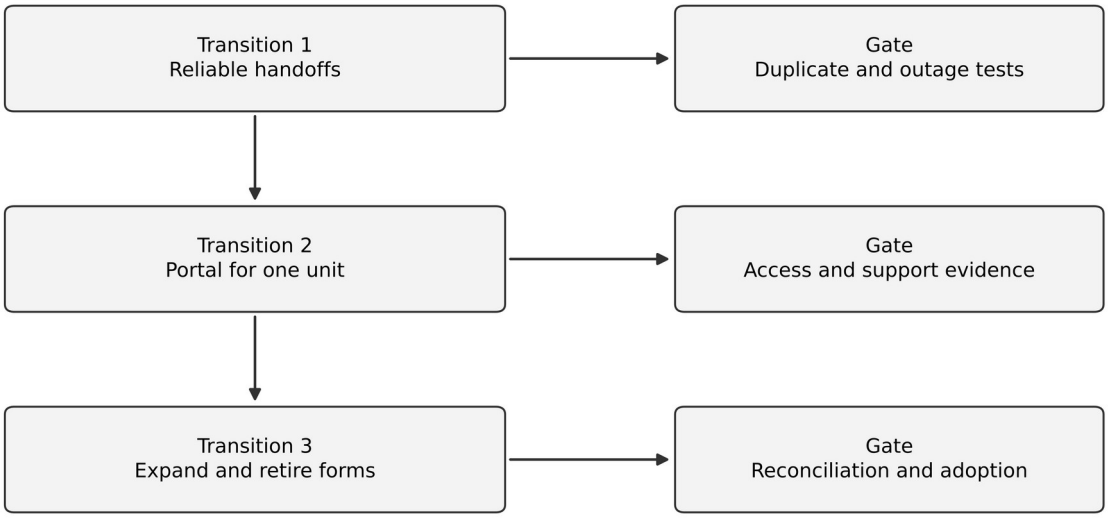


Figure 20 Migration proceeds through operable intermediate states with evidence gates

For each transition, name the capabilities delivered, dependencies, data changes, operating owner, success criteria, and rollback or recovery approach. A roadmap made only of dates and product names hides the work.

Migration patterns

A big-bang migration switches the system at one point. It can shorten the coexistence period but concentrates risk. A phased migration moves users, capabilities, or data gradually. It allows learning but requires coexistence and reconciliation.

The strangler fig pattern routes selected functionality to a new implementation while the old system continues serving the rest. Over time the old responsibilities can be removed. The routing and coexistence design are essential; the pattern is not simply rewriting code in small pieces. [53]

An anti-corruption layer can translate a legacy interface into a clearer internal model. It can make change safer, but someone must own it and avoid turning it into an undocumented permanent integration hub.

Data migration

Profile source data before mapping it. Identify missing values, duplicates, incompatible identifiers, invalid states, and historical exceptions. Agree transformation rules with business owners, and preserve an audit trail for consequential changes.

Rehearse the migration with representative data. Measure duration and validate counts, key coverage, relationships, and important aggregates. Record exceptions rather than silently discarding them.

Define how writes are handled during migration. A frozen source simplifies consistency but creates downtime. Change capture or dual operation can reduce downtime while increasing complexity. Dual writing to two systems introduces partial-failure risks and needs explicit reconciliation.

A rollback after new transactions have occurred is a business and data problem, not just switching a URL. Decide how new records will be preserved or reconciled before the cutover.

Adoption and retirement

Train users and support staff for the new workflow. Update procedures, permissions, and communication. A successful deployment with no adoption may fail the business objective.

Retirement includes disabling old access, stopping integrations and scheduled jobs, resolving retained records, cancelling unnecessary licences, and updating the inventory. Confirm that cost and risk are actually removed.

Exercises

19.1 Build a capability-to-application map for Meridian and identify one duplication that requires investigation before consolidation.

19.2 Propose three transition states for replacing a legacy submission form. Include a measurable gate for moving between states.

19.3 Explain why a technically successful data import is insufficient evidence that a migration is complete.

Key points. Enterprise architecture connects local decisions to shared capabilities and a feasible sequence of change. Good governance clarifies responsibility, while migration design makes intermediate states safe to operate.

CHAPTER 20

Architecture evaluation and evidence

Learning objectives

Evaluate an architecture against prioritised quality scenarios, distinguish evidence from assertion, design a useful experiment, and explain uncertainty without disguising it as a precise score.

Evaluation is an argument about a particular design

An architecture is not good in the abstract. It may support one workload, team, and business objective while being unsuitable for another. Evaluation asks whether a proposed arrangement can plausibly satisfy important requirements under stated conditions, and what evidence would reduce the remaining uncertainty. A checklist helps find omissions; it cannot supply missing evidence.

Begin with the decision to be made. For Meridian, the immediate decision is whether durable local acceptance plus a worker is an acceptable first release. It is not whether event-driven architecture is universally superior. Define the boundary: request receipt, CRM handoff, status projection, and operational recovery. Explicitly exclude unrelated enterprise systems from this review while recording the dependencies that cross the boundary.

The Architecture Tradeoff Analysis Method, or ATAM, uses business drivers, architecture descriptions, prioritised scenarios, and analysis of architectural approaches to identify risks, sensitivity points, and tradeoffs. A sensitivity point is a consequential architectural property for a quality response; a tradeoff point affects multiple qualities. The method makes the reasoning and assumptions visible. [54] The workshop below is a small teaching adaptation, not a claim to perform the complete formal method.

Build a quality tree that leads to tests

Start with an outcome such as reliable equipment support. Break it into qualities, then concrete scenarios. “Reliable” might include durable receipt during a CRM outage, restoration after database failure, and correct processing of retries. “Secure” might include tenant isolation, bounded administrative privileges, and controlled attachment handling. Prioritise scenarios by business importance and uncertainty or technical difficulty. Do not give every scenario the highest priority.

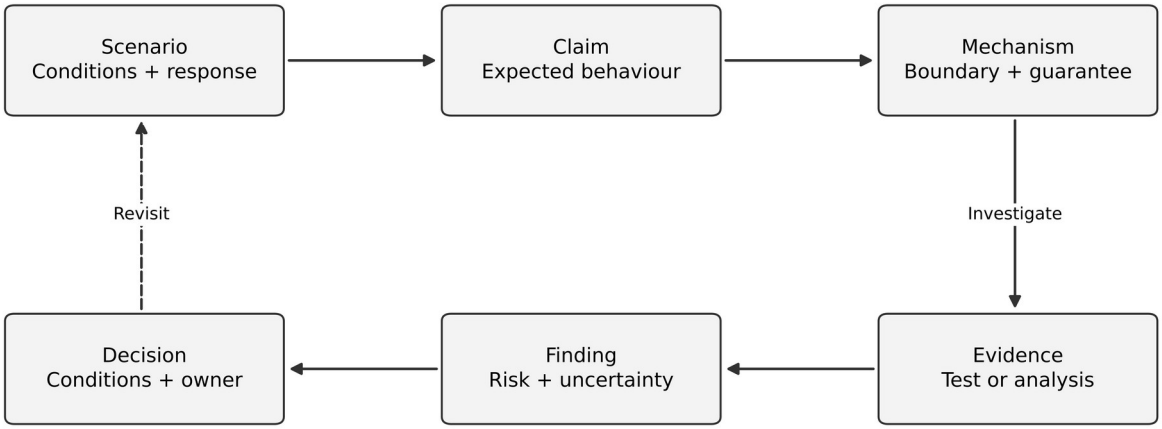


Figure 21 A requirement leads through a design argument to evidence and a decision

The figure distinguishes three objects often collapsed in reviews: the claim, the mechanism, and the evidence. The claim states the promised behaviour. The mechanism explains how the design should produce it. The evidence investigates whether the mechanism works within the declared scope. A test result must name the build, configuration, workload, failure injection, and observation point. An arrow from “test passed” to “release” is unjustified when the test never exercised the risky boundary.

For the CRM-outage scenario, follow the path from API validation to local transaction, outbox backlog, worker pause, controlled recovery, and reconciliation. Ask where data could be lost and how each accepted receipt will be accounted for. The response-time test and the recovery test need different observations. A receipt latency histogram cannot establish that all delayed handoffs eventually reached the CRM.

Identify sensitivity and tradeoffs in the case

The worker's maximum concurrency is a sensitivity point for drain time and vendor load. Increasing it can shorten recovery if the vendor has spare capacity, but can increase throttling if it does not. The local retention window for idempotency keys affects storage and the risk of long-delayed duplicates. Token-validation caching affects identity-provider dependency and the time for some changes to take effect. These are concrete parameters with consequences, not merely a list of desirable qualities.

Suppose three proposals are eligible: direct synchronous creation, local durable acceptance, and the vendor's own portal. First remove any option that cannot meet a mandatory requirement under its verified contract. Then compare the remaining options on operability, experience, cost, and reversibility. A weighted score must not compensate for a failed mandatory tenant-isolation requirement.

Review item	Claim or question	Evidence needed	Decision consequence
E01	One local record per scoped submission key	Concurrent duplicate and payload-conflict tests	Blocks release if invalid
E02	CRM effect can be resolved after timeout	Vendor contract plus uncertain-outcome test	Determines automation versus manual reconciliation

Review item	Claim or question	Evidence needed	Decision consequence
E03	Backlog recovers within agreed window	Measured service rate under vendor quota	Sets intake and recovery limits
E04	Staff cannot read outside authorised scope	Negative API, export, search, and attachment tests	Blocks release if invalid
E05	Purchased portal reduces total operating effort	Demonstration, operating trial, and cost assumptions	Informs option comparison

Treat the last entry differently from the first. A concurrency test can expose a specific correctness defect. Estimating multi-year operating effort has broader uncertainty and depends on adoption, staffing, incidents, and vendor changes. Report a range and the source of the assumptions.

Design an experiment that can challenge the claim

A useful performance question is: can this configuration meet the receipt target during the ten-minute peak while the CRM is unavailable? Independent variables include arrival rate, payload distribution, database size, and worker policy. Dependent variables include receipt latency, valid-request failure fraction, database saturation, and durable backlog. Controlled conditions include build, infrastructure, data seed, and competing workload.

Use arrivals that represent the scenario. In a closed-loop load generator, each simulated user waits for a response before sending again; when the system slows, offered load may fall. In an arrival-driven experiment, requests can continue arriving at the intended rate even when responses slow. State which model you used and whether the generator itself kept up. Otherwise a system can appear stable partly because the test stopped demanding work when it was overloaded.

Include warm-up separately from the measurement interval, and explain whether cold starts are in scope. Retain unsuccessful and timed-out requests. A latency report that includes only successful responses may hide the worst user outcomes. Repeat important experiments and report variation, rather than selecting the most favourable run. A p99 estimated from a tiny sample is unstable; include sample size and the percentile method.

An experiment does not prove unlimited scalability. If the tested configuration has 100,000 records, a warm cache, and one tenant, it does not establish behaviour with ten million records and a highly uneven tenant distribution. External validity concerns whether the result transfers to the intended environment. Internal validity concerns whether the observed difference can be attributed to the proposed cause rather than uncontrolled changes. Construct validity concerns whether the measurement represents the property you intended to study.

Quantitative evidence with uncertainty

Imagine a fictional test with 12,000 valid submissions and 18 receipt failures: the observed bad fraction is 18 / 12,000 = 0.0015, or 0.15 percent. This exceeds a 0.1 percent target in that interval. It is evidence of a problem under the tested conditions, not automatically the monthly production SLO result. Investigate failure distribution, generator behaviour, and repeatability.

If another run reports no failures, do not conclude that the true failure probability is zero. A finite observation has limited information, and requests may be correlated through shared infrastructure. Likewise, multiplying component availability figures is a model with assumptions, not an empirical availability measurement. Use numerical models to identify risks and design tests; keep model output separate from observed data.

A cost comparison needs similar discipline. Separate uncertain estimates from fixed contractual charges. Show which assumptions reverse the choice. If a vendor price advantage disappears with a modest change in operating effort, the decision needs better evidence about that effort. Avoid assigning precise probabilities to risks merely to make a spreadsheet produce one answer.

Read research as an architect

For a paper, identify its research question, model or method, main result, assumptions, and limitations. Then write a transfer statement: what does this result change in your design, and what remains outside its scope? Parnas helps reason about a module boundary but does not select your cloud product. A

consensus paper explains replicated agreement but does not settle your customer-facing compensation policy. A field study of another organisation may suggest a hypothesis without proving the same outcome for your team.

For a standard, locate the normative requirement relevant to your implementation and distinguish mandatory behaviour from optional features. For product documentation, record version, configuration, and scope. For a practitioner pattern, identify the problem it addresses and the extra responsibilities it introduces. Reading sources this way is more valuable than increasing the length of a bibliography.

Report a decision that can be reviewed

End the evaluation with a bounded conclusion: proceed, proceed with explicit conditions, investigate further, or reject the option. Record the architecture version, reviewed scenarios, evidence, material findings, owners, and closure criteria. Mark a finding closed only when the relevant evidence exists. “Team aware” is not a correction to a violated invariant.

For Meridian, a defensible conclusion is conditional approval of the local receipt architecture, with production release blocked until the CRM uncertain-outcome behaviour, tenant access rules, and backlog recovery have been verified. That conclusion respects the evidence available. It does not pretend that every implementation or future workload has been examined.

Exercises

20.1 A design review marks “uses a managed database” as evidence that the five-minute RTO is met. Explain the missing argument and specify a stronger evidence package.

20.2 A fictional test accepts 20,000 eligible requests and records 26 failures. Calculate the observed success percentage. Explain what else is needed before comparing it with a monthly 99.9 percent SLO.

20.3 Design an evaluation of two handoff approaches. Include two mandatory constraints, three prioritised scenarios, one sensitivity point, one possible tradeoff, and a decision rule that admits uncertainty.

Key points. Evaluation connects requirements, mechanisms, and evidence under explicit assumptions. A strong conclusion states its scope and the observations that would change it. The next four chapters use this discipline to develop different architecture cases.

CHAPTER 21

Worked architecture customer portal and enterprise integration

Learning objectives

Construct a consistent set of architecture views, specify a durable submission and integration contract, and trace security, failure, and operational responsibilities through the design.

Start with ownership and the user promise

Meridian's business problem is unreliable movement of equipment service requests between customer channels, support staff, CRM, and ERP. Duplicate entry creates conflicting records, while missing acknowledgements cause customers to submit again. A new interface alone cannot resolve uncertain handoffs or conflicting data ownership.

The first-release promise is deliberately narrow: after a valid receipt, the portal has durably recorded the request and taken responsibility for resolving its handoff. A receipt does not confirm an approved service case, a technician visit, a parts reservation, or a completed repair. The user sees “received” followed by a separately sourced business status. If synchronisation is delayed, the portal displays that fact rather than inventing a current CRM state.

The planning volume, peak, staffing, and targets are those in the opening brief. A thirty-minute CRM outage is a resilience scenario, not permission to ignore local database failure. The portal's monthly receipt SLO applies to a defined eligible-request population; sign-in, uploads, and status freshness require their own indicators. Keep those denominators separate in the architecture dossier.

Data authority is assigned by fact

Fact	Authoritative owner	Portal treatment	Conflict rule
Customer organisation and service entitlement	Approved ERP/customer-master process	Versioned local reference or authorised lookup	Escalate disagreement; do not silently grant entitlement
Asset identity and contract facts	ERP	Read model with source version and freshness	ERP change supersedes the projection
Initial request receipt	Portal	Durable record and scoped idempotency result	Same key and payload returns the same receipt
Support case and business progress	CRM after confirmed handoff	Customer-facing status projection	Reject stale updates using source version where supported
Work order and fulfilment facts	ERP	Selected reference and status	ERP owns scheduling and completion
Credential and session state	Identity systems	Validated principal and bounded session	Follow identity and revocation policy

A local copy is not automatically a second authority. It needs source identity, update rules, freshness, and an explicit response when the copy is too old. If entitlement cannot be established during ERP failure, the portal may accept a request for later review without promising covered service. That alternative requires a different user message and business approval. It must not silently approve an expensive repair from stale entitlement data.

Container view and responsibility map

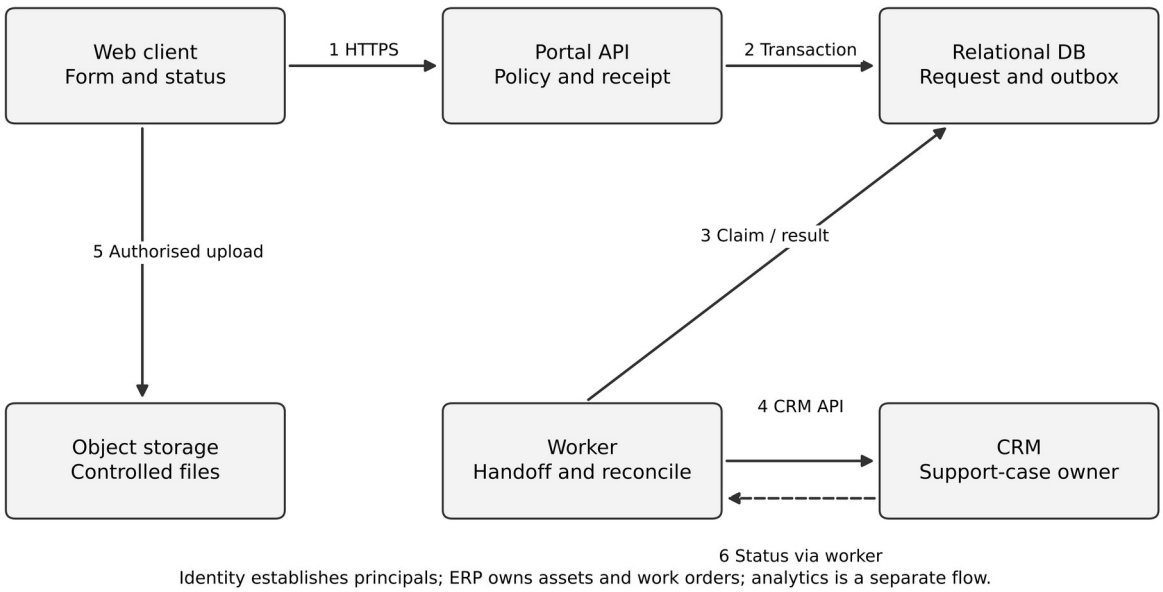


Figure 22 Portal runtime responsibilities and the boundary between local acceptance and external systems

The web client presents forms and status. The portal API authenticates the principal, authorises the resource scope, validates the submission, and commits the receipt with its handoff intent. The worker claims durable work and talks to the CRM through an adapter. Object storage holds attachments in controlled states; the relational database holds metadata and workflow records. CRM and ERP remain

separately operated systems. The identity provider participates in sign-in and token or session establishment; it is not a box through which all business data must pass.

Numbered relationships describe the design: 1 is the browser's protected application request; 2 is a short local database transaction; 3 is a worker claim and result update; 4 is an authenticated CRM API call; 5 is the controlled attachment route; 6 is a versioned status update or reconciliation fetch. Exact transport and authentication choices are implementation contracts, not assumptions created by the arrows. The example uses HTTPS for external APIs and a protected database connection.

No broker is required to make the initial design asynchronous. The database outbox can be polled by a worker at this scale if measurements support it. A broker could later provide distribution, buffering, or several independent consumers; it would add another durability and operating boundary. Adding a queue does not remove the outbox's purpose when the database and broker cannot commit atomically.

Component view inside the portal API

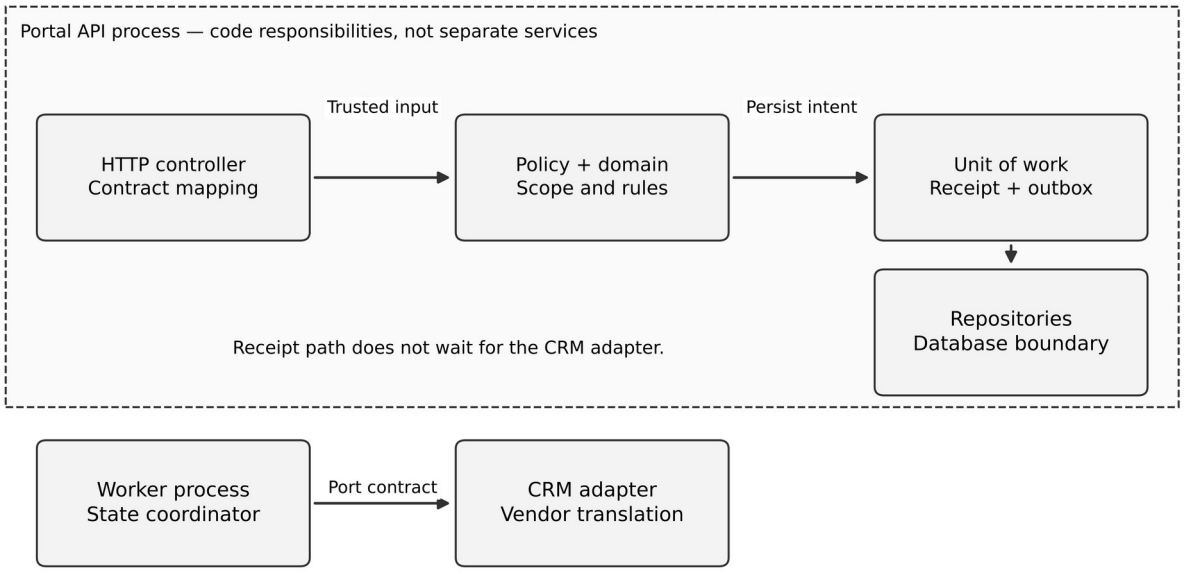


Figure 23 Components inside one portal API process separate policy transaction and vendor concerns

The controller translates the HTTP contract. The policy component establishes trusted tenant and actor scope. The submission service enforces the workflow and invokes a unit of work. Repositories persist the request, idempotency record, and outbox intent in one transaction. The CRM adapter belongs to the worker path; the receipt transaction does not wait for a remote CRM response. These components are code responsibilities inside runtime units, not five new microservices.

This separation makes two kinds of change easier to reason about. A new vendor field should primarily affect the adapter and its contract tests. A new rule about who can submit for an asset affects policy, domain behaviour, and relevant acceptance tests. If the controller contains both vendor mapping and access policy, those changes become coupled and harder to verify.

The transaction boundary should protect a precise invariant: every successfully accepted request has one original receipt result and an initial durable handoff intent. A unique tenant-and-key constraint resolves concurrent use of the same key. Store the authorised owner and a canonical payload digest, compare them on reuse, and reject a conflicting payload. File bytes need their own staged workflow because object storage is outside this database transaction.

The successful sequence

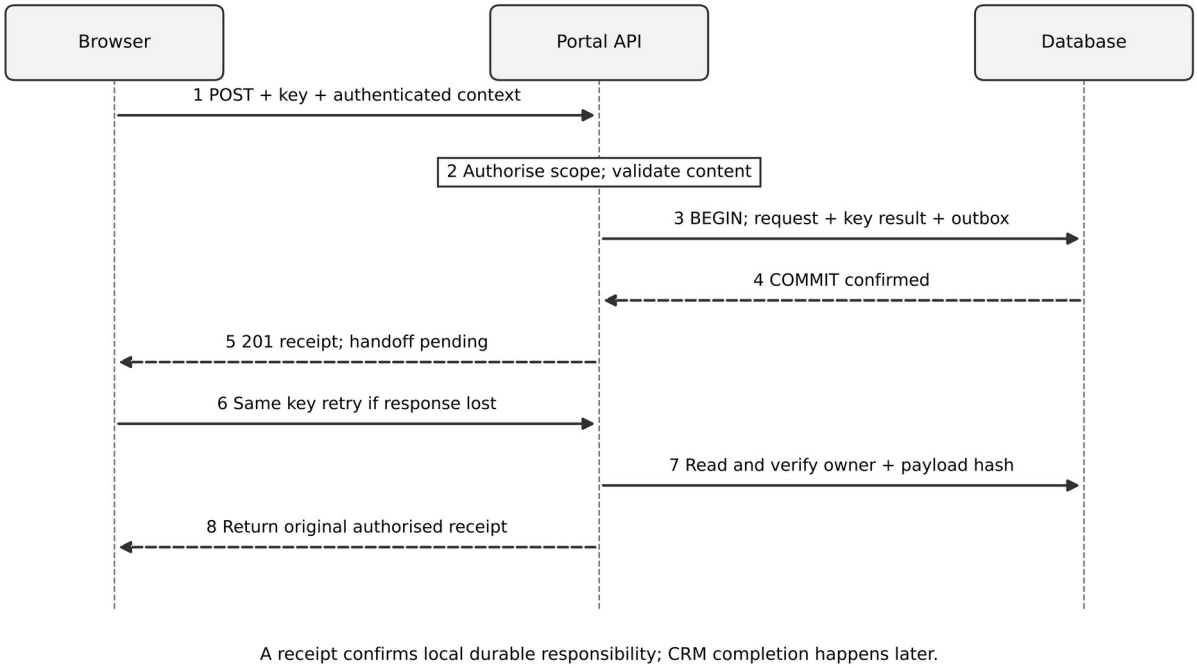


Figure 24 A successful receipt sequence ends at local durable acceptance

The browser submits an authenticated request containing an idempotency key and an attachment reference if applicable. The API derives scope from trusted policy, validates the content, and begins a database transaction. The database records the submission, request hash, receipt result, and outbox command. Only after commit does the API return the receipt. The example response is 201 Created for the newly created local service-request resource; its representation separately reports handoff_state as pending. A 202 contract would also be reasonable for a differently defined resource, but the meaning must be explicit.

If the response is lost, the browser repeats the same key and content. It receives the existing authorised result. If the key is reused with changed content, the API rejects the conflict. If the browser generates a new key for each retry, the contract cannot identify the retry as the same operation. Client behaviour is therefore part of the integration test.

The worker later reads the outbox. It claims work with a lease or another supported atomic mechanism, sends a stable external reference, and records the CRM result. A lease reduces simultaneous work but does not prove remote uniqueness. A worker can pause after its lease expires while another worker takes over; the external operation still needs a duplicate-safe contract or reconciliation.

The uncertain-outcome sequence

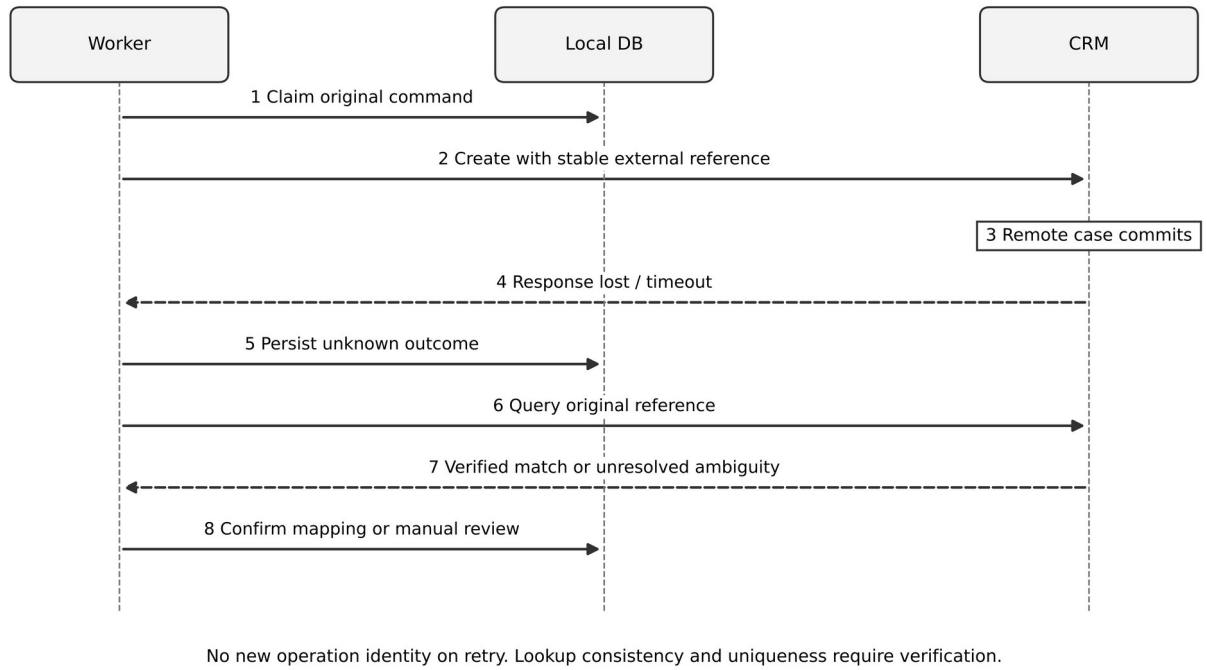


Figure 25 A lost CRM response requires reconciliation before an unsafe repeat

The difficult case is not a clean rejection. The CRM may create a case and then lose its response. The worker observes a timeout and cannot distinguish that outcome from failure before creation. Mark the outcome unknown, retain the original command identity, and query the external reference according to the vendor contract. If the vendor enforces unique references or supports a durable idempotency mechanism, a safe retry can reuse that identity. If lookup is non-unique or stale, the design must not claim exact duplicate prevention.

For the provisional vendor, uniqueness is unconfirmed. The first-release dossier therefore contains a release condition: verify the guarantee or obtain business acceptance of bounded manual reconciliation and residual duplicate risk. A single worker reduces one race but does not resolve a crash after remote commit. The architecture must describe what support does when lookup returns zero, one, or several possible matches.

Zero matches are not automatically proof of no remote effect if the lookup is eventually consistent. One verified match can establish a mapping when identifiers and content agree. Several matches require a controlled duplicate-resolution process. Store evidence of the resolution and avoid destructive cleanup that might remove a legitimate business record.

Attachments and trust boundaries

Treat upload as a stateful process: authorised intent, bounded upload, verification, quarantine, inspection, and controlled release. An uploaded object is not automatically available to staff. The request record points to metadata that states the inspection outcome and access scope. Object names should not be treated as credentials or used to infer the tenant without checking authoritative metadata.

A request may be accepted while an attachment remains quarantined if the business workflow supports that state. The interface must show the difference. Expired upload intents and unattached objects need cleanup. Rejected objects need a retention and investigation policy. Signed upload or download URLs, if used, should be narrowly scoped and short-lived under the selected storage contract.

The customer-to-edge boundary, runtime-to-data boundary, and portal-to-vendor boundary have different threats. The API must check resource authorisation even if the gateway validated a token. The worker needs only the vendor permissions required for its handoff. Administrative recovery credentials should be separate from ordinary workload identities. Attachment inspection runs with restricted privileges because the file is untrusted input.

Deployment and failure domains

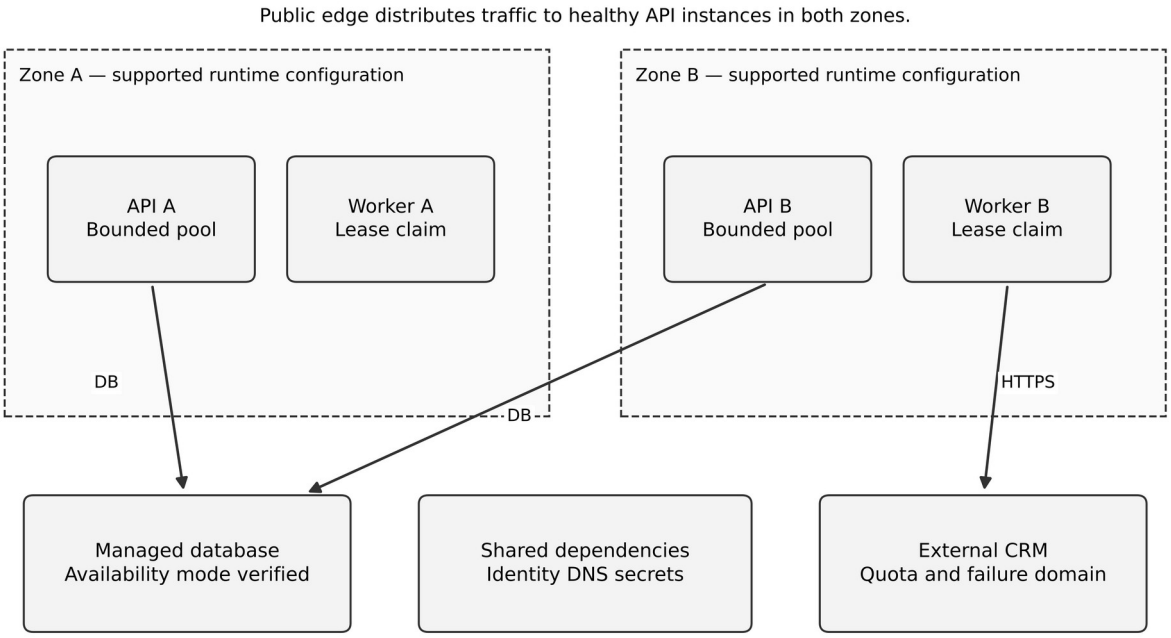


Figure 26 Two runtime zones share explicit data identity and delivery dependencies

The neutral deployment places API instances across two supported zones and provides separately scalable workers. The database uses a managed availability configuration selected and tested for the target service. A diagram showing a database icon across zones does not specify its actual replication or failover guarantee. Verify those details, including client reconnection and any effect on in-flight transactions.

Shared dependencies are shown deliberately: identity, name resolution, database access, secrets, deployment configuration, and the external CRM. Zone redundancy does not protect against a defective release deployed everywhere or a revoked credential used by all workers. Regional disaster recovery is a separate plan with recoverable data, keys, infrastructure definitions, routing, and operator access.

An Azure implementation can evaluate Azure Container Apps, Azure Database for PostgreSQL, Blob Storage, Key Vault, and Azure Monitor. An AWS implementation can evaluate ECS with Fargate, RDS for PostgreSQL, S3, Secrets Manager, and CloudWatch. These reuse the categories introduced in Chapter 13; they are candidates for investigation, not a feature-equivalence or availability claim. Compare region support, private connectivity, identity integration, recovery, scaling, and costs before selecting a configuration.

Capacity and recovery reasoning

At 20 requests per second for 600 seconds, the peak produces 12,000 requests. The annual average is approximately 100,000 / (365 × 24 × 3,600) = 0.00317 requests per second. The difference demonstrates why annual volume is not a peak capacity specification. The peak is an exceptional fictional incident and must not be assumed to continue for the whole year.

If no handoffs complete during that peak and 12,000 remain when the vendor returns, a worker capacity of 25 handoffs per second with ongoing arrivals of 5 per second yields a net drain of 20 per second. Clearing that backlog takes 600 seconds under constant-rate assumptions. Vendor quotas, retries, and variable service times can increase it. The one-minute normal freshness target cannot apply unchanged during this declared outage; define a separate recovery objective and show degraded status.

At the maximum attachment size, 12,000 files of 10 MB each represent 120,000 MB, or 120 GB in decimal units, before replicas, metadata, retries, or versions. This is a worst-case peak input assumption, not expected storage use. Estimate actual file-size distribution, attachment frequency, retention, scanning capacity, and transfer cost before sizing storage or network links.

Release evidence and unresolved decisions

A minimum release review needs scoped authorisation tests, duplicate-key concurrency tests, a lost-response integration test, attachment-state tests, a peak-load report, a CRM-outage recovery exercise, and a restore rehearsal. It also needs operational ownership: who investigates failed handoffs, how a customer is informed, and how every accepted receipt is reconciled.

The unresolved questions are consequential: vendor uniqueness and rate limits, entitlement freshness, retention policy, workload shape, exact recovery objectives, and cloud configuration. Record owners and evidence deadlines. A complete architecture can contain unresolved assumptions if they are visible and governed; it cannot present them as achieved guarantees.

Exercises

21.1 Trace one receipt across the context, container, component, data, sequence, and deployment views. Identify a property that each view cannot establish by itself.

21.2 A backlog contains 18,000 handoffs. The vendor permits 30 successful handoffs per second and ongoing arrivals are 12 per second. Calculate the ideal drain time and identify three reasons the observed time could be longer.

21.3 The sponsor requires customers to submit during ERP failure while forbidding unverified service entitlement. Propose a contract and state model that can satisfy both, or explain the additional decision required.

Key points. A detailed architecture explains responsibility at every boundary: what success means, who owns each fact, what can fail, and how uncertainty is resolved. Consistency between views matters more than the number of boxes.

CHAPTER 22

Worked architecture commerce inventory and payments

Learning objectives

Define invariants across order, inventory, and payment boundaries; distinguish compensation from rollback; and evaluate a modular design against a distributed alternative.

A different problem from the portal

Aster Retail is a fictional retailer selling physical goods through a website. Its initial assumptions are 2,000 orders per day, a promotion peak of 50 checkout attempts per second for five minutes, two warehouses, and an external payment provider. The first release has one fulfilment region and a team of eight engineers. These planning values are not industry benchmarks. The user journey includes browsing, pricing, inventory reservation, payment authorisation, order confirmation, capture, shipment, and possible refund.

The business states three critical rules: a confirmed order must have an accountable fulfilment path; the system must not knowingly sell more units than its available-to-promise policy permits; and a repeat of the same checkout operation must not cause an unintended second charge. Define these precisely before selecting services. “No overselling” is incomplete without deciding whether stock is shared with stores, whether backorders are permitted, and which system owns reservations.

Unlike Meridian's receipt, a commercial order confirmation may create financial and fulfilment commitments. Showing “order confirmed” after merely placing a message on a queue could mislead both the customer and downstream operations. Define “checkout received,” “payment pending,” and “confirmed” as different states when they imply different obligations.

Boundaries and the first architecture

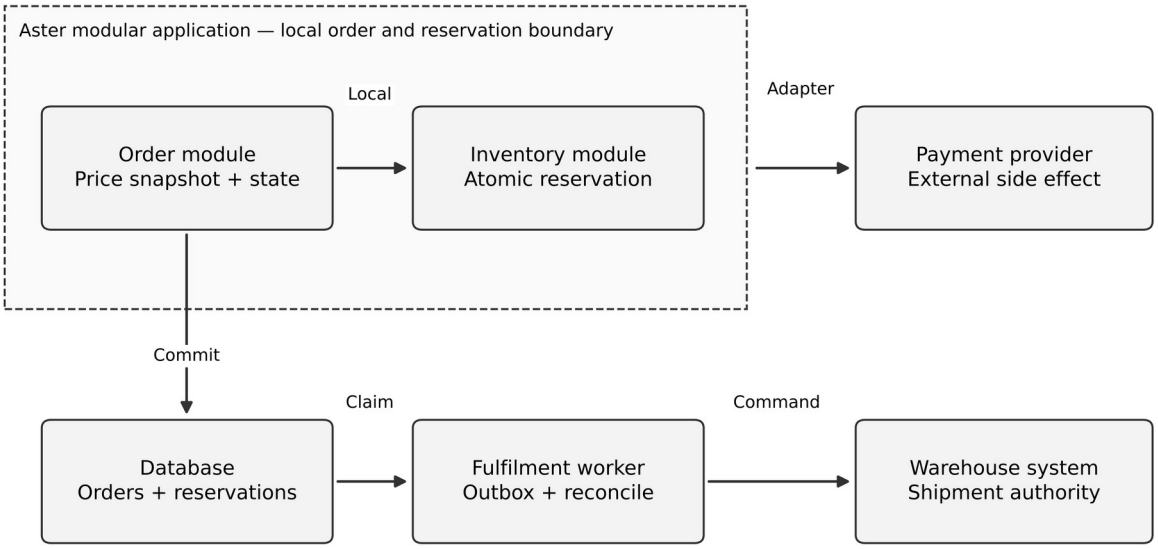


Figure 27 Commerce responsibilities with explicit local and external transaction boundaries

The proposed first release uses a modular application for catalogue access, pricing, orders, and reservations, with clear internal interfaces and controlled table ownership. A payment adapter isolates the external provider. A fulfilment worker communicates with the warehouse system. Analytical feeds are outside the checkout response path. A cache can accelerate catalogue reads but does not become the authority for scarce inventory.

Keeping orders and reservations within one relational transaction boundary simplifies the initial inventory invariant. It is not a claim that every retailer should use one database. If separate teams, regions, or warehouse ownership later require independent inventory services, the architecture must explicitly replace the lost local atomicity with a reservation protocol and recovery model. Extracting a service changes guarantees, not merely deployment packaging.

For the first release, the inventory module maintains stock, active reservations, and committed allocations. Define available-to-promise according to an approved rule, such as usable stock minus active reservations and already committed allocations, without subtracting the same units twice. Returns, damaged stock, and external sales channels need authoritative updates. A stock number that ignores another sales channel cannot uphold a global no-oversell promise.

Protect the stock invariant atomically

A simplified reservation table can use an atomic conditional update on the authoritative stock row. Conceptually, reduce available units by q only where available units are at least q. In the same transaction, record the reservation with a unique operation identifier. If the update affects zero rows, determine whether the item is missing or insufficient stock remains. Do not first read stock, decide in the browser, and then write an unconditional new balance.

With one unit available, two concurrent customers may both see one unit. Only one conditional reservation should commit. The losing request receives an honest out-of-stock or retry response according to the contract. If the business permits backorders, model them explicitly instead of allowing the invariant to be violated accidentally.

For multiple order lines, declare the policy. An all-or-nothing basket can reserve all required rows in one local transaction, using a consistent lock order and handling deadlocks or serialization failures. Partial fulfilment needs a different state model and customer agreement. A distributed inventory design may reserve each line separately and release successful reservations if another line fails. That is additional workflow, not equivalent to one atomic transaction.

Payment authorisation is an external side effect

Authorisation, capture, settlement, void, and refund are distinct concepts whose exact behaviour depends on the provider contract. Do not infer them from a generic “payment complete” field. The teaching design assumes a provider with documented authorisation and capture operations, stable operation identifiers, a query mechanism, and a specified idempotency window. These are requirements to verify, not claims about an unnamed real provider.

The customer completes checkout through a provider-approved payment flow. The application records a token or reference suitable for its contract and avoids collecting raw payment credentials in the course model. The server determines the trusted amount and currency from the agreed price snapshot. It must not accept a client-supplied total as authoritative.

An authorisation timeout creates uncertainty: funds might have been authorised even though the application did not receive a result. Reuse the original operation identity and query or retry according to the provider's verified semantics. A new random identity on each retry defeats duplicate protection. Do not release a reservation and label the order cancelled solely because the authorisation response timed out; first establish the outcome or enter a visible resolution state.

A saga and its compensation policy

The original saga work examines long-lived transactions composed of smaller transactions with compensating actions when completion cannot proceed. [55] In modern service workflows, the important architectural point is that compensation is an explicit business action with its own failure modes. The teaching workflow below uses the pattern under a declared provider contract; it is not a distributed database rollback.

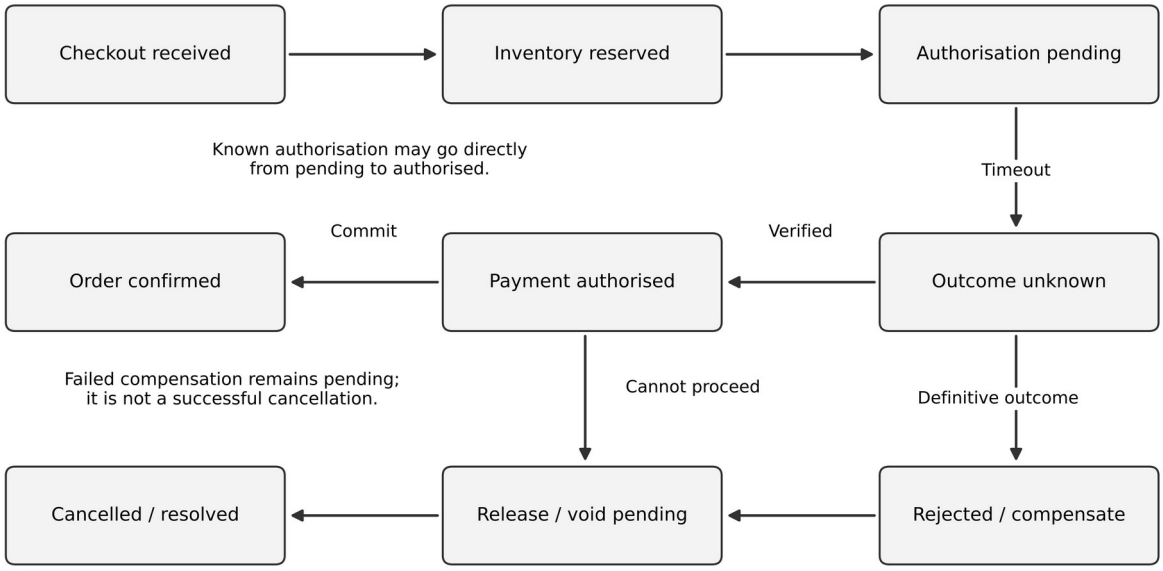


Figure 28 Order coordination separates successful progress uncertainty and compensation

The normal path is checkout received, inventory reserved, payment authorised, order confirmed, and fulfilment requested. A confirmed order requires both the relevant reservation and verified payment authorisation in this case. Capture timing is a business and provider decision; the diagram does not assume capture at checkout. After shipment, a return and refund belong to a separate lifecycle rather than reversing history.

If authorisation is definitively rejected, release the reservation and close the attempt as rejected. If it is unknown, hold a bounded pending state and reconcile. If an authorised order cannot proceed before fulfilment, request a void or other provider-supported reversal, then release inventory when policy permits. A failed compensation leaves work outstanding. It must remain visible, with an owner and a retry or manual process.

Each step needs a durable identity, expected predecessor, outcome, attempt record, and next action. The coordinator can be a database-backed state machine inside the modular application. An orchestration engine might help when the workflow becomes long-running and complex, but it does not invent correct compensation semantics. The payment and inventory contracts still determine what actions are safe.

Reservation expiry is a concurrency problem

Suppose a reservation expires at 12:05, and a payment confirmation arrives at the same instant that an expiry worker runs. The system needs an atomic state transition that chooses a valid outcome. If expiry wins, a later payment result cannot blindly confirm the order against released stock. It may attempt a fresh reservation or enter compensation and customer communication. If confirmation wins, expiry must observe that the reservation is no longer releasable.

Use a version or compare-and-set condition on the reservation state, with the order's transition protected within the chosen local transaction boundary. Store time in a well-defined representation, but do not assume a timestamp alone resolves simultaneous actions. Background jobs can be late; clients can retry; provider notifications can arrive out of order.

A useful invariant is that each reservation is in exactly one accountable state: active, committed, released, or expired, with allowed transitions defined. The inventory balance must change once for each applicable transition. A duplicate release message should return the recorded outcome without adding stock again. Lab 9 explores the same principle with a small deterministic state machine.

Events facts and customer communication

An OrderConfirmed event should state that the defined confirmation conditions were met. It should not be emitted merely because the HTTP handler began processing. A downstream warehouse consumer can record the event in an inbox with its local work item, but the warehouse API may still need idempotency or reconciliation. The external side-effect boundary repeats at every new system.

Choose an ordering key aligned with the aggregate, such as order identifier. A customer can have several independent orders, so ordering every event for all customers through one partition adds a bottleneck without an established need. A source version can help a projection reject stale updates. Refund events do not simply replace the order's historical paid amount; the data model should preserve the financial event and its relationship.

Customer messages must follow verified state. “We received your checkout and are confirming payment” differs from “Your order is confirmed.” If an outcome remains unknown, support needs the order identifier, payment operation reference, reservation state, and last reconciliation result. Do not ask the customer to repeat payment while the first attempt might have succeeded unless the provider flow explicitly makes that safe.

Performance and failure evaluation

At 50 attempts per second, a five-minute promotion offers 15,000 attempts. The normal daily volume cannot be used as the capacity limit. Separate catalogue traffic from checkout traffic; caching the catalogue may do little for contention on one popular stock item. Measure hot-key behaviour, transaction retries, payment-provider latency, and the ratio of attempts to confirmed orders.

A payment-provider outage can leave browsing available while checkout is pending or unavailable. Decide whether to accept bounded pending orders, how long to reserve inventory, and when to stop intake. Unlimited reservation accumulation can make all stock appear unavailable even while customers have not completed payment. The fallback is therefore a business policy, not merely a circuit-breaker setting.

Test concurrent purchase of the last unit, duplicate checkout keys, timeout after authorisation, duplicate provider notifications, late confirmation after reservation expiry, compensation failure, and worker restart before acknowledgement. Measure correctness and customer-visible outcomes as well as throughput. A fast checkout that charges twice fails the design.

Compare a distributed alternative

Separate inventory, orders, and payments when independent ownership, scale, security, or release needs justify the extra interfaces. The design then needs explicit reservation expiry, idempotency, correlation, event contracts, and operational responsibility across teams. Shared identifiers do not create a shared transaction. A synchronous chain across services may combine their latency and availability limitations without giving meaningful independence.

An acceptable decision record can retain the modular design now while defining extraction conditions: inventory traffic exceeds a measured contention limit, warehouse ownership changes, or an independent team requires a different release cadence. Keep internal boundaries clear so later extraction is feasible. Do not promise that extraction will be free; data migration and contract changes still have costs.

Exercises

22.1 Explain why a payment timeout cannot be treated as a declined payment. Specify the information needed to resolve it safely.

22.2 One stock unit is available. Two customers concurrently reserve it. Describe an atomic implementation and show the allowed outcomes. Then explain what changes if inventory is owned by an external service.

22.3 Design the state transitions for a late payment authorisation arriving after reservation expiry. Include customer communication, duplicate handling, and a failed compensation path.

Key points. Commerce architecture must preserve business and financial meaning through concurrency and uncertain external outcomes. Compensation, expiry, and customer communication are part of the architecture, not exceptional details to add after the happy path.

CHAPTER 23

Worked architecture a governed analytical platform

Learning objectives

Design an analytical platform from decision needs and declared grain, handle late and corrected data, and connect lineage, security, and operational evidence to published metrics.

Begin with the decision the data must support

Beacon Analytics is a fictional enterprise data programme combining service, sales, and fulfilment information. The first users are operations managers who need a daily view of service lead time and order completion, and analysts who investigate exceptions. They do not initially require a subsecond dashboard. The assumed first release processes up to one million new or changed source records per day from three source systems, with a 07:00 local-business-time reporting deadline.

Define each metric before designing a pipeline. “Average service time” can mean time from receipt to first response, approval to scheduling, or receipt to closure. Each measures a different process. Define the population, event boundaries, timezone policy, excluded cases, treatment of reopened work, and whether unresolved requests appear. A dashboard that omits all unresolved cases may show improving completion time while a large backlog grows.

Treat freshness and correctness as separate qualities. A report can be fresh but wrong because a join multiplies rows. It can be correct for yesterday but misleading if the timestamp is hidden. The service contract should state both the publication deadline and the completeness or correction policy.

Architecture and data zones

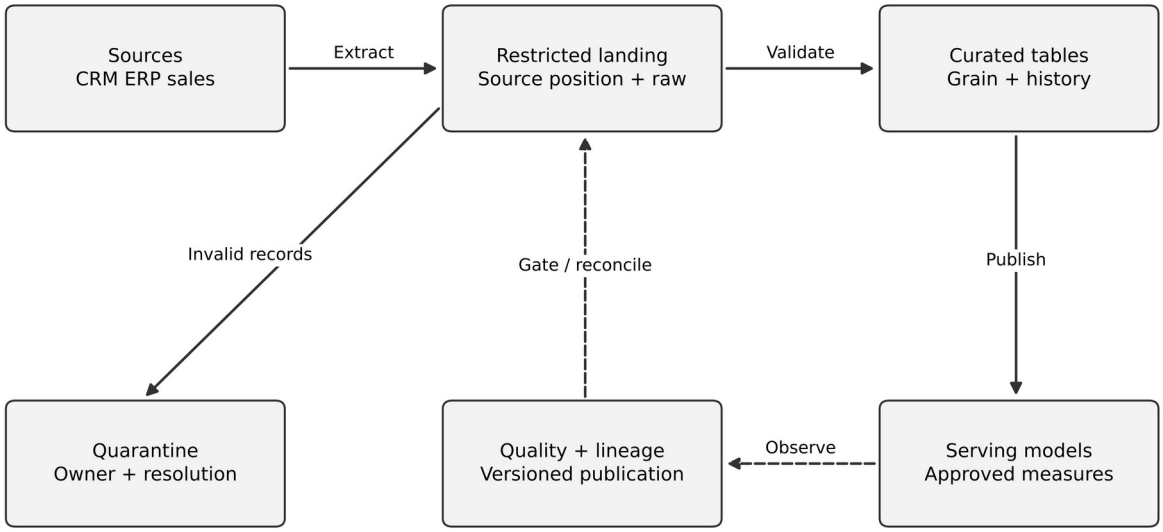


Figure 29 Source changes pass through controlled raw curated and serving responsibilities

Source connectors obtain authorised exports or change records. A restricted landing area preserves enough source evidence for replay under an approved retention policy. Validation checks schema, identifiers, and basic constraints. Curated tables apply business definitions, deduplication, history, and quality rules. Serving models expose approved measures to dashboards and analytical users. A catalogue records ownership, definitions, and lineage; it is not a replacement for access enforcement.

The figure's arrows indicate data movement, with separate control and observation paths. Raw input is not directly available to every analyst. Quarantine stores records that cannot safely enter the published model, with an owner and a resolution path. A failed daily run should not silently replace a valid report with an incomplete one. Publication needs an explicit gate.

ETL can minimise the spread of sensitive source fields by transforming before loading into a broadly accessible target. ELT can preserve flexibility and use target compute effectively, but creates a raw-data access and retention responsibility. The design chooses based on data classification, source limits, transformation needs, and cost. The labels alone do not determine quality.

Grain and measures

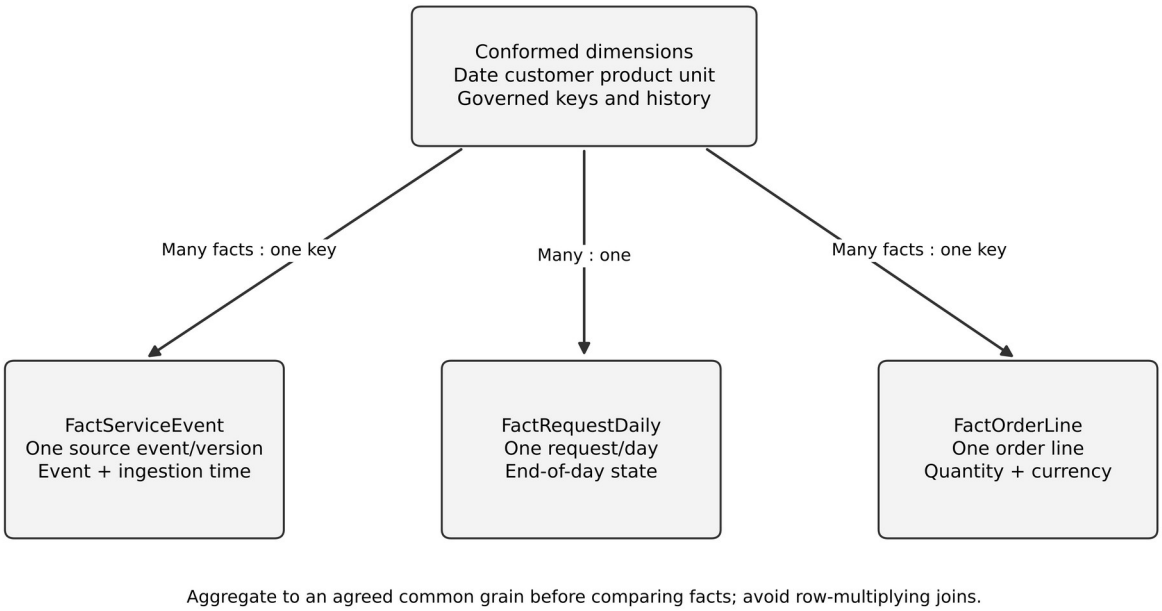


Figure 30 Analytical facts at different grains share conformed dimensions without being joined blindly

Define FactServiceEvent as one row per accepted source event identifier and source version for one service request. It records request key, event type, event time, ingestion time, and relevant dimension keys. Define FactRequestDaily as one row per request per business day, with an agreed end-of-day state. Define FactOrderLine as one row per order line, with quantity and monetary measures in a stated currency. These facts answer different questions.

Conformed dimensions use shared definitions where the business has agreed them: date, customer organisation, product, and business unit. Shared dimensions do not mean every fact can be joined directly at row level. Joining five service events to three order lines for one customer can create fifteen rows and duplicate measures. Aggregate each fact to the intended common grain before comparison, and verify whether the relationship is meaningful.

An additive measure can be summed across appropriate dimensions, such as order-line quantity when units are compatible. A balance is often meaningful at a point in time and should not be summed across daily snapshots as though each day were a new transaction. A ratio should generally be recomputed from its numerator and denominator at the requested aggregation level, rather than averaging precomputed percentages without weights. Declare units and currencies alongside the measure.

Historical context and corrections

Suppose customer C17 moves from business unit North to Central on 1 June. A report asking who owned the customer when a request was handled needs historical assignment; a current responsibility list needs the latest assignment. Both are valid, but they require different queries. A history-preserving dimension can assign a surrogate key to each interval and retain the business identifier. Check that intervals do not overlap for the same governed key and that event-time lookup follows the approved policy.

Late data complicates the join. An event from 30 May arriving on 3 June should not automatically inherit the current June assignment if the report is historical. If the source later corrects the effective date, the platform needs a correction mechanism and an audit trail of republished results. An immutable record of ingestion can coexist with corrected curated tables; immutability is not a reason to publish known errors forever.

Distinguish an update from a duplicate. Repeated delivery of the same event identifier and version should not add a second fact. A higher source version may supersede a previous value under a defined rule. A deletion or withdrawal has its own semantics; it may require removing a personal-data copy, reversing a business measure, or preserving a non-identifying audit fact. The approved lifecycle policy determines the action.

Event time processing time and publication

Event time describes when the business event occurred according to its source. Ingestion time describes when the platform received it. Processing time describes when a particular computation ran. The source clock, timezone, and correction policy affect event-time interpretation. Store enough information to distinguish these meanings rather than placing them all in a field named date.

For the first release, imagine a daily publication at 07:00 with a declared two-day correction window for ordinary late arrivals. The previous two business days are recomputed as new events arrive; older corrections enter an exception process and can trigger a versioned republication. This is a fictional operating policy, not a universal recommendation. It trades processing cost and apparent report stability against completeness.

Label provisional and finalised reporting intervals clearly. A watermark is a processing judgement or progress indicator under a stated policy; it is not proof that the universe contains no older event. If the business requires corrections after closure, preserve a revision identifier and explain why a previously published total changed.

A worked double-counting and lateness example

Three fictional events report completed work: e1 has value 100 on day 1, e2 has value 50 on day 1, and e3 has value 75 on day 2. The ingestion stream repeats e1 once and delivers e2 after the initial day-1 report. Summing arriving rows without deduplication produces 325 across the stream instead of the correct 225 across unique events.

The initial day-1 publication may contain only 100. After e2 arrives, the corrected day-1 value is 150. Day 2 contributes 75. The platform should explain both the historical incompleteness and the correction, rather than calling the first value final or adding the late event to day 2 merely because it arrived then. Lab 10 implements this example and a versioned correction.

For money, the runnable lab uses integer minor units to avoid a binary floating-point representation becoming part of the example. A production model must also handle currency, rounding policy, tax, exchange-rate timing, and the source's accounting semantics. A correct sum of incompatible currencies is not a meaningful financial measure.

Quality gates and reconciliation

Use layered checks. Structural checks detect missing fields or incompatible types. Relational checks detect orphan identifiers. Domain checks detect impossible transitions or negative values where they are forbidden. Distribution checks investigate unusual shifts without assuming every unusual value is wrong. Reconciliation connects the published model back to the source under a declared extraction boundary.

Counts alone are insufficient. Compare distinct key sets, version coverage, totals by meaningful groups, and selected record-level values. A missing row and a duplicate can cancel in the total count. A checksum is useful only if both sides use a defined canonical representation and the comparison scope is understood. Do not publish sensitive raw values in a reconciliation log merely to make debugging convenient.

A daily run should retain source position, transformation revision, schema version, quality results, input and output counts, publication identifier, and exceptions. A failure should leave the last valid publication available with a visible freshness warning when appropriate. The owner must decide whether partial publication is acceptable for a particular report; engineers should not make that business decision silently.

Security and lineage through derived data

Access rules must follow data into extracts, caches, notebooks, dashboard exports, and any search or vector index. Removing direct identifiers does not necessarily anonymise a dataset. Aggregates may reveal sensitive information about very small groups, and external joins can re-identify records. Use the approved privacy and disclosure controls for the intended audience.

Lineage connects a displayed measure to its source fields, transformations, and publication run. Operational lineage helps reproduce a result; business lineage explains its meaning. Both matter when a manager challenges an indicator. A catalogue entry saying “trusted” is weak evidence unless the calculation and checks can be traced.

If an AI assistant explains a dashboard, supply the metric definition, data timestamp, permitted scope, and source evidence. It should not invent causal explanations from a correlation. A rise in service time may

reflect a changed case mix, a source delay, or a real operational problem. Require a human analyst to evaluate those alternatives before recommending intervention.

Operate the analytical service

Measure freshness, completeness, quality-rule failures, correction volume, recovery time, and cost per published dataset or other meaningful unit. CPU alone does not describe whether managers received a trustworthy report. Runbooks should cover source unavailability, schema change, duplicate replay, late corrections, and accidental exposure.

Reprocessing must be intentional and repeatable. Preserve versions of transformation logic and isolate a replay before replacing published data. A change to a definition may require a restatement of historical results, a new metric version, or both. Explain that choice to report users so they do not interpret a modelling change as a sudden business improvement.

Exercises

23.1 A table contains one row per order line and another contains one row per payment event. Explain why joining them by order and summing both amounts can be wrong. Propose a correct reporting approach.

23.2 Using the three-event example, calculate the initial day-1 total, corrected day-1 total, and deduplicated two-day total. State how an explicit correction of e3 from 75 to 70 changes the result.

23.3 Design a publication policy for a report with a 07:00 deadline and late source corrections. Include freshness, completeness, versioning, access, and communication to users.

Key points. An analytical platform is trustworthy when definitions, grain, temporal meaning, lineage, access, and correction rules are explicit. Infrastructure cannot compensate for a metric whose meaning is wrong.

CHAPTER 24

Worked architecture cloud migration and modernisation

Learning objectives

Design operable transition states, preserve a single authority for each write domain, reconcile migrated data, and distinguish traffic rollback from recovery of business state.

Modernisation is a business transition

Orion Modernisation is a fictional programme replacing part of a legacy order and warehouse application. The application runs on virtual machines with a relational database, scheduled file exchanges, and several undocumented consumers. The first release moves customer order status and new order intake to a managed cloud runtime while retaining warehouse execution. The assumed estate contains two million historical orders, a business-approved maintenance window of thirty minutes for the final write cutover, and three dependent reporting teams.

These are local assumptions. They do not imply that a thirty-minute window is suitable for another organisation or that a migration will fit it. The programme must demonstrate timing with representative data and validate the downtime tolerance with affected users. A successful small import does not establish the full migration's duration.

The business goals are supported operation, safer change, and clearer integration ownership. Moving the same defects to a different hosting service does not automatically achieve those goals. Conversely, rewriting every module before obtaining any benefit can increase scope and delay learning. Separate hosting changes, interface changes, data changes, and business-process changes so their risks can be evaluated.

Discover the actual dependency graph

Inventory entry points, scheduled jobs, database readers and writers, file transfers, identities, certificates, network paths, licences, retention obligations, and support procedures. Compare interviews with runtime

observations and logs. A spreadsheet export used once a quarter may be absent from a one-week observation window. A dormant recovery process can still be a critical dependency.

Classify dependencies by ownership and consequence. A read-only reporting extract can often move differently from a warehouse process that writes order state. Shared database access is especially important because it can bypass the application's intended boundary. Before migration, determine whether another job updates the same rows and how that affects the proposed source of truth.

Create a factual current-state view and mark uncertainty. Do not draw the intended future interfaces into the current-state diagram and then assume they already exist. Assign an owner and a verification task to every consequential unknown. The first deliverable may be a dependency inventory and recovery rehearsal rather than a new cloud deployment.

Transition architectures

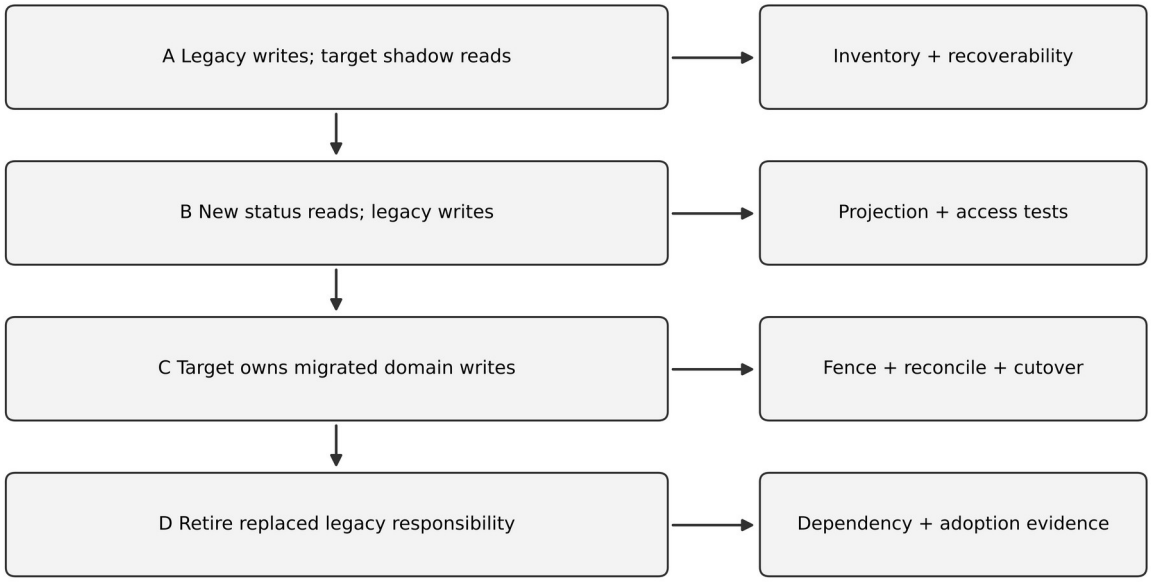


Figure 31 Four operable migration states with one declared write authority per domain

State A keeps all authoritative order writes in the legacy system. The new application receives a read projection for testing, with no customer traffic and no independent authority. State B routes selected read-only status queries to the new application while order writes remain legacy-owned. State C performs a controlled cutover of a defined order cohort or domain after write fencing and final reconciliation. State D retires the replaced legacy responsibility after evidence shows no remaining dependency.

Each state must be supportable on its own. Document routing, write authority, source-to-target replication, identity, monitoring, and recovery. A transition that depends on engineers remembering which database to edit manually is not a sufficiently controlled operating state.

The routing seam can use the strangler approach introduced in Chapter 19. Route by a stable, server-side rule such as migrated customer cohort or capability. Record the rule's version and avoid switching an individual order unpredictably between authorities. A business object with an in-progress transaction needs a consistent owner throughout the relevant lifecycle.

Data mapping as a semantic problem

Profile source records before transformation. Find duplicate business identifiers, nulls, invalid references, overloaded status fields, inconsistent timezones, and historical exceptions. A legacy status of X may mean cancelled in one process and archived in another. A mapping table must express approved business meaning rather than merely converting field names.

Separate lossless representation changes from semantic changes. Converting a date string to a timestamp can introduce a false precision if the source never recorded a time. Splitting one free-text

address into fields may require uncertainty or manual review. Preserve source identifiers and enough migration provenance to explain how a target record was produced.

Declare the reconciliation boundary: source snapshot or change position, included tables or aggregates, transformation version, and excluded records with reasons. Count records, compare key sets, check relationships, and reconcile important totals at a suitable grain. A target with exactly two million rows can still be wrong if one valid row is absent and one other row duplicated.

Copy then catch up without creating two masters

A common staged approach takes an initial consistent source extract and then applies subsequent changes. The exact mechanism depends on the database and export contract. Establish a source position that connects the snapshot with the change stream so records are neither silently missed nor applied twice. The teaching architecture does not assume that an arbitrary file export and an unrelated CDC start time form a correct pair.

While the legacy system remains authoritative, target changes should derive from it through an idempotent, version-aware process. Track deletes and corrections as well as inserts. Monitor lag and failures. A low average lag is insufficient if one partition or table is stalled; inspect the boundaries relevant to the cutover.

Avoid naive application dual writes. If code writes legacy first and cloud second, either step can fail after the other succeeds. Reversing the order only reverses the inconsistency. A durable intent, change capture, or another controlled replication arrangement may address the transition under its contract, but the business still needs one write authority for each domain at a given point.

A concrete cutover argument

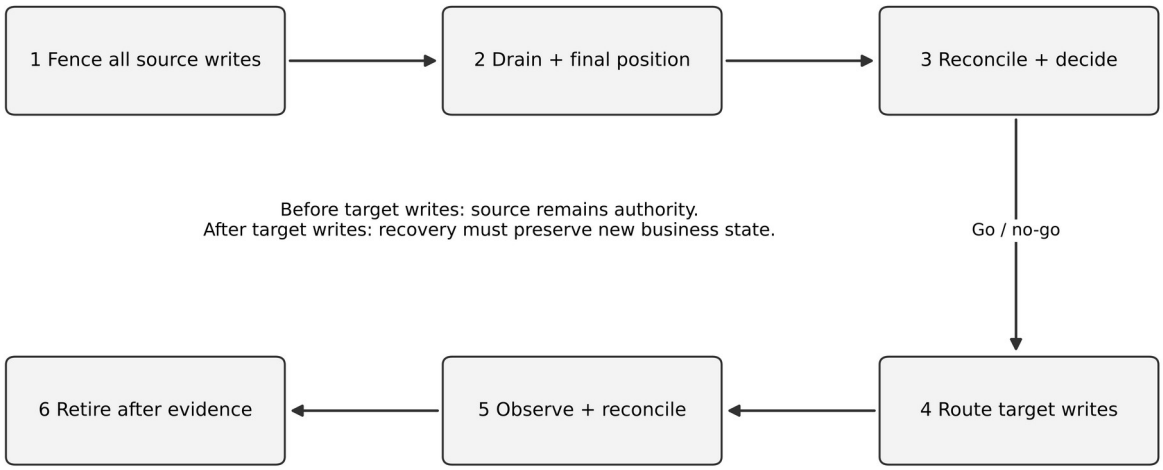


Figure 32 A write fence and reconciliation gate separate reversible routing from new authoritative writes

Before the maintenance window, confirm target capacity, identity, network access, monitoring, backups, source positions, and operator access. Rehearse the procedure and record the time consumed by each step. During the window, fence the relevant legacy writes, wait for in-flight work to reach the agreed boundary, capture the final position, apply remaining changes, and run the release reconciliation checks.

Only after those checks pass should routing permit new authoritative target writes. Keep the legacy system protected from accidental writes. A user-interface maintenance banner is not a write fence if integrations and scheduled jobs can still update the database. The fence must cover all known write paths and be verified.

Suppose a rehearsal measures 4 minutes to establish the fence, 9 to drain and apply remaining changes, 6 for critical reconciliation, 3 for routing and smoke tests, and 2 for the go/no-go decision. The total is 24 minutes, leaving 6 within the fictional 30-minute window. This is arithmetic on hypothetical measurements, not proof of production readiness. Variation, unexpected data, and rollback time must be considered before committing to the window.

Set a latest decision point. If reconciliation is incomplete at that point, restore the pre-cutover operating state while legacy remains authoritative. Do not consume the entire window before deciding whether recovery is still possible. Record who can make the decision and what evidence they need.

Rollback after new writes is a different problem

Before the target accepts authoritative writes, routing back may be straightforward if the source remains intact and current. After new orders exist only in the target, switching the URL back can hide or lose legitimate business work. Recovery must preserve those writes, replay or transform them into the old model if supported, or keep the target authoritative while correcting the defect.

There may be no safe automatic reverse transformation after a semantic change. A new target state might not exist in the legacy schema. In that case, the architecture must explicitly choose a forward-recovery strategy, a compatible transition design, or a constrained period in which new features remain disabled. “Rollback available” is misleading unless it covers data and business state, not just infrastructure.

Distinguish application rollback, routing rollback, database restore, and compensating business actions. Restoring a database backup can remove later transactions; it is not an ordinary undo button. Recovery procedures must state the acceptable loss, how missing transactions are identified, and who authorises the action.

Security and operating readiness

Migration introduces temporary copies, elevated transfer identities, staging environments, and new access paths. Apply data classification and retention to them. A staging export containing production data should not inherit the weaker access policy of a development sandbox. Remove temporary privileges and transfers after the transition, and verify their removal.

Test the new operating model: deployment, incident response, restore, certificate rotation, vendor access, and support escalation. An application that runs correctly under a developer's administrator account is not ready for routine operation. The production service identity and operator identities need the required permissions and appropriate separation.

Cost can rise during coexistence because both platforms, replication, support effort, and temporary tooling remain active. Include the overlap period and retirement work in the business case. A migration does not realise a licence saving until the old service can actually be decommissioned under its contract.

Evidence gates and retirement

Gate A requires a credible inventory and rehearsed recovery. Gate B requires a correct read projection and tested access controls. Gate C requires reconciled data, bounded lag, a verified write fence, and a timed cutover rehearsal. Gate D requires stable operation across an agreed observation period and evidence that old consumers, jobs, and user groups have migrated.

Retirement includes archival or deletion decisions, disabled credentials, stopped scheduled jobs, removed routing, cancelled resources where authorised, updated documentation, and confirmed ownership. Observe for missed dependencies using an approved decommissioning process. Keep records needed for audit and recovery under the agreed lifecycle policy rather than leaving an unsupported live system indefinitely.

The strongest migration dossier explains each intermediate state and its failure response. A target diagram is necessary, but the path between current and target states often contains the greater architectural risk.

Exercises

24.1 Explain why equal source and target row counts do not establish a correct migration. Specify four stronger reconciliation checks.

24.2 Using the hypothetical cutover timings, calculate the total and remaining window. A second rehearsal increases change application by eight minutes. What must change before the cutover can be approved?

24.3 Design a recovery plan for discovering an authorisation defect after the target has accepted 300 new orders. Explain how you preserve those orders while containing the defect.

Key points. Migration architecture is the design of safe intermediate states, data authority, evidence gates, and recovery. A traffic switch is only one part of moving a business service.

CHAPTER 25

Architect practice and the final capstone

Learning objectives

Integrate the course into a defensible design, communicate it under challenge, and assess your readiness for architecture responsibilities.

The architect work cycle

A typical piece of architecture work begins with discovery, moves through requirements and options, produces a decision and implementation direction, then returns to evidence during delivery and operation. The architect should remain involved when assumptions meet real constraints.

In a discovery meeting, ask for examples and measures. In a design session, make responsibilities and tradeoffs explicit. In a delivery review, inspect whether the implementation preserves the important decisions. During an incident, help connect symptoms to dependencies and revisit flawed assumptions.

Use time deliberately. A detailed diagram is premature when the business outcome is unclear. A long options paper is unnecessary when one option fails a mandatory constraint. Focus effort on the uncertainty that could materially change the design.

The capstone brief

Design the first release of Meridian's service management platform using the facts below. These consolidate the initial case; any proposed change must be recorded in your assumption register.

Meridian has 600 customer organisations, 40 service agents, and three business units. It receives 100,000 service requests a year. A widespread equipment incident may produce 20 valid submission requests per second for ten minutes. Attachments can be up to 10 MB. The CRM remains in place for two years and provides a test environment, a service request API, and a lookup by external reference. The vendor has not yet confirmed whether the external reference is unique.

The business wants a durable receipt within one second for 95 percent of valid API submissions at the stated peak, excluding file upload. It proposes a 99.9 percent monthly request-success SLO for this acknowledgement. Normal CRM status propagation should be visible within one minute. A temporary CRM outage of thirty minutes must not silently lose accepted service requests.

Service agents may access only assigned business-unit service requests within their authorised customer-organisation scope. Customers may access their own records under the approved lifecycle. A privacy owner will supply retention and deletion rules before production; the architecture must support configurable, auditable lifecycle actions. Management analytics can refresh each morning.

Six engineers will build and operate the first release with support from shared identity and security teams. The sponsor asks for a vendor-neutral design, one Azure deployment option, one AWS option, and a three year cost comparison. You may use invented planning figures, clearly labelled, but must not present them as vendor prices.

Your deliverables

Produce an architecture dossier that a reviewer can follow from the problem to the release evidence.

1. A two page problem and scope brief, including outcomes, exclusions, stakeholders, and assumptions.
2. A requirement register with at least twelve entries, including quality scenarios and owners.
3. A context view, a container view, a deployment view, a sign-in sequence, and a submission or handoff sequence.
4. A relational data model and a data flow covering attachments, analytics, and lifecycle actions.
5. An API contract for submission, status retrieval, and the CRM handoff, including errors and duplicate handling.
6. Four ADRs covering the initial application style, data ownership, asynchronous integration, and the cloud operating model.
7. A threat model with at least six prioritised threats, controls, owners, and verification evidence.
8. An SLO worksheet, a capacity estimate, a recovery plan, and two operational runbooks.
9. A three year cost model with sensitivity analysis and a vendor evaluation.

10. A phased delivery and migration roadmap with release criteria and unresolved questions.

Use consistent identifiers. A reviewer should be able to trace a critical requirement to a decision, implementation responsibility, test, and operational measure.

Approach the work in stages

First establish the requirements and the boundary of the first release. Resolve or isolate unknowns that could invalidate the design, especially the CRM duplicate behaviour and rate limits. Draw a simple context view before selecting products.

Next design the data and submission contract. Decide exactly when the portal accepts responsibility and how that responsibility survives a failure. Add the integration workflow, identity policy, and data lifecycle.

Then map the neutral responsibilities onto each cloud. Evaluate differences rather than drawing identical product boxes with different logos. Include identity, network, telemetry, backups, and operational access.

Finally challenge the design. Remove the CRM for thirty minutes. Restart a worker at an awkward point. Attempt cross-unit access. Introduce a duplicate submission key with different content. Ask how support determines the outcome of one complaint. Update the design where your evidence is weak.

Extend the portfolio across domains

After completing the Meridian dossier, select one case from Chapters 22–24. Write a five-page comparative review explaining which assumptions and decisions transfer, which do not, and why. Include one additional figure and a failure scenario. For commerce, focus on inventory and payment invariants; for analytics, grain and correction policy; for migration, write authority and recovery. This synthesis is preparation for the design-studio work in Book 2.

Scoring rubric

Dimension	Points	Evidence expected
Requirements and traceability	15	Measurable scenarios and a clear evidence chain
Coherent boundaries and data ownership	15	Consistent views and protected invariants
Integration and failure handling	20	Retry, duplicate, outage, and reconciliation behaviour
Security and privacy design	15	Resource checks, lifecycle, threats, and verification
Operability and delivery	15	SLOs, recovery, telemetry, and safe change
Economics and transition	10	Transparent assumptions and feasible intermediate states
Communication and judgement	10	Clear tradeoffs, uncertainty, and responses to challenge

Aim for at least 75 points with no unresolved critical security or data-loss flaw. The threshold is a course assessment target, not an industry certification. Use Chapter 21 as a worked design baseline and the exercise answers as assessment guidance. Do not copy its diagrams without defending the assumptions.

Present the design

Prepare a ten minute presentation followed by fifteen minutes of questions. Spend the opening minute on the problem and outcome, then explain the main responsibilities and three consequential decisions. Reserve time for residual risks, cost assumptions, and the next evidence needed.

Practise answering: Why this starting architecture? What fails when the CRM fails? How do you prevent cross-tenant access? What does a successful response guarantee? Why do you need this data? What would cause you to choose a different cloud service? Which requirement is most expensive? How would you reverse the decision?

When you do not know, state the uncertainty and the investigation that would resolve it. An architect is expected to distinguish knowledge from assumption, not to improvise a confident answer to every detail.

Preparing for role discussions

Describe your existing background as a base for architecture work: software and data literacy, systems analysis, business processes, and analytical research. Support your claims with the course portfolio and real work you have actually performed. Do not present a fictional capstone as a client deployment.

For an interview case, clarify scope before drawing. Ask about users, load, data sensitivity, integrations, availability, recovery, constraints, and team capability. Make a small set of assumptions visible if answers are unavailable, then design and explain tradeoffs.

Review the position's actual responsibilities. “IT architect” may mean infrastructure, solution integration, software design, or enterprise planning. Use your portfolio to show transferable reasoning and identify where supervised delivery experience would be valuable.

Exercises

25.1 Give a two minute explanation of Meridian's proposed architecture to a service operations manager without using cloud product names.

25.2 Ask a peer to challenge three assumptions in your capstone. Record which decisions changed and which remained justified.

25.3 Write a personal development plan with one technical skill, one delivery experience, and one communication practice to pursue next. Define evidence of progress for each.

Key points. Architectural readiness is demonstrated through clear reasoning, coherent design, evidence, and the ability to improve decisions under challenge. The labs and solutions that follow help you turn the reading into that evidence.

Practical labs

These twelve labs turn the chapters into observable work. Estimated times include writing an explanation of the result; they are not speed targets. Use the companion lab files and fictional data. The Python scripts use only the standard library and do not contact cloud services. Run them with Python 3.10 or newer using python3 followed by the filename. On systems where the interpreter is named python, use that name instead.

The scripts are deliberately small teaching models. In particular, SQLite does not reproduce PostgreSQL's isolation implementation, and the outbox simulation does not implement a production worker lease, real authentication, networking, or retention. Their purpose is to expose a specific mechanism clearly. Keep those limits in your lab notes.

Lab 1 Requirements and the request journey

Estimated time: three hours. Prerequisites: Chapters 1 to 4. Deliverable: a requirement register, a context view, and a request journey.

Start from the capstone brief and the lab1_design_worksheet.md file. Write the desired business outcome in two sentences. Identify at least five stakeholders and one question you would ask each. Separate facts provided in the brief from assumptions you introduce.

Create six requirements. Include a submission behaviour, an access rule, a latency scenario, an availability definition, a recovery expectation, and a data lifecycle capability. Give each an owner and a proposed test. Leave an unresolved policy explicitly unresolved rather than inventing its answer.

Draw a context view with customers, service agents, the portal, CRM, identity provider, ERP, and analytics. Then draw a separate request journey for one submission. Label DNS, the protected connection, API validation, authorisation where applicable, durable storage, the receipt, and the later CRM handoff.

Annotate the point at which each component accepts responsibility. A receipt sent before durable storage needs a different meaning from one sent after commit. Explain the message a customer should see if the CRM is unavailable but the portal has accepted the submission.

Now simulate three failures on paper: DNS does not resolve, the database rejects the transaction, and the CRM is unavailable. For each, state what the user sees, what state exists, and what support needs to inspect. This distinguishes layers and avoids attributing every failure to “the server.”

Expected result. Your diagrams show sign-in and analytics as distinct interactions rather than a single chain through every component. Your requirement register contains measurable conditions and a visible unknown about the CRM's duplicate behaviour.

Reflection. Which requirement could change the initial architecture most significantly? What is the smallest useful investigation to resolve it? Keep your first version and revise it after Lab 4.

Lab 2 Domain rules and access checks

Estimated time: two to three hours. Prerequisites: Chapters 5 and 9, with the access section read early if needed. Deliverable: test results and a revised policy.

Open lab2_domain.py before running it. Identify the Submission value, the structural validation function, and the separate resource-access function. Explain why validate does not establish authorisation.

Run the script. It executes six tests covering a valid submission, length boundaries, service agent assignment, tenant boundaries, customer ownership, and denial for an unknown role. The expected result is six passing tests.

Read the principal object as trusted input supplied by a separate authentication and policy layer. The lab does not authenticate a real person. If a client could set role or tenant_id arbitrarily, the example would not provide a secure boundary.

Add a test for a customer who belongs to the correct tenant but requests another customer's record. Add a test for a service agent who has no assigned_units field. Both should be denied. Then introduce a reviewer role that may read only records explicitly assigned to that reviewer. Extend the fixture and tests before changing the implementation.

Make one intentionally incorrect change in your own copy: remove the tenant comparison. Run the tests and explain which one detects the defect. Restore the check. This is an exercise in connecting a security requirement to evidence.

Expected result. You can explain the difference between validation, trusted identity context, and authorisation. Your new role is denied by default except under the stated assignment rule.

Reflection. Which rule would be difficult to represent using only a broad “service agent” role? Which part of the system should supply the trusted business-unit assignments?

Lab 3 Constraints and transaction rollback

Estimated time: three hours. Prerequisite: Chapter 6. Deliverable: an annotated schema and a transaction explanation.

Run lab3_database.py. It creates an in-memory SQLite database with foreign-key enforcement explicitly enabled. The script inserts two assets in different tenants and one valid service request.

It then attempts a cross-tenant asset relationship, a duplicate submission key, and an invalid status. All three should be rejected by constraints. Read the exact columns in each constraint and explain which invariant it protects.

The final experiment starts a transaction, inserts a request and an outbox row, and raises an error before commit. The expected output reports request=0 and outbox=0 for those inserted identifiers after rollback. This demonstrates local atomicity.

Change the script in your own copy so that the service request commits before the outbox insertion. Inject the same failure and observe the difference. Do not treat a comment saying “atomic” as evidence; the transaction boundary must include both writes.

Add a customer table with a composite tenant and customer key, then enforce that a service request refers to a customer from its own tenant. Use only fictional identifiers. Explain why a foreign key does not decide which authenticated user may read that customer.

Optional concurrency extension. Use two connections to a temporary local database and an explicit transaction to observe SQLite's locking behaviour. Record the result as SQLite behaviour. If you later repeat the experiment in PostgreSQL, document its isolation level and compare results instead of assuming they match.

Expected result. You can identify which errors the database prevents and which still require application policy. You can explain why one transaction protects the service request-plus-outbox invariant.

Reflection. What happens when an external API call is placed between these two database writes? Which locks or resources might be held while the remote system is slow?

Lab 4 The uncertain outcome problem

Estimated time: four hours. Prerequisites: Chapters 8 to 10. Deliverable: a failure sequence, an idempotency contract, and an ADR.

Read lab4_outbox.py. The submit function starts a transaction, checks a submission key within a tenant, verifies owner and payload hash on reuse, and stores the request and outbox intent. The fake CRM is a separate object, representing a system outside that database transaction.

Run the script. A repeated identical local submission returns the same request identifier. Reusing the key with different content is rejected. These behaviours are protected locally.

The worker then calls the fake CRM and is interrupted after remote success but before recording completion. On retry, the non-idempotent provider creates two remote records. With the provider's idempotent option enabled, the same sequence creates one remote record.

Expected results are:

```
Provider idempotent=False: local requests=1, remote records=2
Provider idempotent=True: local requests=1, remote records=1
```

Draw the failure sequence. Mark the local transaction boundary, remote side effect, crash point, and retry. Explain why the outbox prevents a lost local intent but does not alone prevent a duplicate remote side effect.

Add a scenario in which the remote call fails before creating anything. Add another in which the caller repeats the local key under a different owner. Describe the safe expected behaviour before editing the code.

The fake idempotent provider uses a deterministic in-memory lookup in a single-threaded simulation. This is not proof that a real provider enforces unique external references under concurrency. For Meridian, that capability remains a vendor question and a release condition if the requirement demands strict uniqueness.

Write an ADR that states the exact guarantee you can currently make and the residual risk. Include a reconciliation plan for looking up uncertain outcomes and a manual review process when the provider cannot guarantee uniqueness.

Expected result. Your explanation distinguishes local atomicity, message redelivery, and external effect idempotency. It does not claim end to end exactly-once behaviour from the existence of a queue.

Reflection. Would writing “completed” before the remote call solve the problem? Demonstrate the failure that would then become possible.

Lab 5 Capacity backlog and recovery

Estimated time: three hours. Prerequisites: Chapters 12 and 13. Deliverable: a capacity worksheet and a recovery hypothesis.

Run lab5_capacity.py. The model uses constant arrival and service rates. It is a simple fluid approximation, not a simulation of task-duration distributions or a cloud performance forecast.

The first case receives 30 messages per second and processes 24 for fifteen minutes. The backlog grows to 5,400. The second assumes a constant 20 per second during a thirty minute processing outage, producing 36,000 messages. The third restores processing at 35 per second while arrivals remain 20, leaving a net drain of 15 per second and a forty minute drain time.

These outage assumptions are deliberately more sustained than the capstone's ten minute service incident peak. Do not silently reuse the 36,000 result as Meridian's forecast. Use a time-varying workload for the capstone and explain your chosen baseline outside the peak.

Create three scenarios: normal business demand, a service incident peak, and an outage followed by recovery. State units for every input. Calculate whether the backlog clears before the business deadline.

Then add a vendor rate limit. If the CRM permits fewer requests than the worker can produce, cap the effective service rate. Explain why adding workers beyond that point can increase throttling without improving throughput.

Estimate storage for the queued work separately from attachment storage. At an average one kilobyte of handoff metadata, 36,000 items represent about 36 MB in decimal units before database, index, replication, and storage overhead. At ten megabytes per attachment, the same number of maximum-size attachments represents 360 GB before overhead. The actual workload may be much smaller.

Expected result. Your worksheet shows backlog growth and drain, identifies the downstream limit, and labels averages, peaks, and assumptions.

Reflection. Which measurements would you collect from a real workload before sizing production? How would skewed task durations change the risk?

Lab 6 Threats privacy and evidence

Estimated time: four hours. Prerequisite: Chapter 15. Deliverable: a threat model and lifecycle register.

Use lab6_threat_worksheet.md and your latest diagrams. Mark public entry points, request and data boundaries, administrative access, external vendors, and the analytics route.

Identify at least six credible threats. Include cross-tenant access, credential compromise, malicious attachments, replayed events, excessive resource consumption, and disclosure through logs. For each, name the asset, necessary conditions, likely consequence, control, owner, and evidence.

Choose one threat and trace its controls across layers. For cross-tenant access, this might include trusted principal scope, resource-level checks, composite data relationships, restricted exports, and negative tests. Explain which controls are independent and which rely on the same assumption.

Create a lifecycle register for the operational database, object storage, analytics, search or vector indexes if present, logs, and backups. Identify the policy owner and mechanism for each. Use “policy required before release” for an unknown retention rule rather than choosing an arbitrary number of months.

Review an AI summarisation extension as an optional exercise. Treat the document content as untrusted input, enforce retrieval permissions, define quality measures, and require a human to review the output before consequential use. Keep the extension outside the initial release unless the requirements justify it.

Expected result. The model connects controls to concrete threats and tests. It distinguishes an implemented technical capability from an approved legal or business policy.

Reflection. Which residual risk needs explicit business acceptance? Which evidence would be invalidated by changing the identity provider or attachment processor?

Lab 7 SLOs telemetry and incident response

Estimated time: three hours. Prerequisite: Chapter 17. Deliverable: an SLO report and a short runbook.

Run lab7_slo.py against service_observations.csv. The file contains 1,000 eligible requests and 20 ineligible requests. Two eligible requests lack a durable receipt. The success indicator is therefore 998/1,000, or 99.8 percent.

At a 99.9 percent target, the allowed bad count for this sample is one. The observed count is two. This short sample fails that target, but it is not a complete monthly SLO report. Avoid treating a tiny sample as a stable forecast.

The script uses the nearest-rank percentile definition. The p95 latency is 400 milliseconds and p99 is 1,600 milliseconds. It computes latency for all eligible rows separately from the durable-receipt success indicator. If you combine success and latency into one “good event” definition, recompute the result using that definition explicitly.

Add columns for the downstream CRM outcome, acceptance timestamp, and first CRM-visible timestamp. Choose an observation cutoff. Make some rows pending even when the receipt is durable. For completed handoffs, calculate visibility time minus acceptance time. For still-pending rows, report current age and whether the target is already breached; do not omit them silently or invent a completion time. Produce a separate propagation-delay indicator rather than classifying every pending CRM result as a failed receipt.

Write an alert for a sustained increase in unsuccessful submissions and another for old handoff work. Include the first diagnostic step, the responsible role, and conditions for escalation. Do not page someone for an observation that has no actionable response.

Expected result. Your report states the eligibility rules, denominator, observation period, percentile method, and distinction between acceptance and synchronisation.

Reflection. What happens if monitoring misses an hour? How should a service report communicate the resulting uncertainty?

Lab 8 Cost sensitivity and a decision review

Estimated time: three hours. Prerequisites: Chapters 18 and 19. Deliverable: a cost comparison and a decision memo.

Run lab8_cost.py. It reproduces the fictional three year comparison and sensitivity scenarios. The baseline custom total is 427,000 euros and purchased total is 387,000 euros. A purchased support-effort increase to 45,000 per year produces 432,000 euros.

Create a third option that retains the current portal and improves integration only. Estimate its one-time and recurring components. Clearly label your invented values and explain what real evidence would replace them.

Add non-financial requirements: access control, data export, support, integration semantics, and user experience. Separate mandatory requirements from weighted preferences. An option that fails a mandatory requirement should not win solely through low cost.

Write a one page decision memo describing the recommended option, assumptions that drive the result, two alternatives, major risks, and a review trigger. Include a phased implementation so the organisation can learn before committing to the complete change.

Expected result. The memo shows arithmetic, uncertainty, and business consequences without presenting hypothetical inputs as a vendor quote.

Reflection. Which uncertainty is worth paying to investigate? Which decision can be postponed without damaging the first release?

Lab 9 Order saga and uncertain payment

Estimated time: four hours. Prerequisite: Chapter 22. Deliverable: transition table, test results, and a failure argument.

Run lab9_saga.py. The deterministic model reserves one unit, records an uncertain payment, reconciles it as authorised, and confirms the order. A duplicate confirmation does not consume more stock. A second scenario receives a definitive rejection, releases stock once, and tolerates a repeated release. A third records failed compensation as work still requiring attention. Expected output is three scenario summaries followed by a successful assertion message.

The model stores state in memory and is not crash-durable. It deliberately separates business transitions from provider implementation. Read the assertions as invariants within this tiny model, not proof of real payment correctness. In a production design, transitions and durable intent need suitable transactional storage, authentication, provider idempotency, and reconciliation.

Add a reservation-expiry transition. Specify what happens if authorisation arrives after expiry. Test both event orders and explain the customer message for each. Do not make a timeout equivalent to rejection. Draw a sequence showing where a real process could crash between provider success and local recording.

Expected result. You can explain the difference between rejected, unknown, authorised, and compensated outcomes, and identify the persistence boundary missing from the teaching model.

Lab 10 Late events and corrected analytical totals

Estimated time: three hours. Prerequisite: Chapter 23. Deliverable: analytical contract and versioned result explanation.

Run lab10_analytics.py. The script deduplicates by event identifier and keeps the highest explicit source version. Amounts use integer minor units. It distinguishes event day from ingestion day. Expected initial day-1 total is 100.00; corrected day 1 is 150.00; the two-day total before a correction is 225.00 and after revising e3 is 220.00.

The source contract here provides a monotonic version per event. Explain why a real source without that contract needs another conflict rule. Add two records with the same event identifier and version but conflicting content; the script's integrity check must reject that ambiguity. Do not let ingestion order silently select a winner.

Add a deletion marker with an explicit business meaning, then update the materialisation and tests. Decide whether the deletion removes a fact, reverses a measure, or triggers a privacy action. Document the policy rather than choosing whichever implementation is easiest. Keep publication timestamps and correction reasons in your report.

Expected result. Your result accounts for duplicates, late arrival, and versioned correction without adding the same business effect twice.

Lab 11 Migration reconciliation and the false comfort of counts

Estimated time: three hours. Prerequisite: Chapter 24. Deliverable: discrepancy report and cutover gate.

Run lab11_migration.py. Source and target each contain three rows. The target has one missing key, one extra key, and one changed amount. The report identifies all three discrepancies even though the counts match. Repair the fictional target and rerun the comparison to obtain a clean result.

Extend the records with a child relationship and a source change position. Define how your gate detects an orphan child or a target that has not applied the final change. Add an intentionally transformed field and document the approved comparison rule; a raw byte comparison is not sufficient when transformations are legitimate.

Write a cutover rule that combines reconciliation, write fencing, access testing, and timed recovery. Explain why the script alone cannot approve a migration. It compares small in-memory records and does not prove snapshot consistency, production completeness, or business acceptance.

Expected result. Your evidence distinguishes row counts, identity coverage, value correctness, and transition readiness.

Lab 12 Architecture evaluation workshop

Estimated time: five hours. Prerequisites: Chapters 20–24. Deliverable: evaluation report using lab12_evaluation_worksheet.md.

Choose one case and freeze a version of its proposed architecture. Identify three stakeholder concerns, five quality scenarios, and two credible options. Separate mandatory constraints from preferences. For each high-priority scenario, trace the responsible components and failure paths, identify a sensitivity point, and propose the smallest useful experiment.

Conduct the review yourself in two passes or ask a peer to challenge it. In the second pass, introduce a change: a lower vendor quota, an unavailable region, a late source correction, or a failed compensation. Record which decisions remain justified and which need revision. Keep the original findings rather than rewriting history to make the first design look correct.

Conclude with proceed, proceed with conditions, investigate, or reject. Tie each condition to an owner and observable closure evidence. Use the Chapter 25 rubric to review the argument. This is a teaching workshop inspired by structured evaluation; it is not independent certification or a full formal ATAM engagement.

Expected result. A reviewer can distinguish checked facts, untested assumptions, contextual judgement, and unresolved risks in your conclusion.

Exercise answers and assessment guidance

Attempt each exercise before reading its answer. Numerical answers use the assumptions in the question. Design answers give one defensible approach and criteria for judging alternatives. A different design can be valid when it satisfies the requirements and explains its costs and failure behaviour.

Chapter 1

1.1 Database selection is architectural when it establishes important data guarantees, platform dependencies, team skills, operating costs, or migration constraints for the system. For a small component inside an already approved PostgreSQL platform, selecting the standard instance may be a routine implementation choice. The distinction depends on consequence and reversibility, not the product name alone.

1.2 A suitable brief is: Meridian wants to reduce duplicate records and manual effort between service request submission and work order creation. Service agents currently copy information between systems and have difficulty tracing failed handoffs. The existing CRM must remain for two years. Six engineers will build and support the first release. The CRM's behaviour when an external reference is repeated has not yet been confirmed. This describes the problem without choosing a portal framework or messaging product.

1.3 Ask which outcomes “modern” should provide: independent releases, higher scale, easier integration, or better maintainability. Compare a modular monolith and a small service design against those outcomes, team capability, and measured workload. Propose a prototype or evidence review for the important uncertainty. A strong answer neither dismisses the stakeholder nor treats the requested pattern as an established requirement.

Chapter 2

2.1 One acceptable scenario is: when an authenticated service agent from business unit A requests an attachment belonging only to business unit B, the service denies access and records a safe audit event; automated tests cover the download, preview, and export routes. Include the policy owner and the expected response behaviour. “Uses encryption” would describe one possible control, not the complete access requirement.

2.2 The unavailable fraction is 1 - 0.999 = 0.001. Multiplying 43,200 minutes by 0.001 gives 43.2 minutes. A request-based SLO weights requests rather than elapsed time. Ten minutes during a service incident peak could affect far more requests than ten minutes overnight, producing a different success ratio. The calculation does not define maintenance exclusions or monitoring rules; these must be agreed.

2.3 Ask what counts as accepted data, which failure scenarios are covered, whether zero loss includes a regional disaster, and what starts and ends the recovery clock. Clarify which functions must return within five minutes, whether dependencies and human steps are included, and what budget and staffing are available. Distinguish RPO from RTO and validate the requirement with recovery tests. A synchronous local write alone does not prove zero loss under every disaster.

Chapter 3

3.1 A container view shows the operational database as a data-store responsibility used by the API and worker. A deployment view shows its instances or managed service placement, zone or region, access path, replicas, backups, and relevant failover dependencies. The same logical database can have several physical or managed deployment elements.

3.2 A valid view contains seven elements: customer, service agent, portal, identity provider, CRM, ERP, and analytics. Label customer-to-portal as submitting and tracking service requests; service agent-to-portal as reviewing assigned service requests; portal-to-identity provider as sign-in interaction; portal-to-CRM as submission handoff; CRM-to-portal as status; portal-to-ERP as approved work order creation; and portal-to-analytics as controlled reporting data. Equivalent responsibilities are acceptable if the ownership is clear.

3.3 The chain suggests that every user request waits for analytics and that identity, storage, and reporting are serial stages of one operation. Use a submission sequence for the immediate request and a separate data flow for later reporting. Show sign-in and resource access as distinct interactions. The improvement is semantic accuracy, not simply a more attractive arrangement.

Chapter 4

4.1 More application instances may open more database connections and increase contention for an already constrained resource. They can also increase queued work and memory use. Measure connection-pool wait time, active queries, database locks, query latency, and request concurrency. Fix the limiting query or capacity, bound concurrency, or introduce appropriate backpressure before assuming more API replicas will help.

4.2 One valid plan is 10.20.1.0/24 for edge-related infrastructure, 10.20.2.0/24 for application workloads, and 10.20.3.0/24 for data services. All are within 10.20.0.0/16 and do not overlap. A subnet assignment does not grant or deny access; routing, firewall rules, service policies, and identity controls govern communication. Account for the chosen platform's address reservations.

4.3 A resolver or client may still hold a cached DNS answer, or an existing connection may remain open to the old endpoint. Some clients also implement their own caching behaviour. Keep both endpoints compatible during a transition, reduce TTL in advance when appropriate, monitor traffic, and retire the old endpoint only after the planned overlap. Lowering TTL after caches are populated does not retroactively change their stored values.

Chapter 5

5.1 Length, required-field, and deterministic formatting checks can be pure functions. Permission checks need trusted principal and policy context. Asset-eligibility checks may need current database state.

Persistence, queue publication, and vendor calls are effects. A useful design can supply trusted state to a pure policy function, but must still ensure that state is current enough and that the eventual write preserves the invariant.

5.2 Test a new key creating one result, an identical retry returning the same result, conflicting content under the same key being rejected, and two concurrent or uncertain-outcome attempts not creating two local service requests. A lost-response test should commit the first action, hide its response, and then retry. Testing two sequential dictionary lookups alone does not establish concurrency safety. State the database or atomic mechanism on which the guarantee depends.

5.3 Shared code can reduce duplication for stable, genuinely common rules. A broad common module can also couple unrelated domains, force coordinated releases, and make ownership unclear. Prefer small shared technical utilities or carefully governed domain concepts. If two contexts use the same word with different meanings, separate their models and translate at the boundary.

Chapter 6

6.1 An asset-only foreign key can prove that v2 exists without proving that it belongs to the request's tenant. A composite relationship on tenant and asset requires the pair to exist, preventing a request in t1 from referencing t2's asset through that relation. It still does not authenticate a user or enforce every access rule.

6.2 Under PostgreSQL Read Committed, transaction A updates version 7 to 8 and commits. Transaction B then evaluates or re-evaluates the condition against the current row and updates zero rows. Its caller should reread the state if authorised and decide whether the intended action is still valid. Under Repeatable Read or Serializable, a transaction whose earlier snapshot saw version 7 can instead receive a serialization failure and must abort and retry the transaction. Handle both outcomes according to the chosen database and isolation level. Blindly substituting version 8 can overwrite a meaningful intervening decision. [15]

6.3 First, tune the query and indexes, which may reduce cost but cannot eliminate all contention. Second, use a reporting replica, accepting replication lag and the cost of another database while verifying isolation from the primary workload. Third, create a separate analytical pipeline and model, improving workload separation but adding transformation, freshness, and reconciliation work. Select based on scale, freshness, and ownership rather than assuming a warehouse is always necessary.

Chapter 7

7.1 Use one row per request status interval or one row per status-transition event, with a defined method for deriving intervals. Include request identity, status, event or effective time, and enough sequence information to handle duplicates and late events. One customer can have several service requests and each service request several statuses, so one row per customer loses the required temporal detail.

7.2 Check required keys, allowed transitions or statuses, timestamp plausibility, and reconciliation of source and target key sets. A uniqueness rule on event identifiers or a grouped count by the declared grain can reveal duplicates even if total row counts match. Compare batch identifiers and missing-key sets as well as totals. Quarantine invalid records and make the reporting consequence visible.

7.3 Ask what quality purpose requires retention, what personal data is included, who can access it, whether a vendor processes it, and how correction and deletion work. A bounded alternative stores a minimised evaluation dataset under an approved retention rule, with access controls and de-identification where appropriate, rather than retaining every raw prompt. Separate operational diagnostics from model evaluation. The lawful policy must be approved for the actual processing.

Chapter 8

8.1 A response can remain valid JSON with the same field type while changing meaning. For example, status accepted might originally mean durable local receipt and later mean CRM synchronisation. Clients that trigger actions on that value would behave differently even though schema validation succeeds. Document state semantics and treat consequential meaning changes as contract changes.

8.2 Define the key's namespace, such as tenant plus submission key, and bind its stored result to the authenticated owner and relevant payload hash. Persist it atomically with the service request, enforce uniqueness under concurrency, return the original result for an authorised identical retry, and reject conflicting reuse. Specify retention and post-expiry behaviour, protect result retrieval, and provide reconciliation for delayed retries. The key is not an access token.

8.3 Unbounded responses can exhaust database, memory, network, and client resources. A broad export can also expose excessive personal data or bypass ordinary access scoping. Use explicit authorisation, bounded pagination, a stable order, limited fields, and rate controls. For large exports, consider an asynchronous job with a controlled download, expiry, and audit trail. Make snapshot or consistency semantics clear.

Chapter 9

9.1 A valid signature only establishes integrity and origin under the verified key. The ID token's audience and purpose are authentication information for the client, not necessarily access to this API. The API must accept a credential intended for its resource and validate issuer, audience, lifetime, and permissions according to the access-token contract.

9.2 Permit download only when the principal is authenticated, the attachment belongs to a request within the principal's allowed tenant and resource scope, the requested action is permitted for that role or ownership, and the attachment's lifecycle state allows release. Check the actual stored relationship rather than trusting client-supplied tenant or object metadata. Authorise any temporary download URL and keep its scope and lifetime appropriate.

9.3 Existing access tokens may remain valid until expiry, the API may cache permissions, or the directory change may not yet propagate. A browser logout is not always token revocation. Choose token lifetimes, session validation, introspection or revocation mechanisms, and cache invalidation to meet an explicit revocation target. Immediate revocation may increase dependency and availability requirements.

Chapter 10

10.1 The broker may redeliver because it did not receive an acknowledgement, causing the local update to repeat. Store the event identifier in an inbox table and commit it atomically with the consumer's local state change. A duplicate then detects that completed local action. If an external side effect occurs outside the transaction, it still requires its own duplicate or reconciliation design.

10.2 Retry transient connection failures or throttling within a bounded policy that respects the vendor's guidance. Apply backoff and jitter, preserve the stable operation identifier, and record safe error context. Stop automatic retries for invalid data or permission failures. Escalate work whose age threatens a business deadline or whose outcome remains uncertain. Replay only through an authorised, auditable process that handles duplicates.

10.3 Determine the exact transaction boundary, failure assumptions, producer and consumer settings, retention, and behaviour during replay. Ask whether the guarantee includes the ERP system's write, which is usually outside the broker. Verify whether the ERP API enforces a stable idempotency key under concurrency and how uncertain responses are reconciled. Do not convert a broker-local guarantee into a claim about every external business effect.

Chapter 11

11.1 A modular monolith can enforce clear interfaces and data ownership while using local calls and one controlled deployment. Poorly designed microservices can share mutable data, require synchronous chains, and demand coordinated releases. Deployment separation alone does not reduce semantic or operational coupling. Compare how a change propagates through each design.

11.2 A defensible starting decision is a modular application with a separate worker process and managed data services. It fits a small team, local submission transactions, and moderate initial demand. Reject a large microservice estate because independent ownership or scale is not yet demonstrated, and reject direct CRM-dependent submission because the outage requirement is important. Use a measurable review trigger: revisit the worker boundary if a representative peak repeatedly fails the one minute propagation target despite query tuning and the permitted worker capacity. First establish whether the limit is local processing or the vendor API; extracting a service does not solve a vendor rate limit.

11.3 A table-per-service design often splits one business invariant across remote calls and makes ordinary transactions distributed. It can increase network traffic, contract count, and coordinated change. Examine which capability owns the data, which rules must commit together, and which parts truly change independently. Tables are storage structures, not automatic service boundaries.

Chapter 12

12.1 Net growth is 30 - 24 = 6 messages per second. Fifteen minutes is 900 seconds. The additional backlog is 6 × 900 = 5,400 messages, assuming no initial backlog, constant rates, and no dropped or expired work.

12.2 Possible causes include lock contention affecting a minority of requests, a slow downstream dependency, and queueing during bursts. Inspect per-request traces, database lock and connection metrics, dependency latency distributions, and time-aligned queue or concurrency data. Check whether one tenant, query type, or deployment version dominates the tail. An average alone cannot distinguish these mechanisms.

12.3 In an isolated environment, arrange for the CRM to accept the operation, then terminate the worker before it records completion or acknowledges the task. Restart and retry with the same stable identifier. Verify the number of remote records and the final local state, inspect the audit trail, and test reconciliation. Repeat with a provider that lacks idempotency to document the residual risk instead of concealing it.

Chapter 13

13.1 Meridian still owns appropriate data use, schema and queries, application access, identities and permissions, chosen configuration, capacity decisions, recovery objectives, and validation of backups and restores. The exact division of patching, replication, maintenance, and availability responsibilities depends on the purchased service. Read that contract rather than assuming “managed” means no customer work.

13.2 One mapping uses Azure Container Apps, Azure Database for PostgreSQL, Blob Storage, and Service Bus; another uses ECS with Fargate, RDS for PostgreSQL, S3, and SQS. Add the appropriate identity, secrets, telemetry, and network services. Investigate message ordering and duplicate semantics, deployment networking and identity, database failover and backup options, regional support, quotas, and pricing. Equivalent responsibilities do not imply interchangeable configuration.

13.3 Ask which dependency is concerning, what exit time is required, whether there is a resilience need across providers, and whether the team can operate both. Compare export and migration readiness with active operation on two clouds. Include duplicated skills, identity, observability, testing, data transfer, and consistency costs. A tested exit plan may address the real concern with less complexity.

Chapter 14

14.1 Restarting all instances can discard useful state, create connection storms, repeatedly run initialisation, and consume resources while the database remains unavailable. Liveness should detect a process that needs replacement, while readiness and degraded behaviour should reflect dependency limitations. Apply bounded retries and restore capacity gradually.

14.2 Place two API replicas behind a controlled entry point and run a worker group that reads durable pending work. Store request records and outbox state in durable database storage and attachments in durable object storage. Treat local files, process memory, and instance identity as replaceable unless explicitly designed otherwise. Bound worker concurrency by downstream capacity.

14.3 Determine why replacement is proposed, whether it destroys data, what dependencies change, and whether a safer in-place or staged migration exists. Verify tested backups, restore time, keys, maintenance window, data synchronisation, application compatibility, rollback or roll-forward, and authorised review. Inspect the exact plan and affected resources. A tool's successful plan generation is not approval to destroy a database.

Chapter 15

15.1 At the public-to-API boundary, a caller may alter an object identifier to read another customer's record, or upload a file that exhausts resources. Controls include resource-level access checks and bounded upload validation with appropriate processing isolation. Evidence includes denied cross-user access tests and controlled oversize-file tests. State the residual risks and the role responsible for them.

15.2 The application normally decrypts or receives decrypted data through its authorised access path. An overprivileged account can therefore request and disclose data even when storage media are encrypted. Reduce privileges, apply resource checks, minimise returned data, and monitor consequential access. Encryption at rest addresses a different part of the threat model.

15.3 Start from the approved policy and identity match. Create a tracked lifecycle request, locate relevant records and copies, apply authorised deletion or restriction in each system, and reconcile completion. Handle analytical aggregates, search indexes, retained logs, backups, and holds according to their policy

and technical mechanism. Prevent a restored backup or delayed event from silently reintroducing deleted data. Keep appropriate evidence without retaining unnecessary personal content.

Chapter 16

16.1 Review the change, validate the contract and code, build an identified artefact, run risk-based tests, scan and review dependencies, deploy to staging, and verify critical journeys and recovery. Promote the same artefact with controlled configuration, then observe production behaviour. Evidence includes test results tied to the build, change approval where required, deployment identity, configuration version, and post-release checks.

16.2 The old code may still query the removed column and fail after rollback. Use expand, migrate, and contract: add compatible structures, deploy transitional code, move data and consumers, then remove the old structure only when no supported version requires it. Plan data recovery separately. Some failures are safer to roll forward than to reverse blindly.

16.3 Check whether smaller or more frequent deployments are actually safer, whether tests represent production, whether configuration bypasses the pipeline, and whether one class of change causes most failures. Examine recovery time and business impact as well as counts. Improving one metric while increasing rework is not enough evidence of a better delivery system.

Chapter 17

17.1 The permitted bad fraction is 1 - 0.9995 = 0.0005. Multiplying by 800,000 gives 400 allowed bad requests. With 120 failures, 120/400 = 0.30, so 30 percent of the allowance is used. This assumes the defined period and eligibility rules are complete.

17.2 A metric can report oldest pending handoff age to show deadline risk. A structured log event can record a handoff attempt, event identifier, safe error category, and result for investigation. A trace span can measure the vendor-call duration within a workflow. Keep personal content out of routine telemetry and avoid using unbounded identifiers as metric labels.

17.3 A suitable alert states that the oldest uncompleted critical handoff has exceeded a threshold chosen below the business deadline for a sustained period, identifies the integration on-call role, and links to backlog and provider-health views. The first response is to confirm impact and distinguish provider outage, throttling, invalid data, and worker failure. Escalation and safe replay steps belong in the runbook.

Chapter 18

18.1 The purchased total becomes 45,000 + 55,000 + 3 × (54,000 + 45,000) + 35,000 = 432,000 euros. It is 5,000 euros above the baseline custom total of 427,000. This comparison changes only the specified input and retains all other fictional assumptions.

18.2 Include application runtime, database compute and storage, attachment storage, data transfer, messaging operations, identity or user licences where applicable, telemetry retention, backup storage, engineering support, and vendor integration costs. Storage, transfer, message operations, and telemetry are often volume-sensitive; staffing and licences may have thresholds. State whether the model includes build effort, tax, discounting, and exit.

18.3 Present the export limitation as an exit and continuity risk, with the business consequences and options: negotiate an adequate export, add a tested integration or escrow arrangement if appropriate, accept a documented risk, or select another vendor. If usable export is mandatory, exclude the option until it meets the requirement. Do not bury a mandatory failure inside a weighted average.

Chapter 19

19.1 Map service operations to the CRM and portal, work-order fulfilment to ERP, and reporting to analytics. If three tools support service management, investigate whether they serve distinct entities, policies, or specialist processes before consolidation. Show owners and dependencies as well as application names.

19.2 First introduce reliable handoffs behind the existing form and pass duplicate and outage tests. Next launch the portal for one business unit and demonstrate access control, support readiness, and reconciliation. Finally expand and retire the old form after adoption, data completeness, and dependency checks. Each transition must be operable on its own and have a recovery plan.

19.3 A successful import may still have incorrect meaning, broken relationships, missing attachments, failed integrations, changed permissions, untrained users, or incomplete ongoing change capture. Validate

business outcomes, reconciliation, support, access, adoption, and retirement. Migration completion is an operational and business state, not merely a data-load exit code.

Chapter 20

20.1 A managed database provides capabilities under a particular service and configuration contract, but the complete recovery path also needs detection, decision, access, client reconnection, application capacity, data validation, and dependent systems. Require an agreed failure scenario and recovery boundary, the chosen database configuration, a timed restore or failover exercise using representative data, and evidence that the business function is usable within five minutes. Record manual steps and variation. A provider feature list is insufficient.

20.2 Failures are 26/20,000 = 0.0013, or 0.13 percent. Observed success is 99.87 percent. It is below 99.9 percent for that test interval. To compare with a monthly production SLO, establish matching eligibility, success definition, observation point, maintenance and dependency rules, representative workload, measurement completeness, and the actual reporting period. A short experiment can challenge a capacity assumption without being a full monthly SLO report.

20.3 One evaluation compares direct CRM creation with local durable receipt plus a worker. Mandatory constraints include tenant isolation and no successful receipt without durable responsibility. Scenarios cover peak load, thirty-minute vendor outage, and timeout after remote commit. Worker concurrency is a sensitivity point; faster recovery versus vendor throttling is a possible tradeoff. Approve only an option that meets mandatory constraints and has sufficient evidence for high-priority scenarios. Mark vendor uniqueness unresolved and require a test or an accepted manual-resolution policy. Other options are valid if the same evidence discipline is applied.

Chapter 21

21.1 The context view locates ownership but cannot establish database atomicity. The container view locates runtime responsibilities but cannot establish code-level enforcement. The component view explains code responsibility but cannot establish physical fault isolation. The data model shows relationships but cannot authenticate a principal. The sequence shows ordering and response meaning but cannot prove deployed capacity. The deployment view shows placement and dependencies but cannot demonstrate measured recovery. A strong answer uses the same request and handoff identifiers throughout and names the missing evidence for each view.

21.2 Net drain is 30 - 12 = 18 handoffs per second. 18,000/18 = 1,000 seconds, or 16 minutes 40 seconds. The ideal model assumes constant successful throughput and arrivals, with no extra retries. Throttling, variable service duration, failed attempts, worker restarts, increasing arrivals, or other consumers sharing the quota can extend recovery. If arrivals reach or exceed effective capacity, the backlog does not drain under this model.

21.3 Accept a durable report in an entitlement-pending state without promising covered service, approval, or fulfilment. Retain the verified customer and asset relationship needed for safe intake, and prevent approval or work-order creation until entitlement is established. Show pending status and give operations an owner and deadline. If the sponsor intends “submit” to mean an immediately approved covered service, the requirements conflict during loss of authoritative entitlement data unless an approved offline rule or sufficiently trusted cached entitlement can satisfy the policy. Escalate that business decision explicitly.

Chapter 22

22.1 A timeout reports lack of an observed response, not the provider's financial outcome. The provider may have authorised payment. Retain the original operation identifier, order and amount/currency context, provider reference if available, and attempt history. Query or repeat only under verified idempotency and lookup semantics. Keep the order pending or under resolution until the outcome is established; do not start a new charge with a new identity merely because the first response was lost.

22.2 Atomically decrement available stock only if at least one unit remains, and insert a unique reservation in the same local transaction. One customer commits a reservation; the other observes insufficient availability or a retryable concurrency outcome and re-evaluates. Both must not receive a successful reservation for the same unit. With external inventory, the order system needs the inventory service's reservation identity, atomicity, expiry, idempotency, and confirmation contract. Its own database transaction cannot make the remote reservation atomic.

22.3 If expiry has committed, a late authorisation moves the order to resolution rather than directly to confirmed. Attempt a fresh reservation only if policy permits; if it succeeds, confirm under the new reservation identity. Otherwise request a provider-supported void or compensation. Record pending

compensation until its outcome is verified, tolerate duplicate callbacks, and alert an owner when resolution exceeds its deadline. Tell the customer that confirmation is pending or that the attempt is being reversed according to the verified outcome. Reward designs that preserve stock and payment accountability even when compensation fails.

Chapter 23

23.1 If an order has three lines and two payment events, a direct join can create six rows. Each line value and each payment value is repeated. Aggregate each fact independently to the intended order or reporting grain before joining, and distinguish authorisation, capture, refund, and other event meanings. Validate currency and business scope. A one-row-per-order bridge can help relate aggregates, but it cannot fix an undefined measure.

23.2 Initial day 1 is 100; corrected day 1 is 150; deduplicated two-day total is 225. A verified newer version of e3 changing 75 to 70 gives a total of 220, not 295 from treating the correction as an additional event. Preserve the source version and publication revision so the change can be explained. The repeated e1 contributes once.

23.3 A defensible policy publishes an explicitly timestamped snapshot at 07:00, states known source completeness, retains the last valid snapshot when critical checks fail, and marks provisional intervals. It supports an agreed late-data correction window plus an exception route for older corrections, with versioned republication. Access rules apply to exports and derived stores. Notify users when a correction changes a consequential decision measure. The exact window is a business choice justified by observed lateness and reporting needs, not a number copied from the example.

Chapter 24

24.1 Equal counts can hide missing and duplicated rows or incorrect values. Compare distinct key sets, relationship integrity, values or canonical hashes for corresponding records, and business totals by meaningful groups. Also verify source change position, deletes, exceptions, and transformation version. Explain how the comparison handles intentionally transformed or excluded records. Counts are useful but not sufficient.

24.2 The baseline total is 4 + 9 + 6 + 3 + 2 = 24 minutes, leaving 6 minutes. Eight additional minutes produces 32 minutes, exceeding the 30-minute window by 2 minutes before allowing for variation or recovery. Reduce and remeasure the work, enlarge the approved window, change the transition method, or postpone. Do not remove critical reconciliation solely to make the arithmetic fit. Set a latest go/no-go point that preserves a feasible recovery path.

24.3 Contain the affected access route or permissions immediately through an authorised incident procedure while preserving evidence. Keep the 300 target orders durable and accountable. Determine whether the defect can be corrected or the application rolled back against the current compatible schema. Route back only with a verified plan to transfer or retain the new orders and prevent duplicate fulfilment. Assess disclosure scope, reconcile state, and follow the organisation's communication process. A bare DNS rollback does not resolve the data or security problem.

Chapter 25

25.1 A strong explanation says that the portal gives customers one place to submit and receive a durable receipt, while a monitored background process updates the existing CRM. Service agents retain access only to their assigned work, and approved service cases can initiate an ERP work order. Reporting uses a separate controlled data flow. Explain that the design tolerates temporary vendor outages and makes unresolved work visible.

25.2 Judge the exercise by whether the challenges were specific and the response used evidence. A changed assumption should lead to an updated requirement, ADR, model, or test where necessary. Keeping a decision is valid when its rationale still holds. Merely recording that a peer “approved the diagram” is weak evidence of review.

25.3 One suitable plan is to implement and explain a duplicate-safe integration, participate in a supervised release and incident review, and practise ten minute design presentations with feedback. Define evidence such as a tested prototype, a documented delivery responsibility, and a revised presentation that answers previously unresolved questions. Choose opportunities relevant to the intended architect role rather than collecting unrelated tools.

Second edition advanced exercises

6.4 The invariant concerns the pair, while each boolean constraint concerns one value. Both false values are individually valid. A common lock must be acquired before either transaction reads the roster and must be respected by every writer. It serialises the relevant decision. A serializable transaction can reject an unsafe schedule; the application must retry the whole read-decide-write unit and expose exhaustion. Neither solution automatically coordinates an external effect. Award credit for a correctly guarded alternative representation with all writers covered.

9.4 Tenant equality is necessary under this case policy but insufficient. The requester must be the submitter or possess a valid organisation-wide delegation; scan state and the requested action also matter. Test T1/P requesting T1/Q's object without delegation and expect denial, then grant the permitted delegation and expect access. Add a T2 control case. A policy that permits the first case is tenant-only; a policy that denies every case is also wrong.

12.4 With no intervening write, real-time precedence forces write-1 before the read; that sequential register state cannot return 0. If the read begins before the write completes, the intervals overlap and the read may be placed before the write, subject to the rest of the history. This does not allow arbitrarily stale reads once a later call begins after confirmed completion. State assumptions about a single register, initial 0 and no hidden writes.

12.5 A majority of five is three; none of the groups has it. Progress requires an appropriate communicating majority and the algorithm's other conditions. Restoring a connection does not remove voting, persistence or log rules. The payment provider needs a separate effect identity, lookup or reconciliation contract, retry and retention limits, and handling of unknown outcomes. Local agreement about intent does not prove the payment happened exactly once.

15.4 The proxy needs authoritative or sufficiently fresh decisions on every new request, a specified policy-update path and an explicit rule for active streams. Direct capability delivery needs enforceable expiry or revocation covering the ten-second bound; a longer-lived irrevocable bearer URL fails it. Include propagation and tolerated clock uncertainty in the bound. Test removal just after issuance, direct storage access and every alternate delivery route. A design may reject the requirement as infeasible under an identified provider contract; claiming it without evidence is not acceptable.

15.5 Best-effort events cannot guarantee immediate revocation when delivery can be delayed. With a maximum decision age of sixty seconds, the residual grant window can approach that age, plus declared timing tolerance; independently renewed cached grants would invalidate that bound. Deny restricted new access after expiry if fresh policy is unavailable. For a tighter bound, shorten the permitted age or require an online decision and accept its availability implications. Specify the operation and what happens to already active access.

Glossary and final review

Use the glossary for retrieval practice. Cover each definition and explain the term using Meridian before checking it.

ADR. Architecture decision record documenting a significant choice, its context, and consequences.

API. An agreed interface through which software requests or receives behaviour.

Authorisation. A decision about whether a principal may perform an action on a resource under stated conditions.

Authentication. Establishing evidence about a principal's identity.

Availability. Whether the defined service is usable under a stated measurement and failure model.

Backpressure. A mechanism that limits or slows producers when downstream capacity is constrained.

Bounded context. A boundary within which a domain model and its terminology have consistent meaning.

Capability. An ability the business needs, independent of one particular implementation.

CDC. Change data capture, a mechanism for obtaining changes from a source system.

Circuit breaker. A mechanism that temporarily stops calls to a failing dependency and later permits controlled probes.

Cohesion. The degree to which responsibilities within a module belong together.

Consistency. A term with several contexts; database constraint preservation and distributed read/write guarantees must be distinguished.

Container. A packaged runtime process environment; in C4, a separately runnable application or data store.

Contract. The interface's syntax, semantics, guarantees, limits, and change expectations.

Coupling. Dependencies that cause one part of a system to be affected by another.

Data lineage. Information about a value's sources, transformations, and processing history.

Data owner. The role responsible for decisions about the business meaning and approved use of data.

Data steward. A role supporting definitions, quality, and governance of data.

Dead letter queue. A place for messages that could not be processed under the normal policy and require controlled handling.

Durability. Preservation of committed data under the storage system's stated guarantees and failure assumptions.

Elasticity. Adjusting provisioned resources in response to demand within service and configuration limits.

Event. A record that a meaningful occurrence has happened.

Idempotency. Repetition having the same intended effect as a single operation within a defined scope.

Infrastructure as code. Versioned definitions and automated management of infrastructure.

Invariant. A condition that must remain true across allowed state changes.

Latency. Elapsed time for an operation, with its measurement boundaries specified.

Least privilege. Giving a principal only the authority required for its task.

Observability. The ability to understand system behaviour through available outputs and instrumentation.

Outbox. A durable outgoing-work record committed with a local business change.

Partition. A division of data or a network connectivity failure, depending on context; state which meaning is intended.

Principal. A person, service, device, or other entity to which identity and authority are assigned.

RPO. Recovery point objective, the acceptable data-loss window measured back in time.

RTO. Recovery time objective, the acceptable time to restore a defined service.

Saga. A coordinated process across local transactions with explicit handling or compensation for failure.

SLI. Service-level indicator, the defined measurement of a service property.

SLO. Service-level objective, a target for a service-level indicator.

SLA. Service-level agreement, including agreed conditions and consequences.

System of record. The authoritative source for a defined set of business facts.

Technical debt. Future change or operating cost created by design or implementation choices.

Tenant. A logically separated customer, organisation, or administrative scope in a shared system.

Throughput. Completed work per unit time.

Trust boundary. A boundary across which assumptions about identity, authority, or data trust change.

Unit economics. Costs or value expressed per meaningful unit of business output.

Final retrieval practice

Explain why a timeout does not prove that an operation failed. Explain why a private subnet does not establish authorisation. Explain why backups differ from replicas. Explain why two diagrams can both be correct while answering different questions. Explain why a vendor-neutral design can still choose a managed service.

Then choose one decision from your capstone and defend it without naming any product. State the requirement, the alternatives, the evidence, the tradeoff, and the condition that would change your choice. If you can do that clearly, you are practising the central work of an architect.

Research study guide

Use these readings to practise transferring an argument into a design decision. The summaries in the chapters are deliberately brief; read the relevant original passages and write your own critique. A source's age does not invalidate a durable theoretical contribution, while an old implementation detail may no longer describe a current product. Keep those questions separate.

Foundational research and university work

Reading	Focus in the source	Your analytical task
Parnas on modular decomposition [13]	Alternative decomposition criteria	Identify a changeable decision in your design and test whether its module hides it
Codd on the relational model [17]	Data independence and logical representation	Separate a business dependency from a physical access path in your schema
Saltzer Reed and Clark on end-to-end arguments [11]	Placement of correctness responsibility	Explain why a transport acknowledgement is insufficient for a business receipt
Lamport on distributed ordering [33]	Partial order and logical clocks	Draw concurrent events and state what a timestamp can and cannot establish
Gilbert and Lynch on CAP [30]	Formal definitions and partition assumptions	Describe one operation that must refuse progress to preserve its consistency contract
Ongaro and Ousterhout on Raft [34]	Election replication and safety conditions	Explain why majority replication is part of an algorithm rather than a complete design slogan
Dean and Barroso on tail latency [35]	Outliers in large distributed requests	Challenge the independence assumption in the course fan-out calculation
Fielding dissertation Chapter 5 [21]	REST constraints and their consequences	Audit one API against the constraints without equating REST with JSON
Garcia-Molina and Salem conference paper [55]	Long-lived transactions and compensation	Contrast the original setting with the commerce workflow and list the extra external-system assumptions
Kazman Klein and Clements technical report [54]	Architecture scenarios risks and tradeoffs	Evaluate one important decision and identify evidence still missing

The journal and conference articles are scholarly publications. Fielding is a university dissertation; Sagas is cited here as a conference paper, and ATAM as a university technical report. These categories have different review histories. The source list does not imply that this course itself has undergone independent academic peer review.

Standards and operational references

Read HTTP, TLS, OAuth, and OpenID Connect references for the specific protocol property discussed in the chapter. Read NIST publications for the scope and intent of their guidance. Read database documentation for the chosen version's actual constraints and isolation behaviour. Treat Azure, AWS, orchestration, and observability documentation as implementation evidence. Treat C4, ADR, domain-driven design, and related practitioner methods as useful design and communication approaches, not controlled experiments proving universal superiority.

A source critique you can assess

For each selected reading, write 400–600 words covering: the question addressed; the method or argument; the assumptions; one conclusion you can support; one limitation or boundary; and a concrete implication for your case. Cite the exact source and relevant section. Distinguish what the author

establishes from your inference. A strong critique does not need to reject the source; it shows where the result can responsibly be used.

Finish by proposing an observation that could challenge your contextual recommendation. For example, a module boundary justified by expected vendor change might be less valuable if the actual dominant change is cross-cutting policy. The foundational argument and your application of it should remain distinguishable.

References

References are numbered in order of first citation. Online sources were accessed on 4 October 2026. Product documentation should be checked again before implementation.

[1] Amazon Web Services, "The pillars of the framework," Accessed: Oct. 4, 2026. [Online]. Available: https://docs.aws.amazon.com/wellarchitected/latest/framework/the-pillars-of-the-framework.html

[2] Microsoft, "Microsoft Azure Well-Architected Framework pillars," Accessed: Oct. 4, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/well-architected/pillars

[3] M. Nygard, "Documenting architecture decisions," Nov. 15, 2011. Accessed: Oct. 4, 2026. [Online]. Available: https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

[4] C. Jones, J. Wilkes, and N. R. Murphy, with C. Smith, "Service level objectives," in Site Reliability Engineering, B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds. O'Reilly Media, 2016, ch. 4. Accessed: Oct. 4, 2026. [Online]. Available: https://sre.google/sre-book/service-level-objectives/

[5] S. Brown, "The C4 model for visualising software architecture," Accessed: Oct. 4, 2026. [Online]. Available: https://c4model.com/

[6] P. Mockapetris, "Domain names concepts and facilities," RFC 1034, Nov. 1987. Accessed: Oct. 4, 2026. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc1034

[7] S. Thomson, C. Huitema, V. Ksinant, and M. Souissi, "DNS extensions to support IP version 6," RFC 3596, Oct. 2003. Accessed: Oct. 4, 2026. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc3596

[8] E. Rescorla, "The Transport Layer Security protocol version 1.3," RFC 9846, Jul. 2026. Accessed: Oct. 4, 2026. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc9846

[9] R. Fielding, M. Nottingham, and J. Reschke, Eds., "HTTP semantics," RFC 9110, Jun. 2022. Accessed: Oct. 4, 2026. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110.html

[10] E. Rosen and Y. Rekhter, "BGP/MPLS IP virtual private networks," RFC 4364, Feb. 2006. Accessed: Oct. 4, 2026. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc4364

[11] J. H. Saltzer, D. P. Reed, and D. D. Clark, "End-to-end arguments in system design," ACM Transactions on Computer Systems, vol. 2, no. 4, pp. 277-288, Nov. 1984. Accessed: Oct. 4, 2026. [Online]. Available: https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf

[12] S. Chacon and B. Straub, "Getting started about version control," in Pro Git, 2nd ed. Accessed: Oct. 4, 2026. [Online]. Available: https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

[13] D. L. Parnas, "On the criteria to be used in decomposing systems into modules," Communications of the ACM, vol. 15, no. 12, pp. 1053-1058, Dec. 1972, doi: 10.1145/361598.361623. Accessed: Oct. 4, 2026. [Online]. Available: https://www.cs.lafayette.edu/~gexia/cs301/resources/parnas.html

[14] PostgreSQL Global Development Group, "Constraints," PostgreSQL 18 documentation. Accessed: Oct. 4, 2026. [Online]. Available: https://www.postgresql.org/docs/18/ddl-constraints.html

[15] PostgreSQL Global Development Group, "Transaction isolation," PostgreSQL 18 documentation. Accessed: Oct. 4, 2026. [Online]. Available: https://www.postgresql.org/docs/18/transaction-iso.html

[16] PostgreSQL Global Development Group, "Using EXPLAIN," PostgreSQL 18 documentation. Accessed: Oct. 4, 2026. [Online]. Available: https://www.postgresql.org/docs/18/using-explain.html

[17] E. F. Codd, "A relational model of data for large shared data banks," Communications of the ACM, vol. 13, no. 6, pp. 377-387, Jun. 1970, doi: 10.1145/362384.362685. Accessed: Oct. 4, 2026. [Online]. Available: https://www.engineering.upenn.edu/~zives/03f/cis550/codd.pdf

[18] R. Kimball, "Fact tables," Nov. 5, 2008. Accessed: Oct. 4, 2026. [Online]. Available: https://www.kimballgroup.com/2008/11/fact-tables/

[19] E. Tabassi, "Artificial Intelligence Risk Management Framework AI RMF 1.0," NIST AI 100-1, Jan. 2023. Accessed: Oct. 4, 2026. [Online]. Available: https://doi.org/10.6028/NIST.AI.100-1

[20] OpenAPI Initiative, "OpenAPI Specification," version 3.2.1, Sep. 10, 2026. Accessed: Oct. 4, 2026. [Online]. Available: https://spec.openapis.org/oas/v3.2.1.html

[21] R. T. Fielding, "Representational State Transfer REST," in Architectural Styles and the Design of Network-based Software Architectures, Ph.D. dissertation, University of California, Irvine, 2000, ch. 5. Accessed: Oct. 4, 2026. [Online]. Available: https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

[22] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, "OpenID Connect Core 1.0 incorporating errata set 2," Accessed: Oct. 4, 2026. [Online]. Available: https://openid.net/specs/openid-connect-core-1_0.html

[23] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett, "Best current practice for OAuth 2.0 security," RFC 9700, Jan. 2025. Accessed: Oct. 4, 2026. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9700

[24] OWASP Foundation, "OWASP Top 10 API security risks 2023," 2023. Accessed: Oct. 4, 2026. [Online]. Available: https://api-security.owasp.org/editions/2023/en/0x11-t10/

[25] RabbitMQ, "Consumer acknowledgements and publisher confirms," Accessed: Oct. 4, 2026. [Online]. Available: https://www.rabbitmq.com/docs/confirms

[26] Apache Software Foundation, "Design," Apache Kafka 4.1 documentation. Accessed: Oct. 4, 2026. [Online]. Available: https://kafka.apache.org/41/design/design/

[27] Amazon Web Services, "Transactional outbox pattern," AWS Prescriptive Guidance. Accessed: Oct. 4, 2026. [Online]. Available: https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

[28] E. Evans, "Domain-driven design reference definitions and pattern summaries," Domain Language, Mar. 2015. Accessed: Oct. 4, 2026. [Online]. Available: https://www.domainlanguage.com/wp-content/uploads/2016/05/DDD_Reference_2015-03.pdf

[29] A. Cockburn, "Hexagonal architecture," 2005. Accessed: Oct. 4, 2026. [Online]. Available: https://alistair.cockburn.us/hexagonal-architecture

[30] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent available partition tolerant web services," ACM SIGACT News, vol. 33, no. 2, pp. 51-59, Jun. 2002, doi: 10.1145/564585.564601. Accessed: Oct. 4, 2026. [Online]. Available: https://www.cs.princeton.edu/courses/archive/spr22/cos418/papers/cap.pdf

[31] J. D. C. Little, "A proof for the queuing formula L = lambda W," Operations Research, vol. 9, no. 3, pp. 383-387, 1961, doi: 10.1287/opre.9.3.383. Accessed: Oct. 4, 2026. [Online]. Available: https://pubsonline.informs.org/doi/abs/10.1287/opre.9.3.383

[32] Microsoft, "Circuit breaker pattern," Accessed: Oct. 4, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker

[33] L. Lamport, "Time clocks and the ordering of events in a distributed system," Communications of the ACM, vol. 21, no. 7, pp. 558-565, Jul. 1978. Accessed: Oct. 4, 2026. [Online]. Available: https://lamport.azurewebsites.net/pubs/time-clocks.pdf

[34] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in Proceedings of the 2014 USENIX Annual Technical Conference, 2014, pp. 305-319. Accessed: Oct. 4, 2026. [Online]. Available: https://www.usenix.org/system/files/conference/atc14/atc14-paper-ongaro.pdf

[35] J. Dean and L. A. Barroso, "The tail at scale," Communications of the ACM, vol. 56, no. 2, pp. 74-80, Feb. 2013, doi: 10.1145/2408776.2408794. Accessed: Oct. 4, 2026. [Online]. Available: https://barroso.org/publications/TheTailAtScale.pdf

[36] M. P. Herlihy and J. M. Wing, "Linearizability: A correctness condition for concurrent objects," ACM Transactions on Programming Languages and Systems, vol. 12, no. 3, pp. 463-492, Jul. 1990. Accessed: Oct. 4, 2026. [Online]. Available: https://www.cs.cmu.edu/~wing/publications/HerlihyWing90.pdf

[37] P. Mell and T. Grance, "The NIST definition of cloud computing," NIST SP 800-145, Sep. 2011. Accessed: Oct. 4, 2026. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-145

[38] Microsoft, "Azure for AWS professionals," Accessed: Oct. 4, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/aws-professional/

[39] Docker, "What is a container," Accessed: Oct. 4, 2026. [Online]. Available: https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/

[40] The Kubernetes Authors, "Overview," Accessed: Oct. 4, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/overview/

[41] HashiCorp, "State," Terraform documentation. Accessed: Oct. 4, 2026. [Online]. Available: https://developer.hashicorp.com/terraform/language/state

[42] V. Drake, "Threat modeling," OWASP Foundation. Accessed: Oct. 4, 2026. [Online]. Available: https://community.owasp.org/Threat_Modeling

[43] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," Proceedings of the IEEE, vol. 63, no. 9, pp. 1278-1308, Sep. 1975, doi: 10.1109/PROC.1975.9939. Accessed: Oct. 4, 2026. [Online]. Available: https://www.cs.virginia.edu/~evans/cs551/saltzer/

[44] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," NIST SP 800-207, Aug. 2020. Accessed: Oct. 4, 2026. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-207

[45] European Parliament and Council of the European Union, "Regulation EU 2016/679," Official Journal of the European Union, L 119, pp. 1-88, May 4, 2016. Accessed: Oct. 4, 2026. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2016/679

[46] OWASP Foundation, "Application Security Verification Standard," version 5.0.0. Accessed: Oct. 4, 2026. [Online]. Available: https://owasp.org/projects/asvs

[47] N. Harvey, "DORA's software delivery performance metrics," Accessed: Oct. 4, 2026. [Online]. Available: https://dora.dev/guides/dora-metrics/

[48] National Institute of Standards and Technology, "Secure Software Development Framework SSDF version 1.1: Recommendations for mitigating the risk of software vulnerabilities," NIST SP 800-218, Feb. 2022, doi: 10.6028/NIST.SP.800-218. Accessed: Oct. 4, 2026. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/218/final

[49] OpenTelemetry Authors, "Signals," Accessed: Oct. 4, 2026. [Online]. Available: https://opentelemetry.io/docs/concepts/signals/

[50] R. Ewaschuk, "Monitoring distributed systems," in Site Reliability Engineering, B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds. O'Reilly Media, 2016, ch. 6. Accessed: Oct. 4, 2026. [Online]. Available: https://sre.google/sre-book/monitoring-distributed-systems/

[51] FinOps Foundation, "Unit economics," Accessed: Oct. 4, 2026. [Online]. Available: https://www.finops.org/framework/capabilities/unit-economics/

[52] The Open Group, "The TOGAF Standard," 10th ed. Accessed: Oct. 4, 2026. [Online]. Available: https://www.opengroup.org/togaf

[53] Microsoft, "Strangler fig pattern," Accessed: Oct. 4, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/patterns/strangler-fig

[54] R. Kazman, M. Klein, and P. Clements, "ATAM: Method for architecture evaluation," Carnegie Mellon University, Software Engineering Institute, CMU/SEI-2000-TR-004, Aug. 2000. Accessed: Oct. 4, 2026. [Online]. Available: https://www.sei.cmu.edu/documents/629/2000_005_001_13706.pdf

[55] H. Garcia-Molina and K. Salem, "Sagas," in Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data, 1987, pp. 249-259. Accessed: Oct. 4, 2026. [Online]. Available: https://www.cs.cornell.edu/andru/cs711/2002fa/reading/sagas.pdf